

Interfacing FPGA external Components

Today's Agenda

Intended topics for today's session

- Interfacing FPGA external Components

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Interfacing FPGA external Components

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Interfacing FPGA external Components

Today's Agenda

Intended topics for today's session

- Nonideal clocking effects
- Single vs. Multiple Clock Domains
- Metastability and Synchronization Techniques

Interfacing FPGA external Components

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

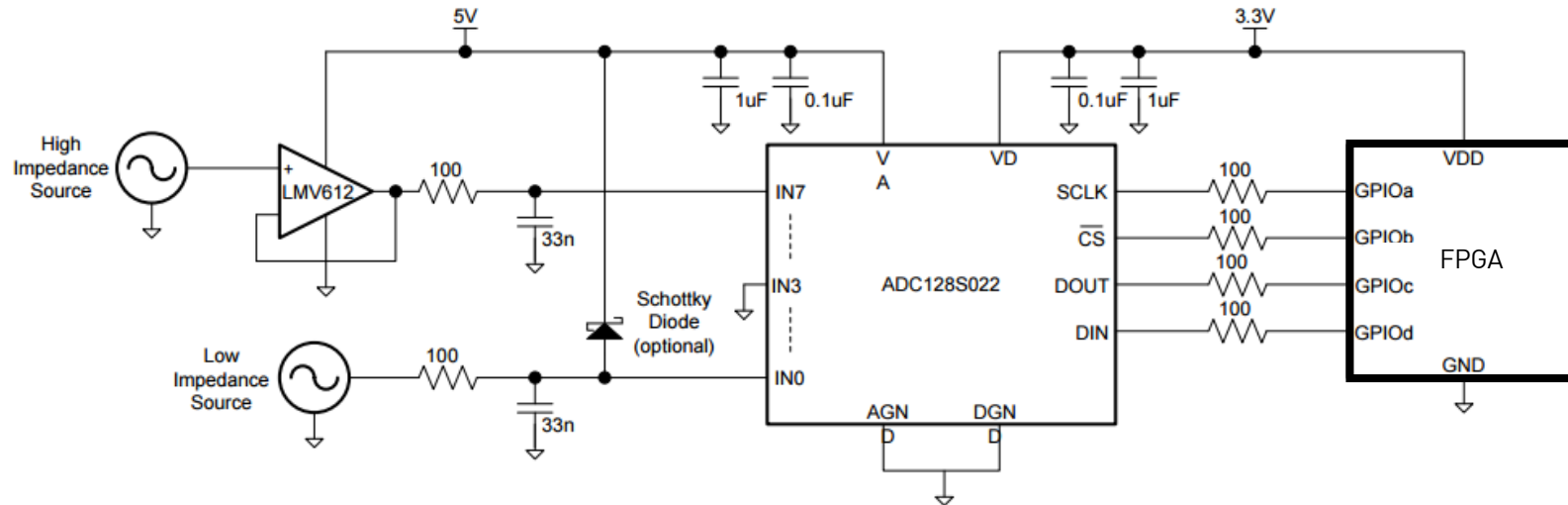
fbeit

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Use Case

Texas Instruments ADC128S022

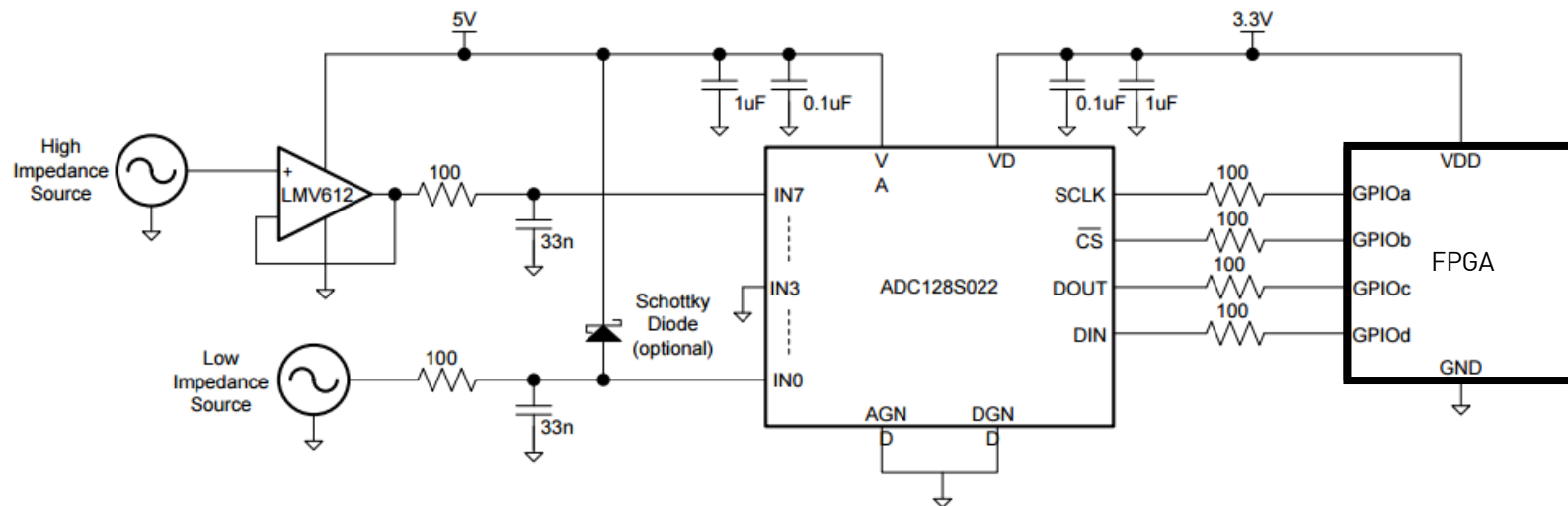
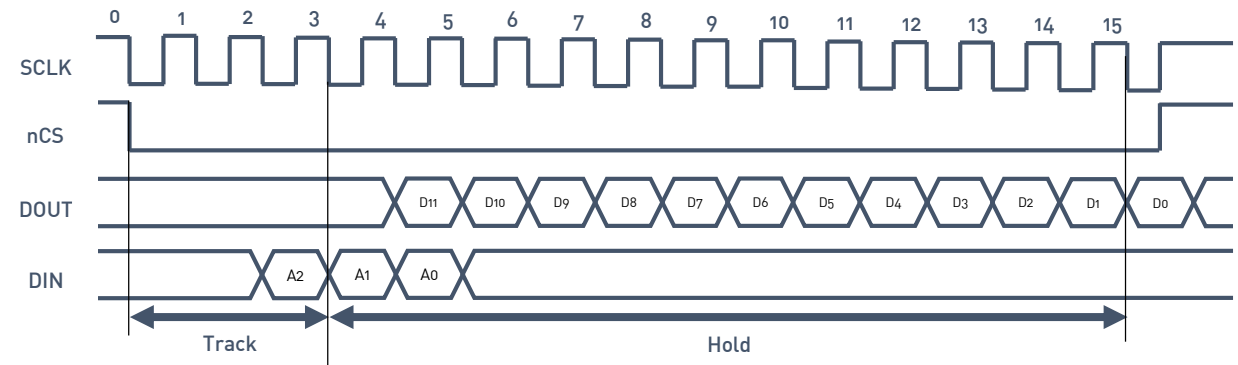


Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Use Case

Texas Instruments ADC128S022

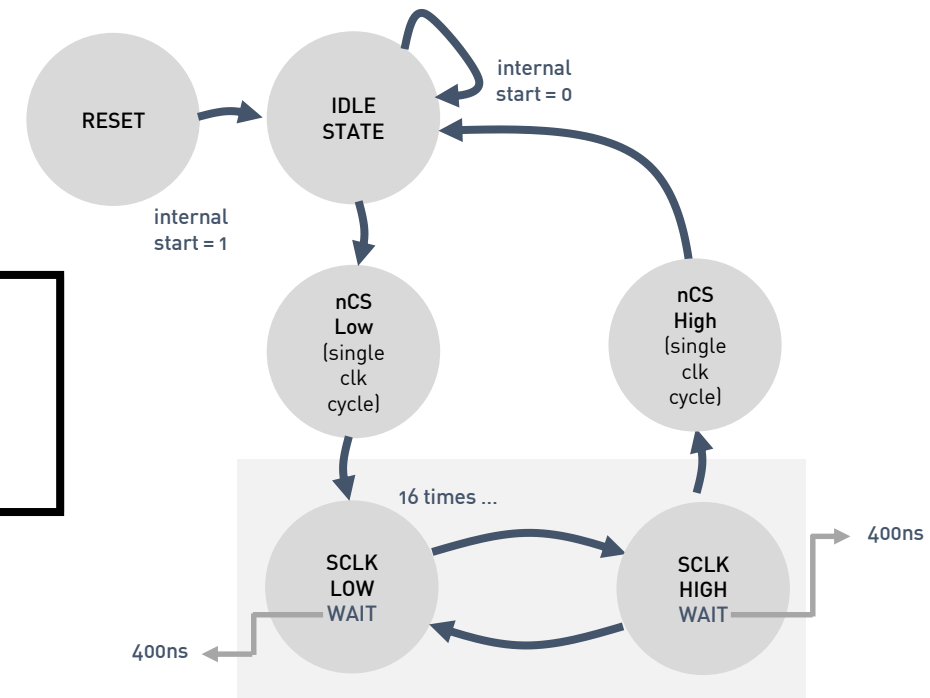
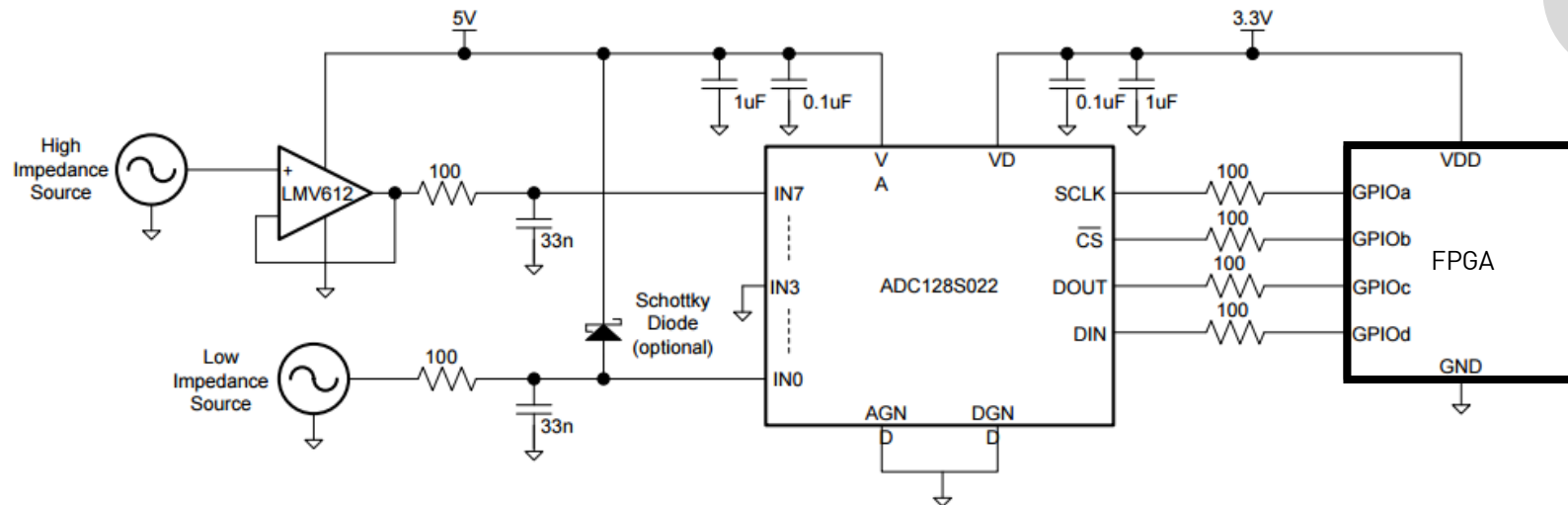
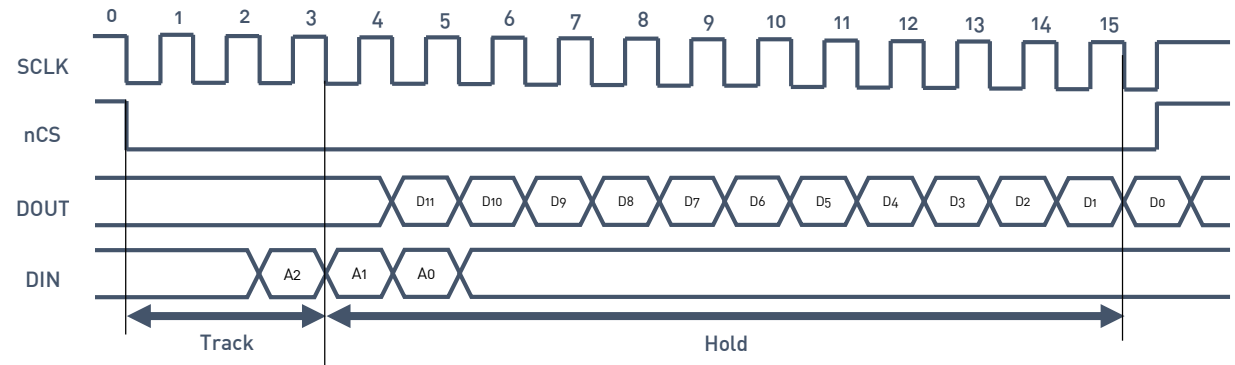


Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Use Case

Texas Instruments ADC128S022



Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

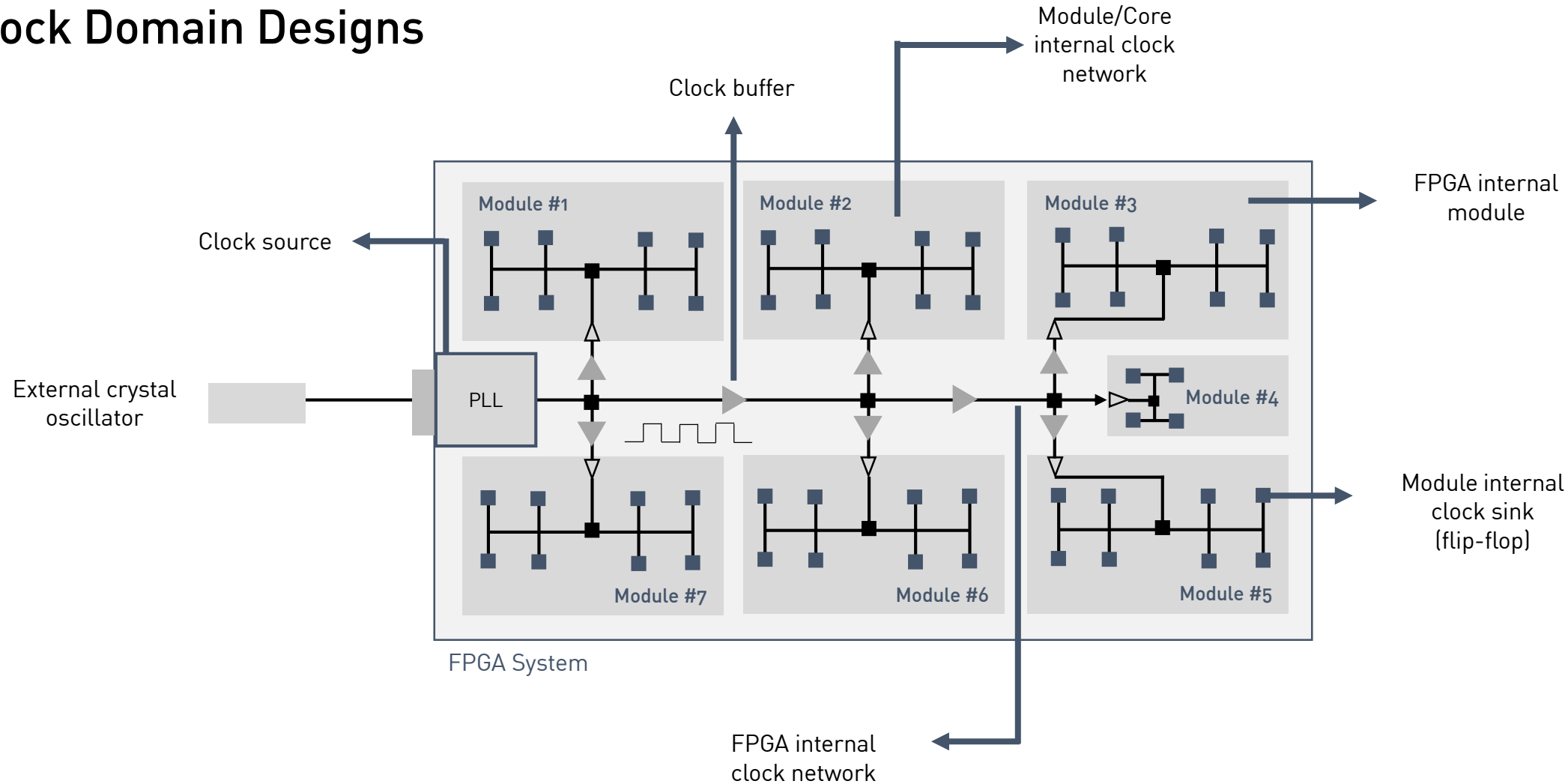
So far, we only considered single clock domain designs

FPGA based System Design

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

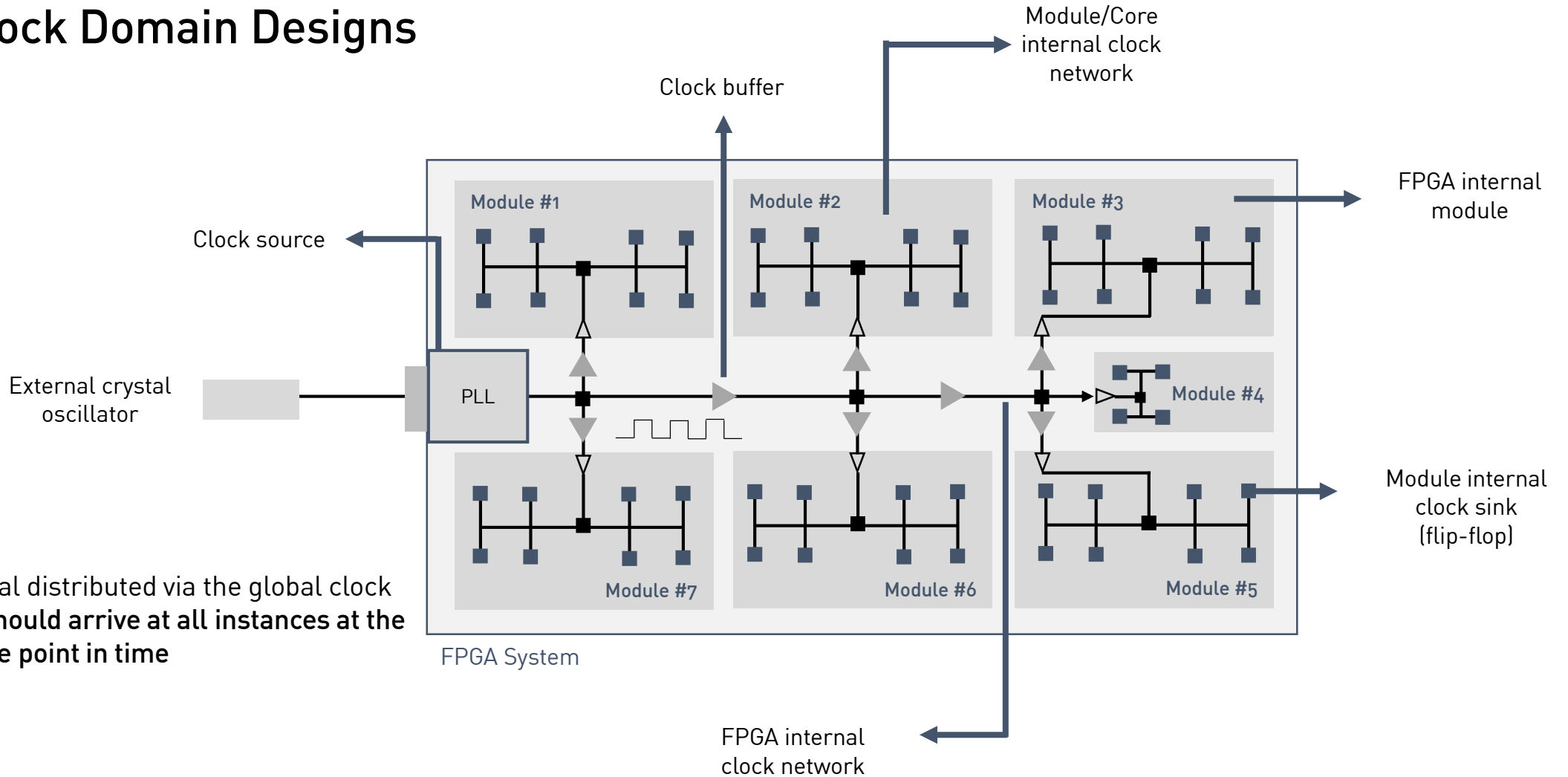
Single Clock Domain Designs



Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Single Clock Domain Designs



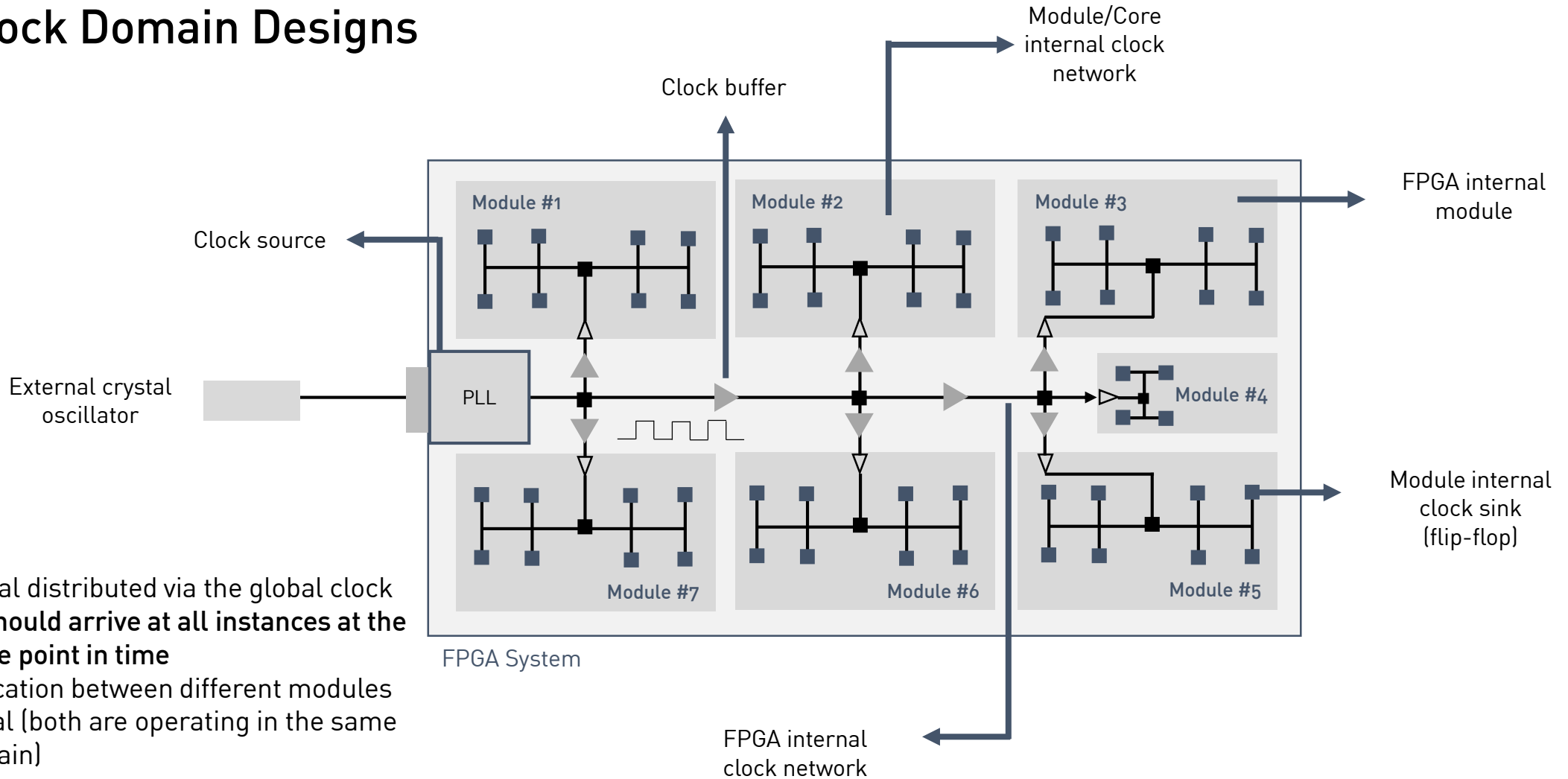
Notes

- Clock signal distributed via the global clock network **should arrive at all instances at the exact same point in time**

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Single Clock Domain Designs



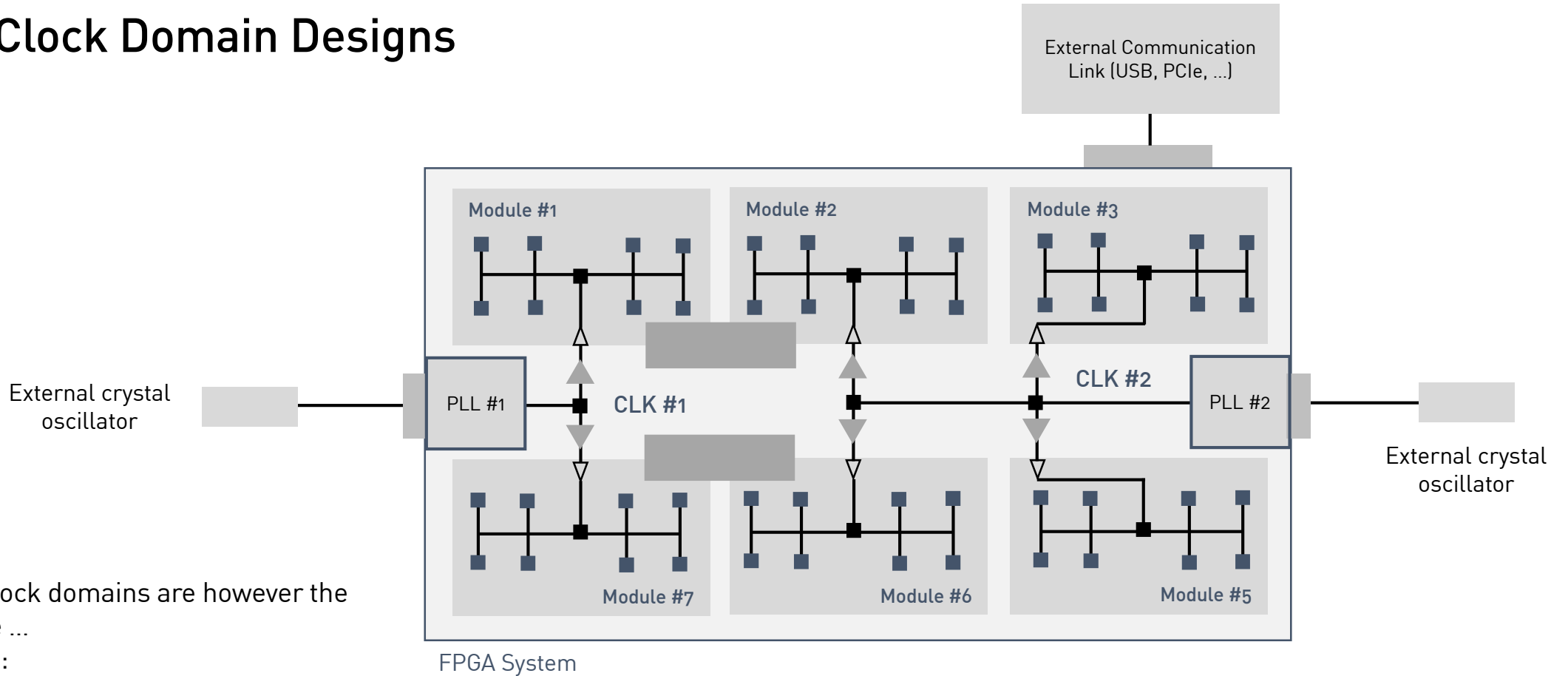
Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Multi Clock Domain Designs

FPGA based System Design

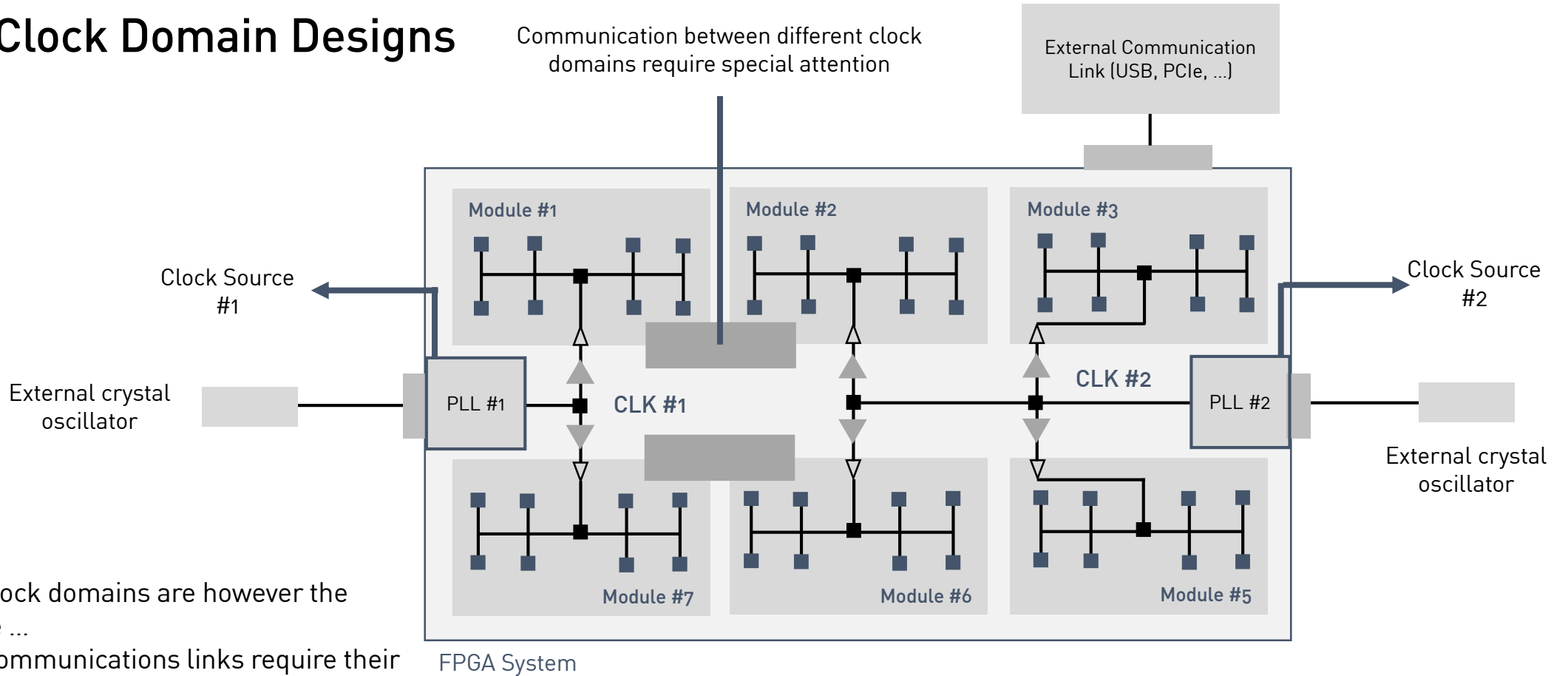
Multiple Clock Domain Designs



Notes

- Multiple clock domains are however the usual case ...
- **Motivation:**
 - External communications links require their own operating frequency
 - Dynamic power consumption is prop. to the switching frequency ...

Multiple Clock Domain Designs



Notes

- Multiple clock domains are however the usual case ...
- External communications links require their own operating frequency

Interfacing FPGA external Components

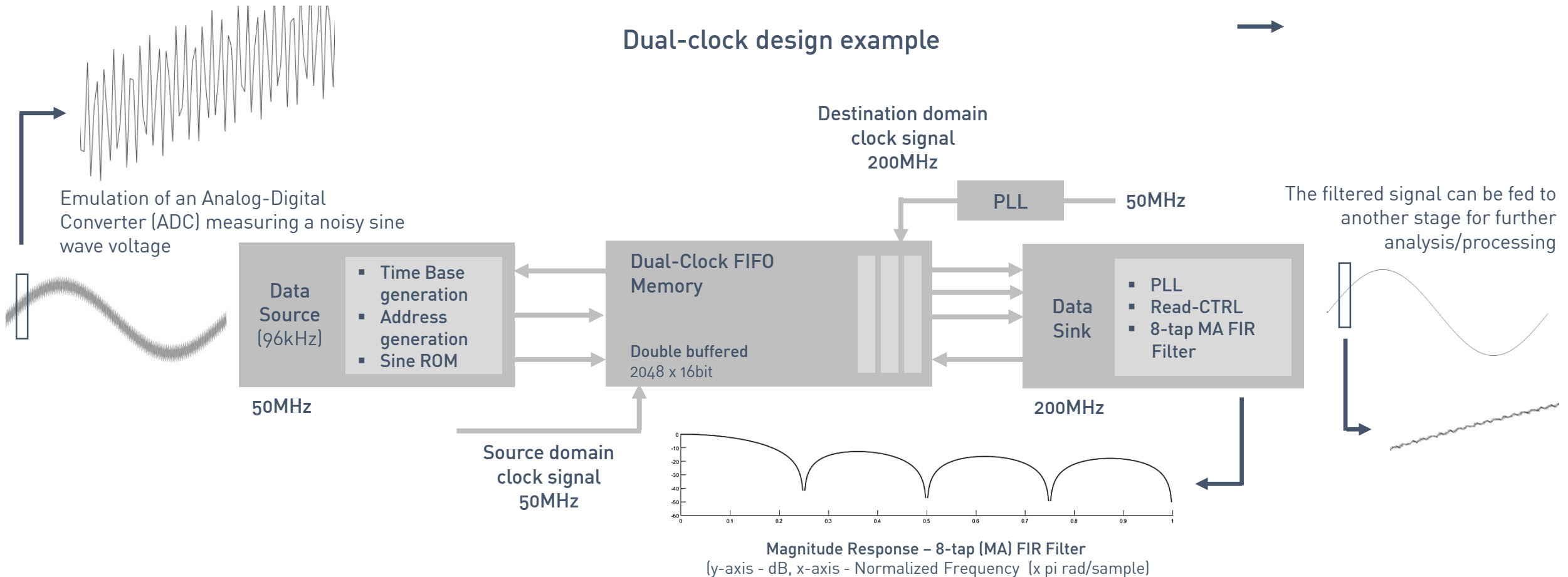
h_da – fbeit - FPGA-based SoC Design

Multi Clock Domain Example Design

FPGA based System Design

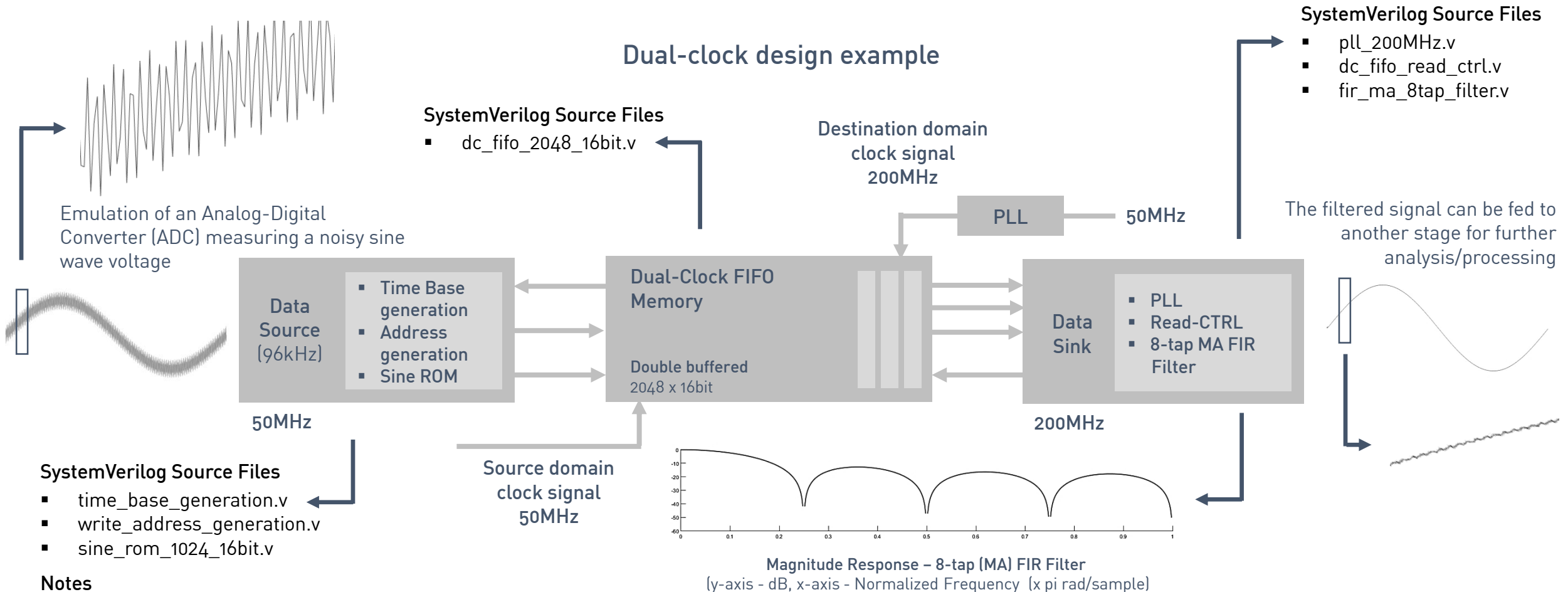
Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design



Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design



Notes

- The signal (sine + HF noise) is generated using MATLAB and used to initialize the data source ROM. Data is written to the DCFIFO at a rate of 96kHz using a time base generation circuitry, i.e. every 10,416µs a sample is stored in memory. The read operation is initiated by a dedicated controller takes place with 200MHz. Therefore, the FIR filter processing of 1024 samples happens in just 5,12µs.

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Challenges involved – Meet timing for every single path in the design ...

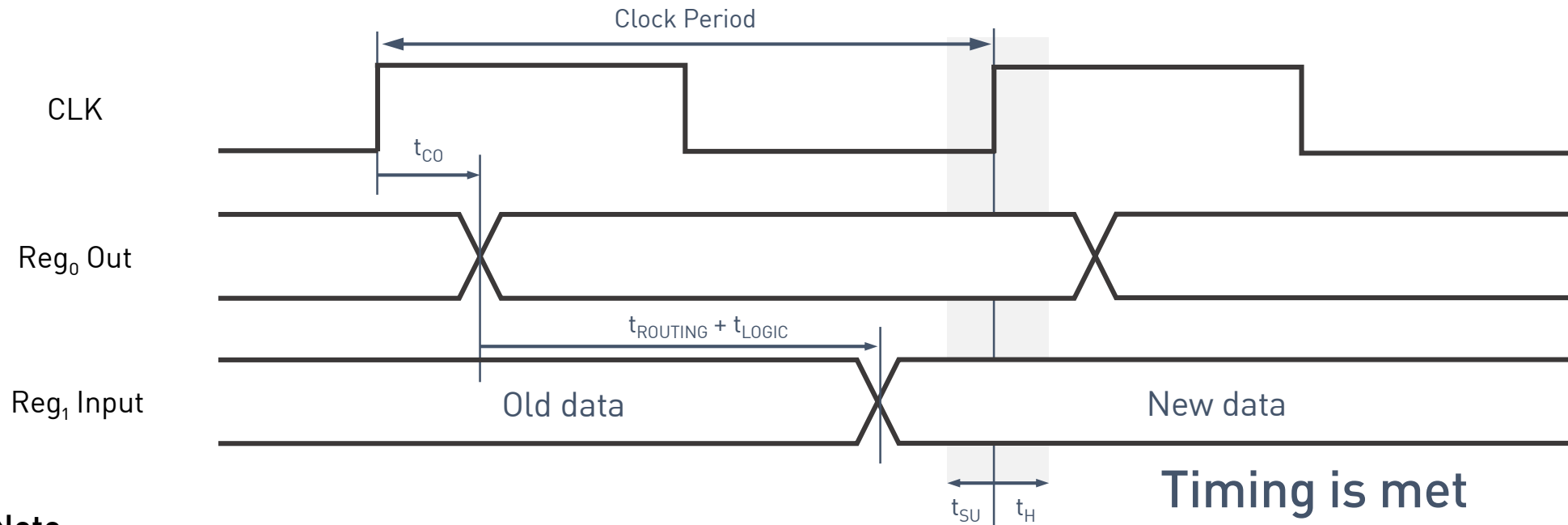
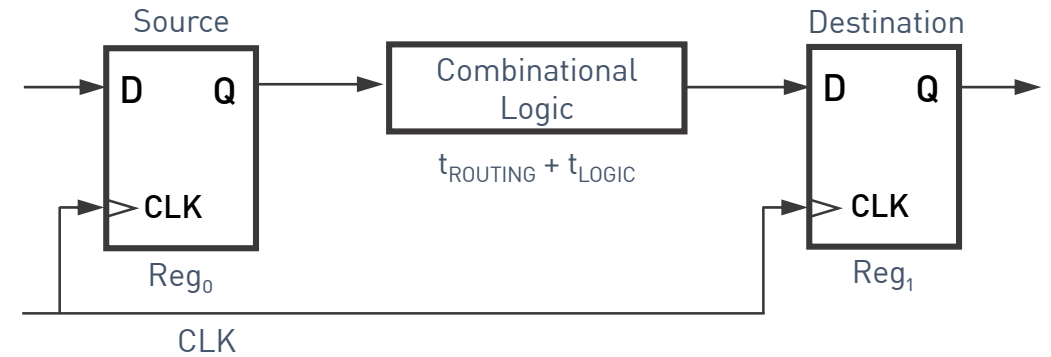
FPGA based System Design

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Clock Sync. Design

Register transfer takes place within one clock cycle



Note

- Path delay: $T_{\text{CLK}} > t_{\text{CO}} + t_{\text{LOGIC}} + t_{\text{ROUTING}} + t_{\text{SU}}$

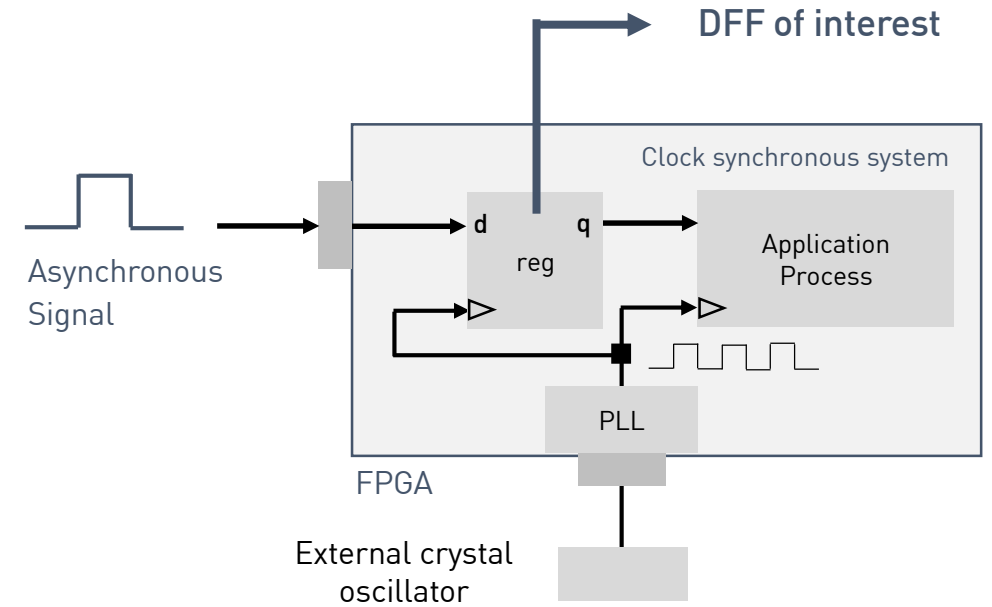
Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

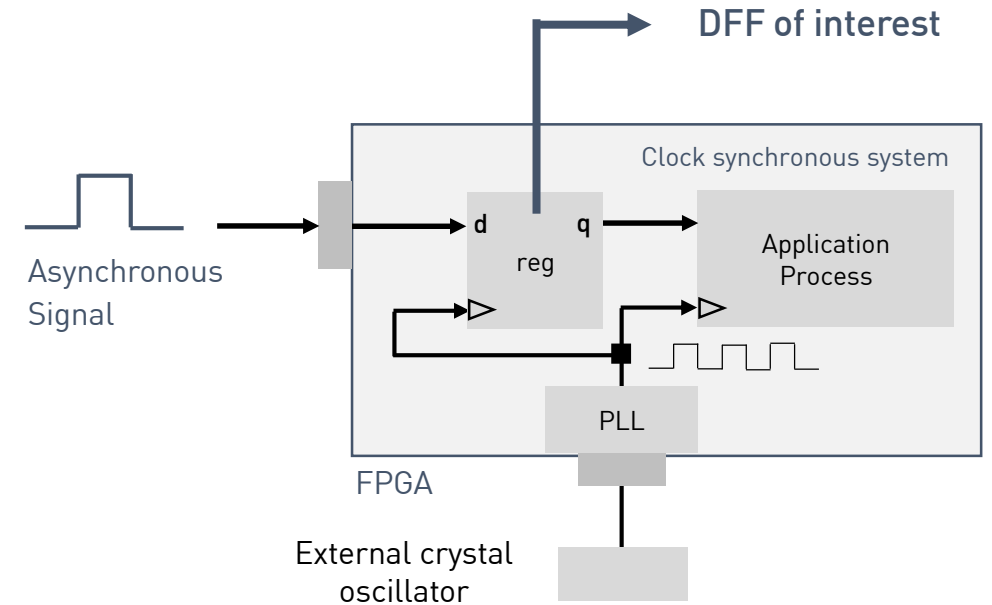
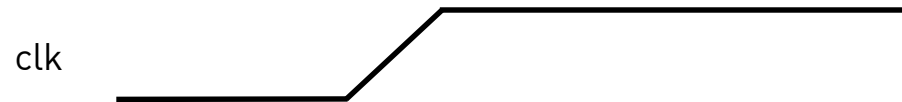
Metastability and Synchronization Techniques

FPGA based System Design

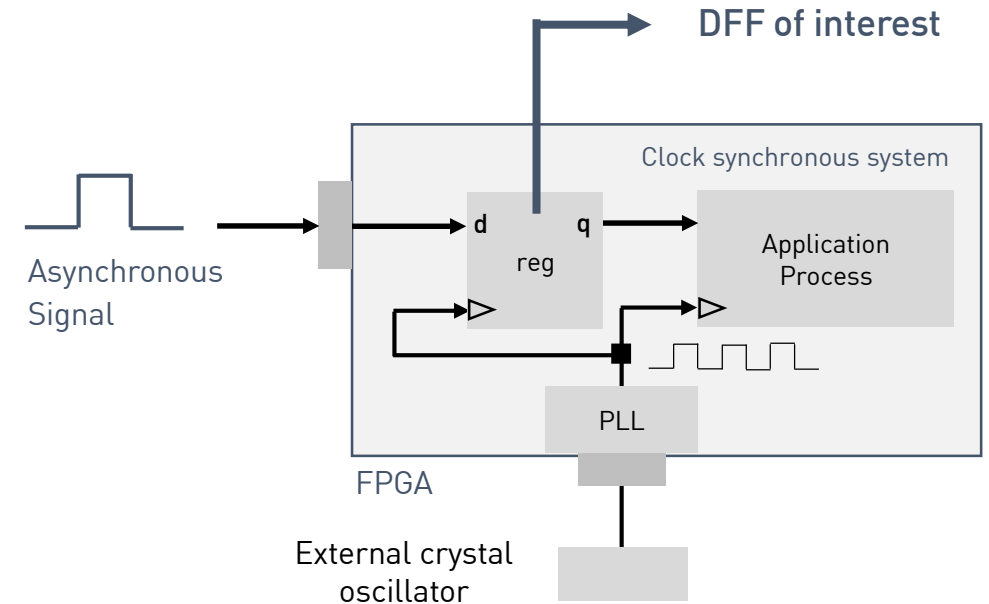
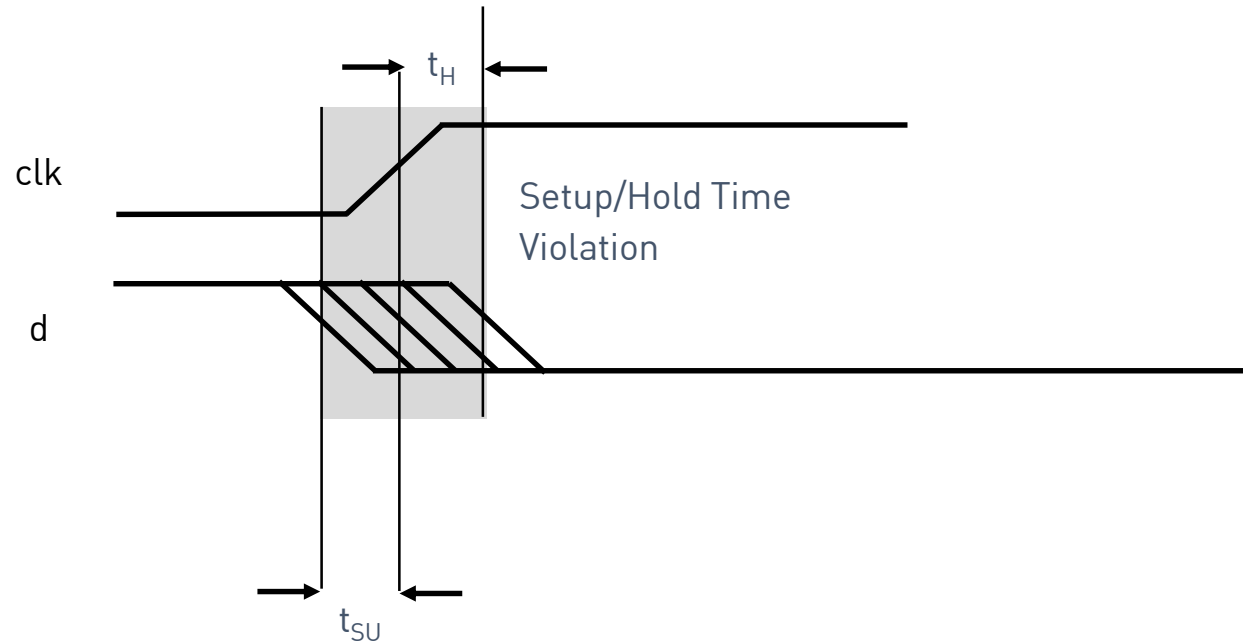
Metastability and Synchronization Techniques



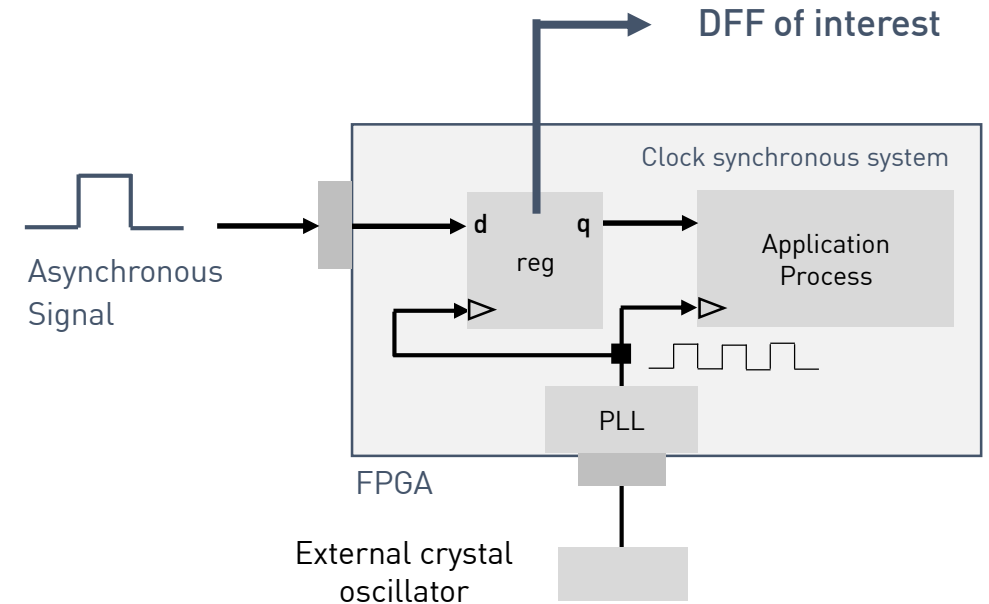
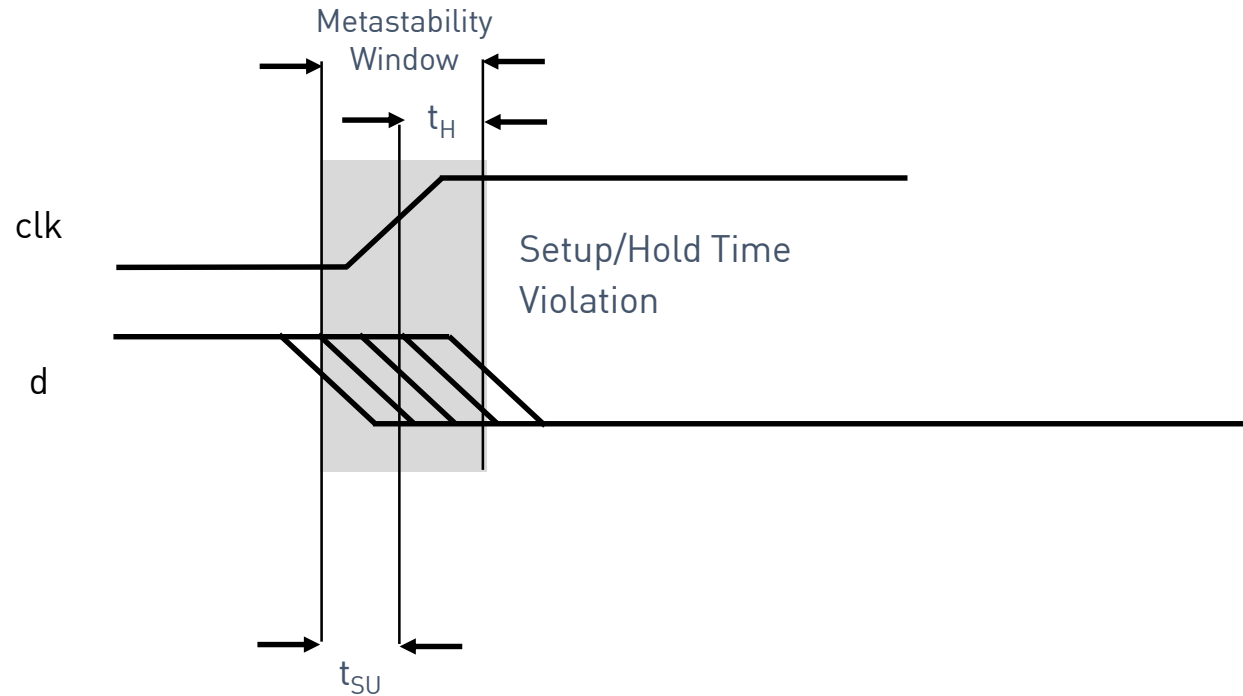
Metastability and Synchronization Techniques



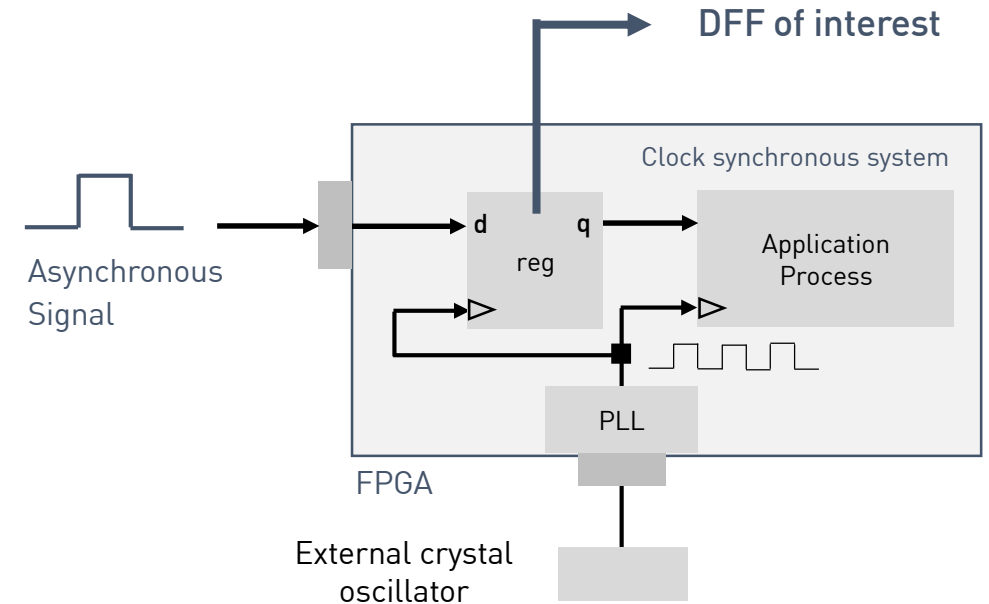
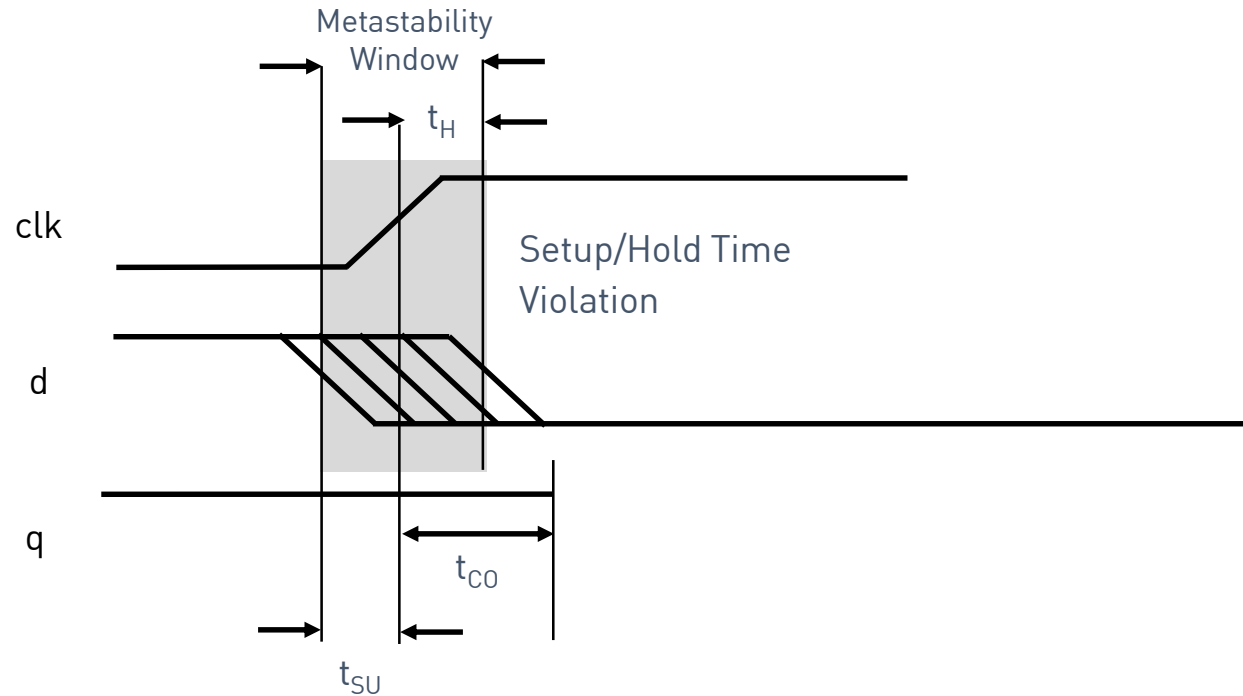
Metastability and Synchronization Techniques



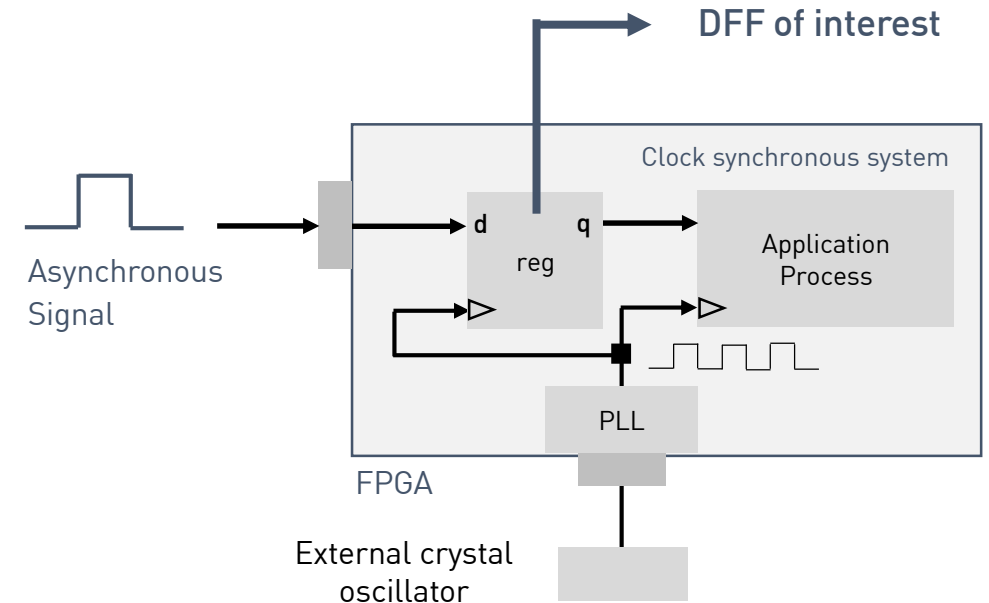
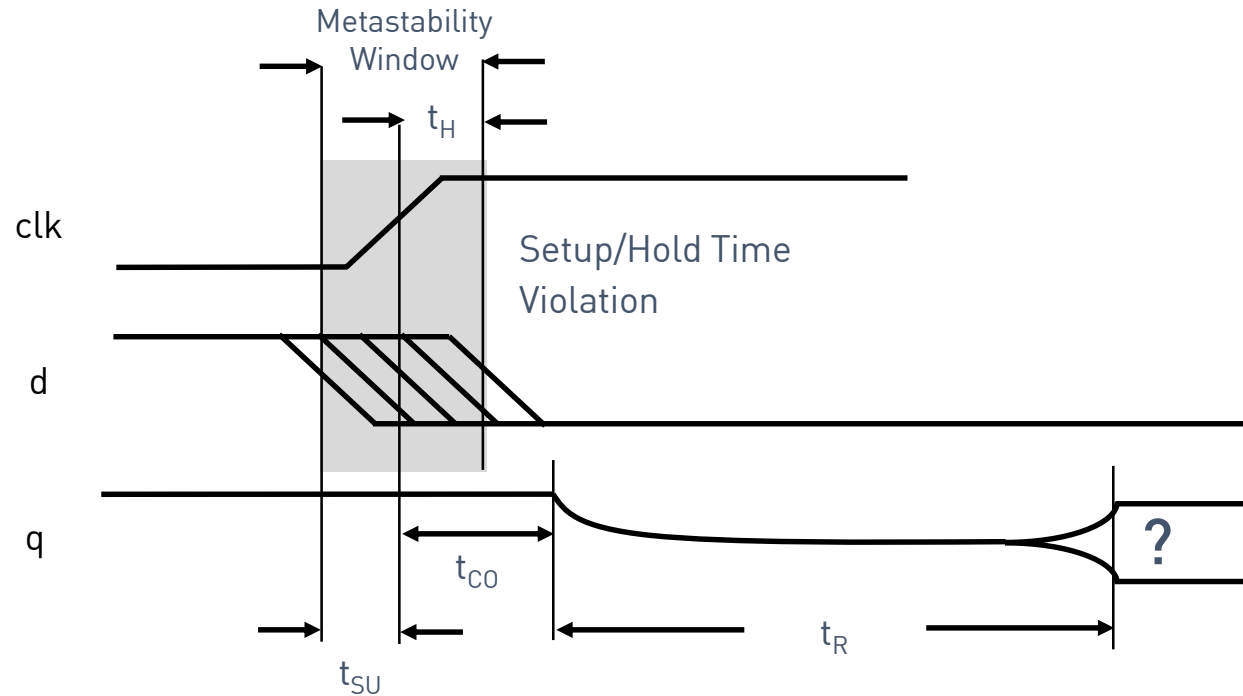
Metastability and Synchronization Techniques



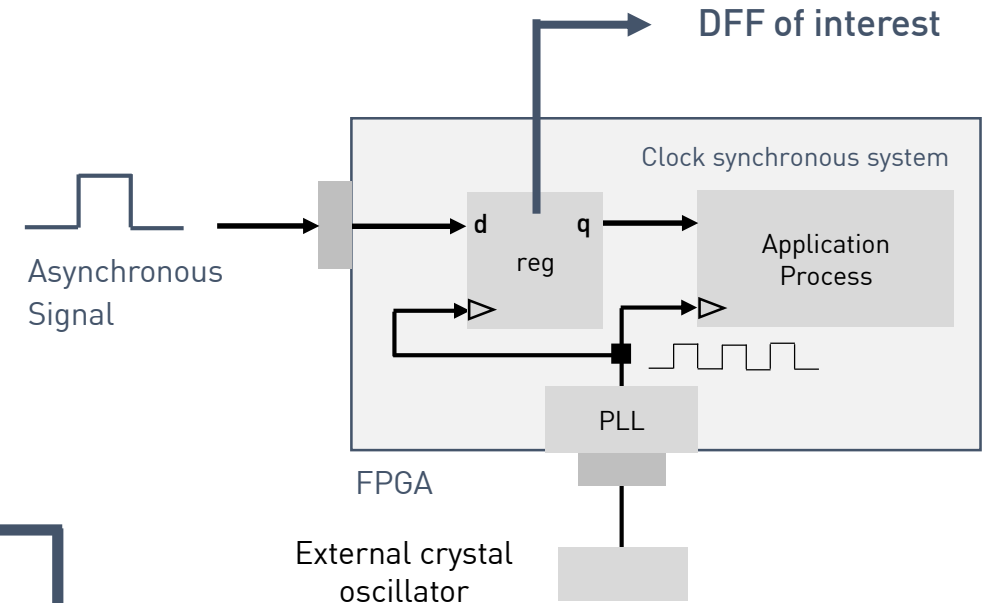
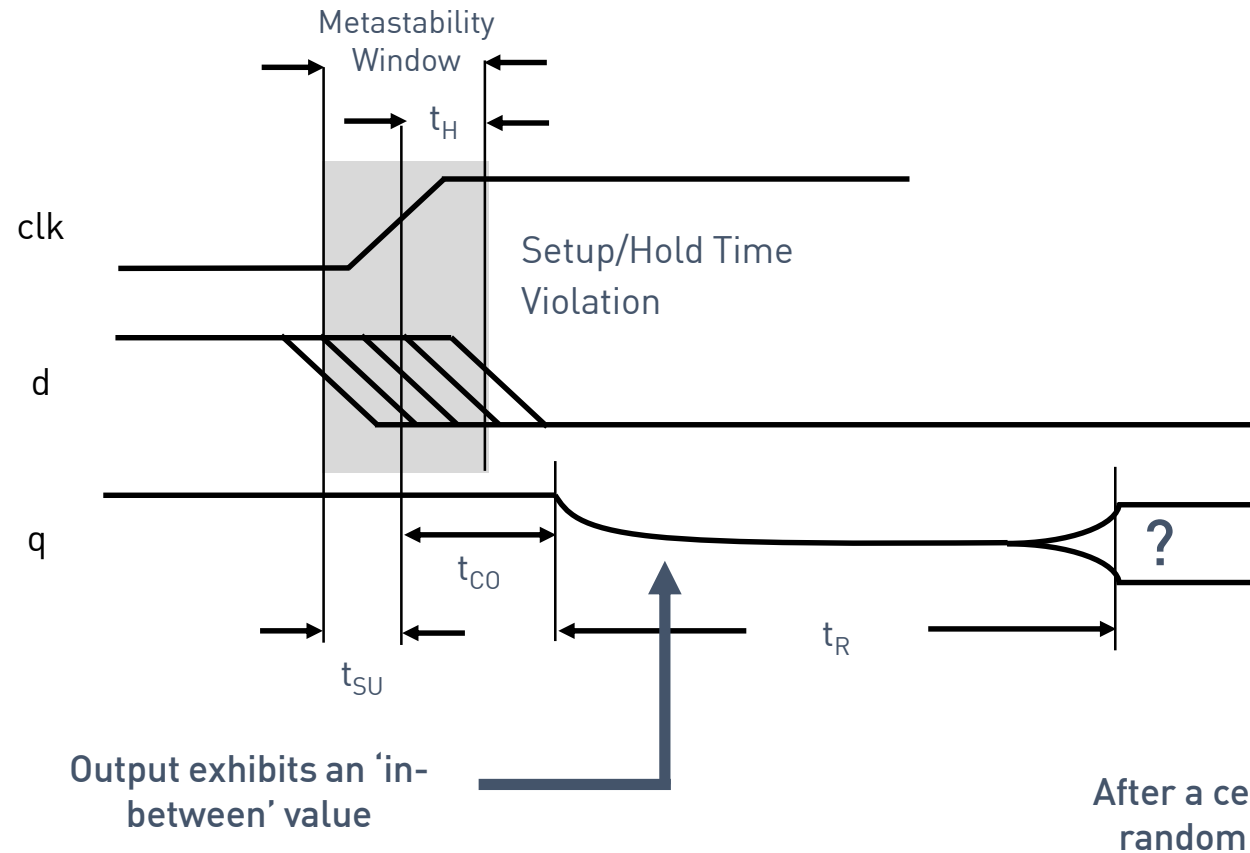
Metastability and Synchronization Techniques



Metastability and Synchronization Techniques



Metastability and Synchronization Techniques



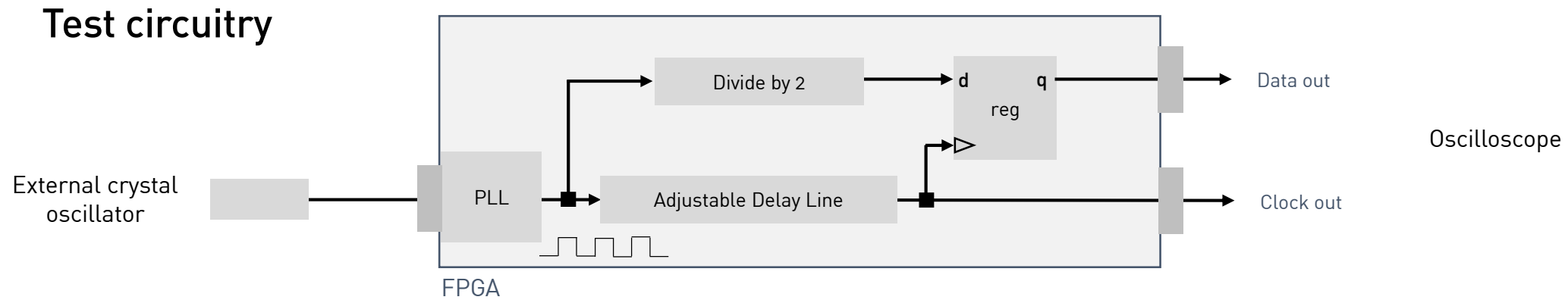
Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

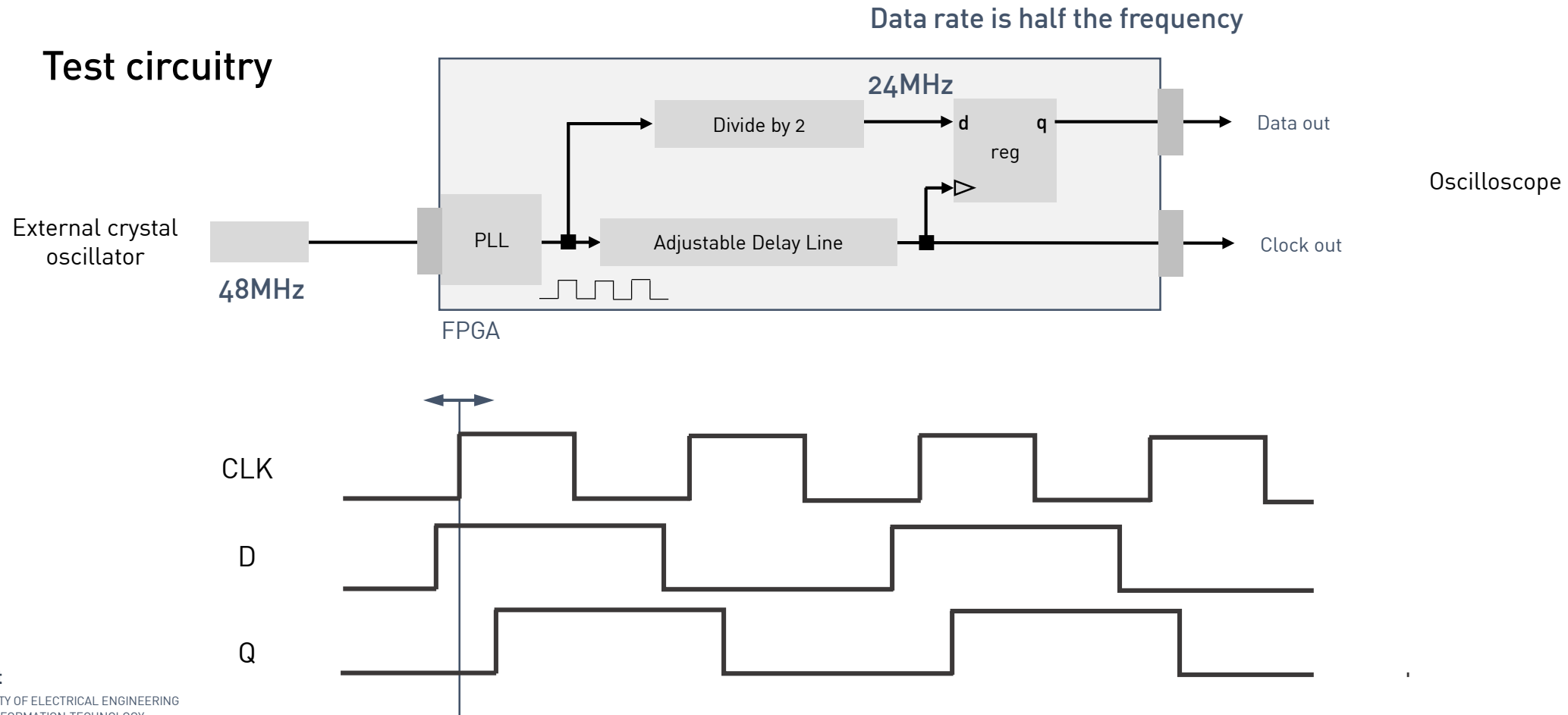
Metastability Test Circuitry

FPGA based System Design

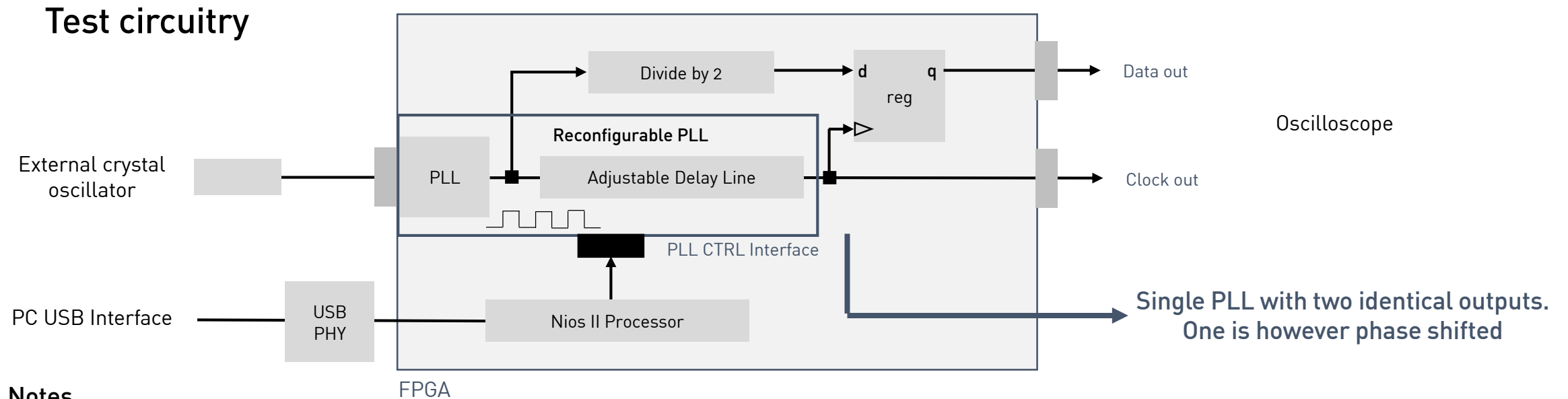
Metastability Test Circuitry



Metastability Test Circuitry



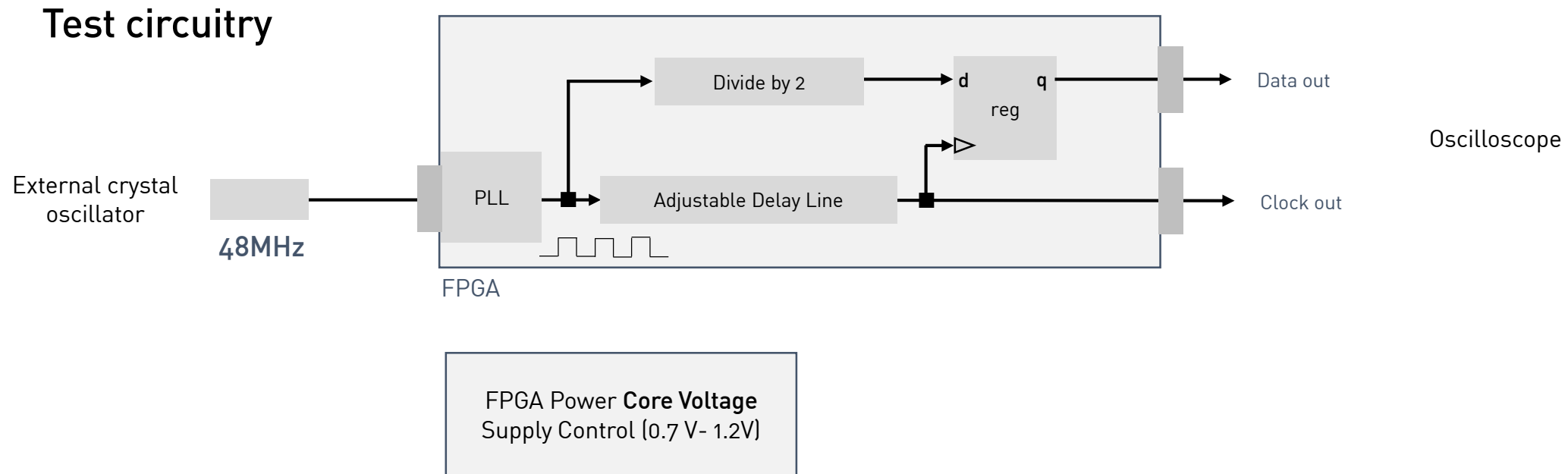
Metastability Test Circuitry



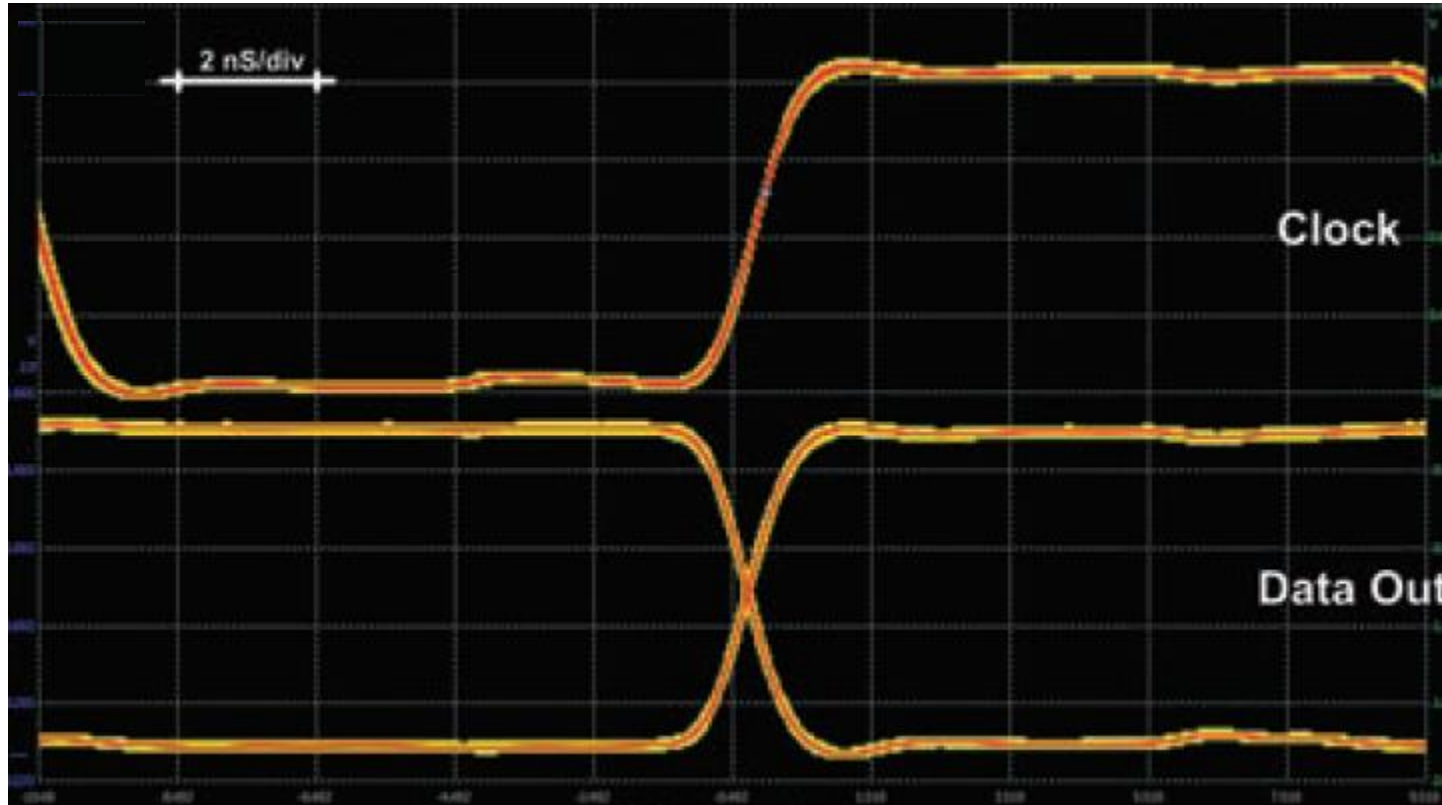
Notes

- The phase offset can be changed during run-time via a dedicated USB PC interface
- A Nios II processor system is used to implement the communication channel and

Metastability Test Circuitry



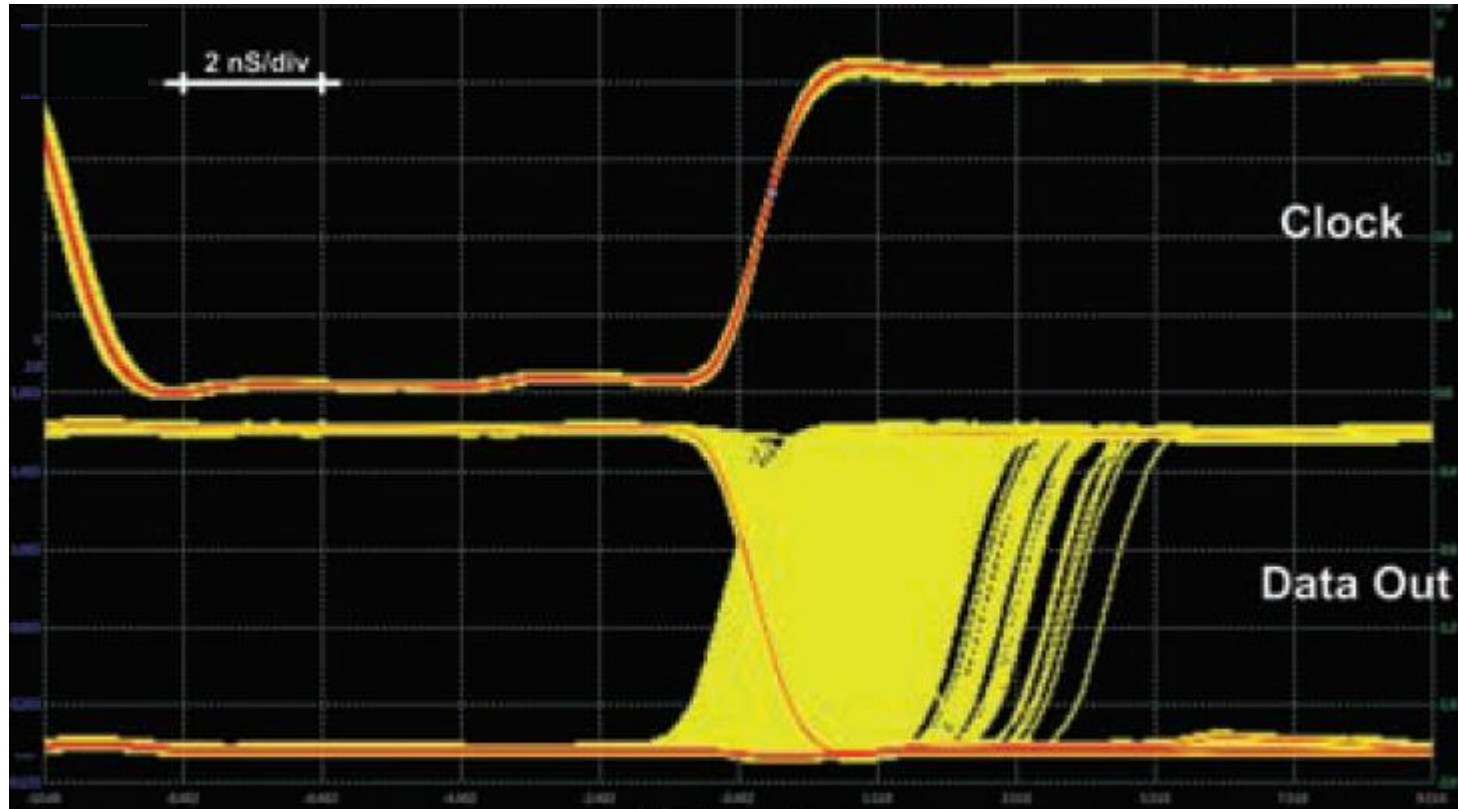
Metastability Test Circuitry



Scenario #1

- The input data and clock of are meeting the setup and hold time specifications of the flip-flop (FF), so the output of the FF is operating normally.

Metastability Test Circuitry

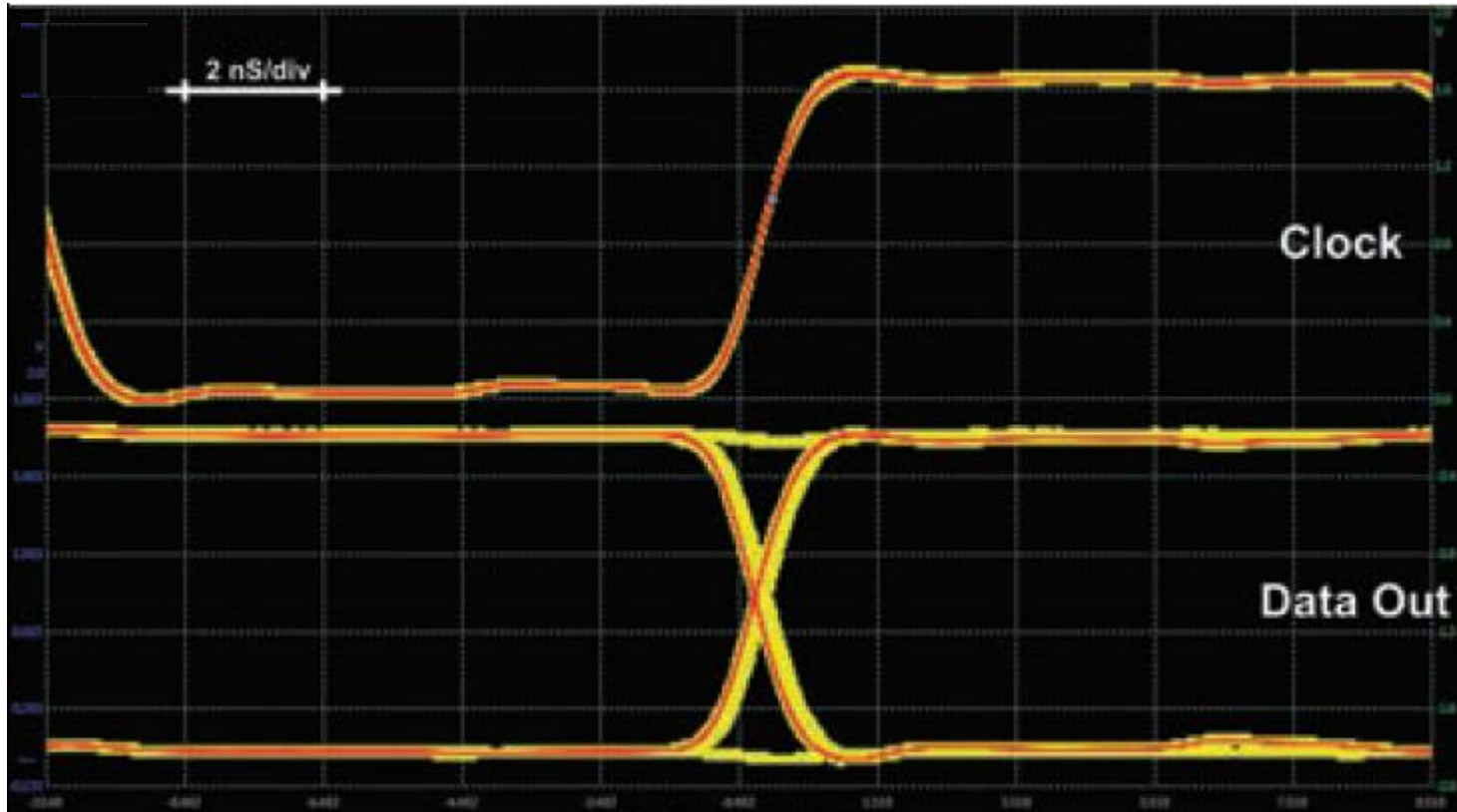


PicoScope 6404C, 2ns/DIV, 2,5GS/s, Range: +/- 2V

Scenario #2

- Setup time is being violated, and metastable states are resulting at the output, which takes additional time to resolve.
- Note that the core voltage of the FPGA is set to 0.7 V to reinforce the metastable effect.

Metastability Test Circuitry



PicoScope 6404C, 2ns/DIV, 2,5GS/s, Range: +/- 2V

Scenario #3

- the FPGA at 1.2 V (normal core voltage) still shows the metastable state taking longer to resolve to a final value.

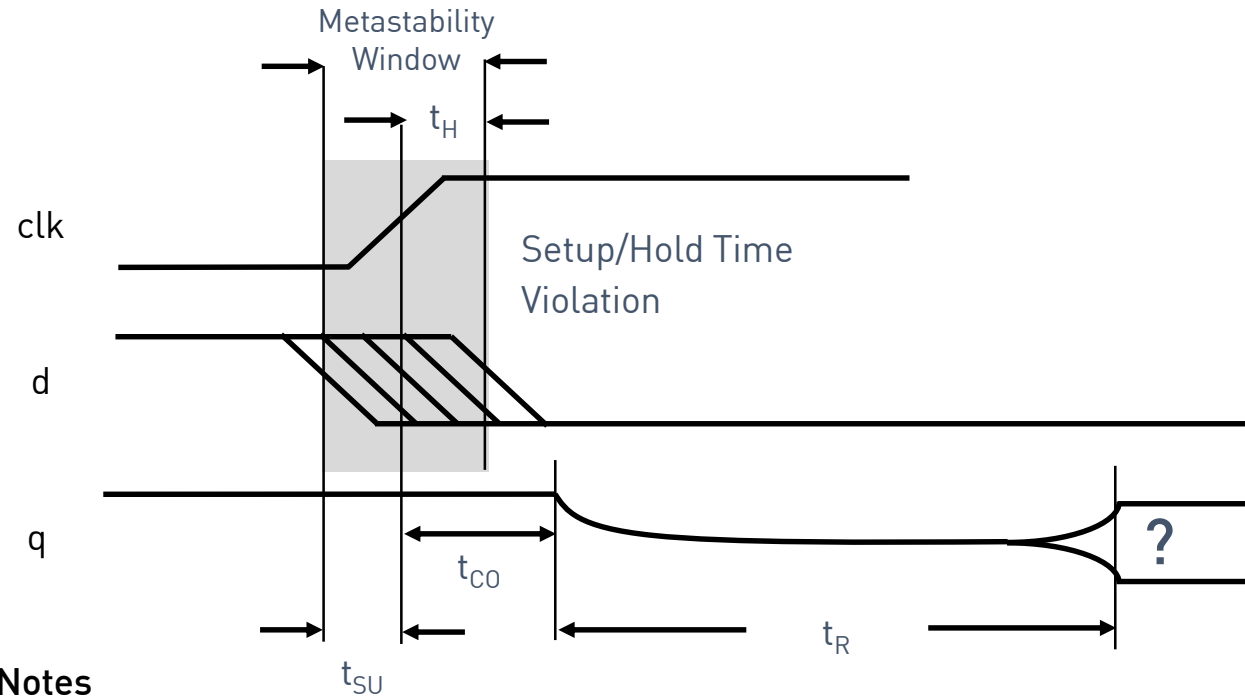
Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Summary/Conclusion so far ...

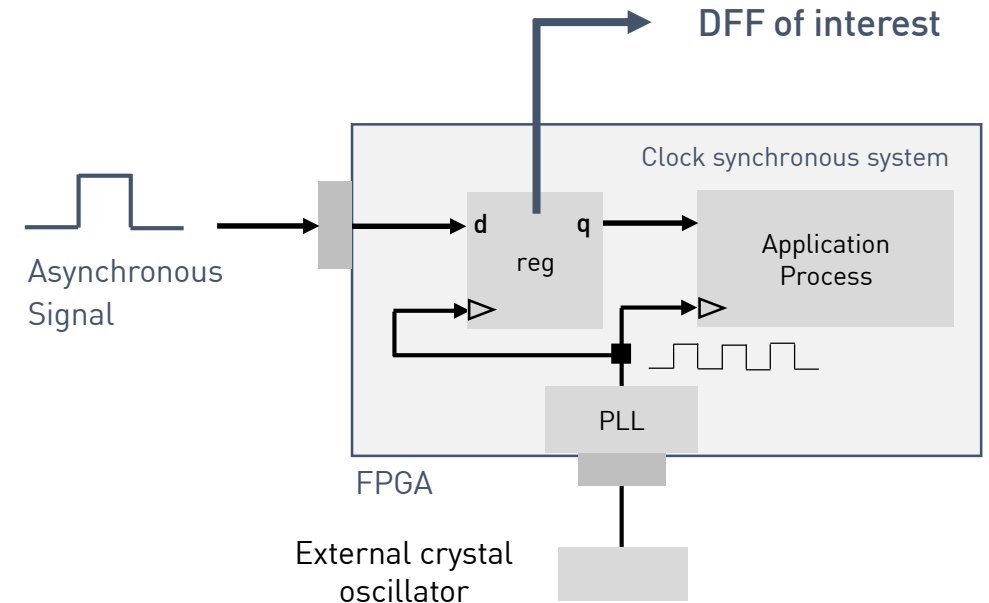
FPGA based System Design

Metastability and Synchronization Techniques

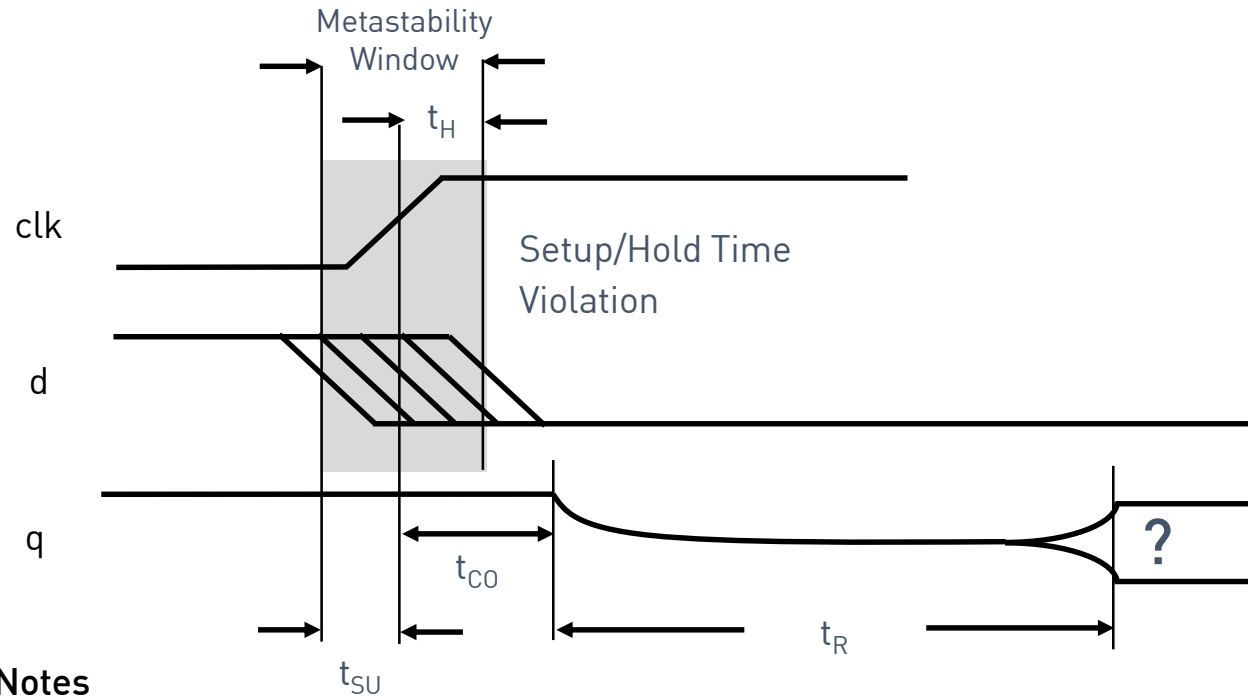


Notes

- Metastability arises as a result of violating the setup/hold times of flip-flop

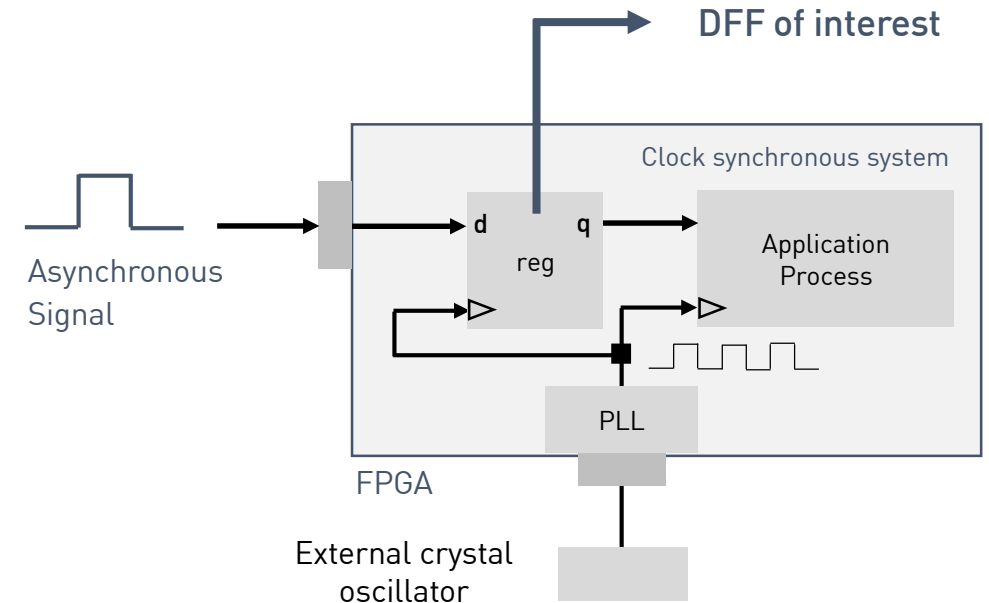


Metastability and Synchronization Techniques

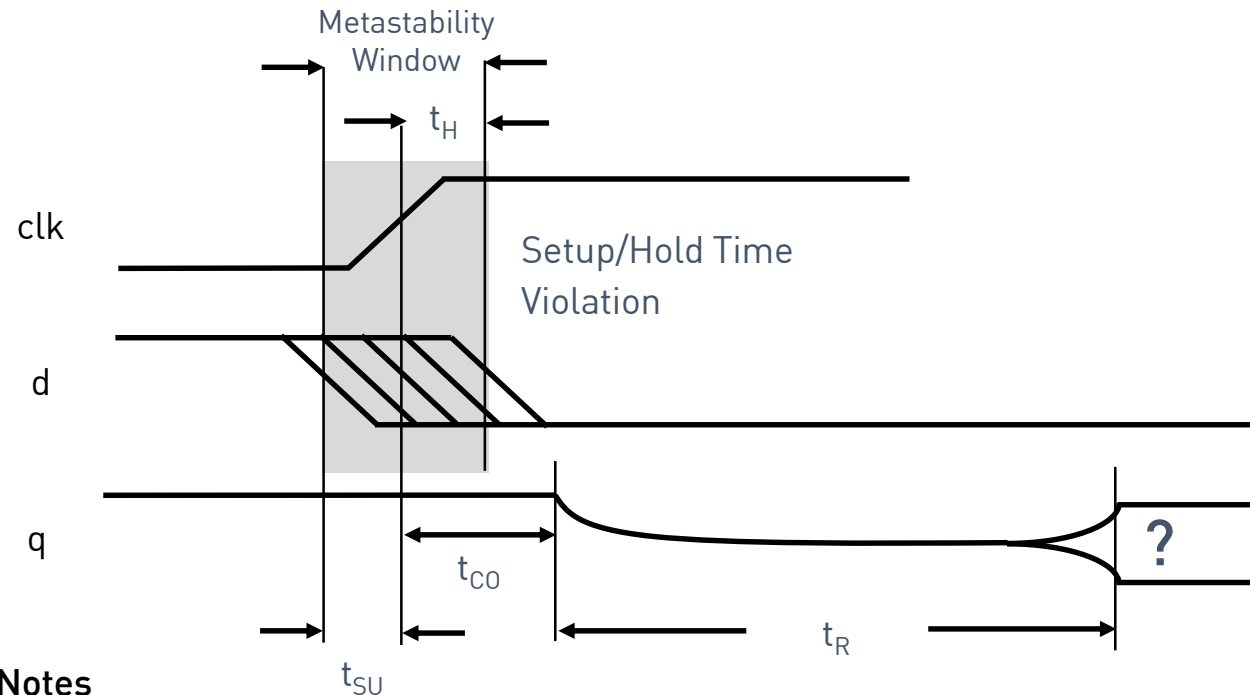


Notes

- Metastability arises as a result of **violating the setup/hold times** of flip-flop
- Every flip-flop has a time during which the data input is not legally permitted to change before/after a rising clock edge

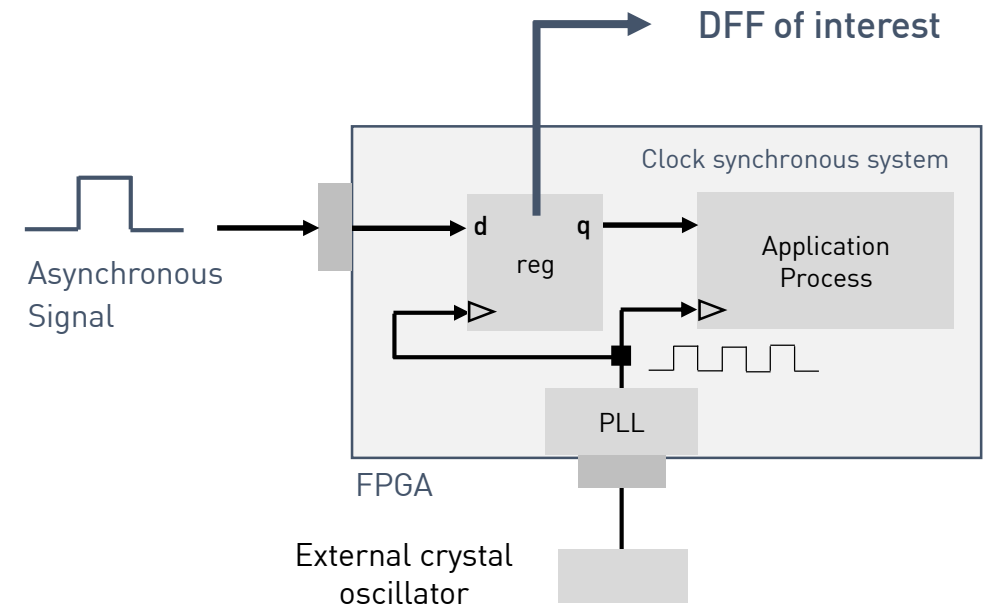


Metastability and Synchronization Techniques

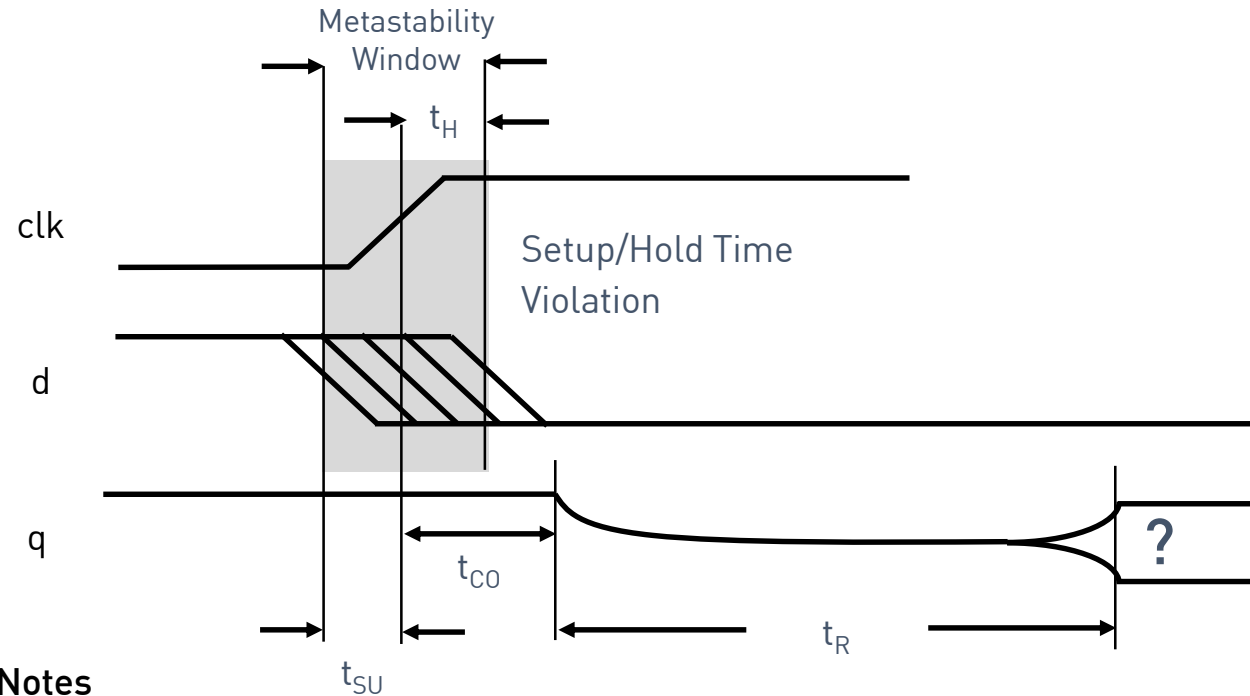


Notes

- Metastability arises as a result of **violating the setup/hold times** of flip-flop
- Every flip-flop has a time during which the data input is not legally permitted to change before/after a rising clock edge
- If the data signal changes during this time window, the output can become metastable ...

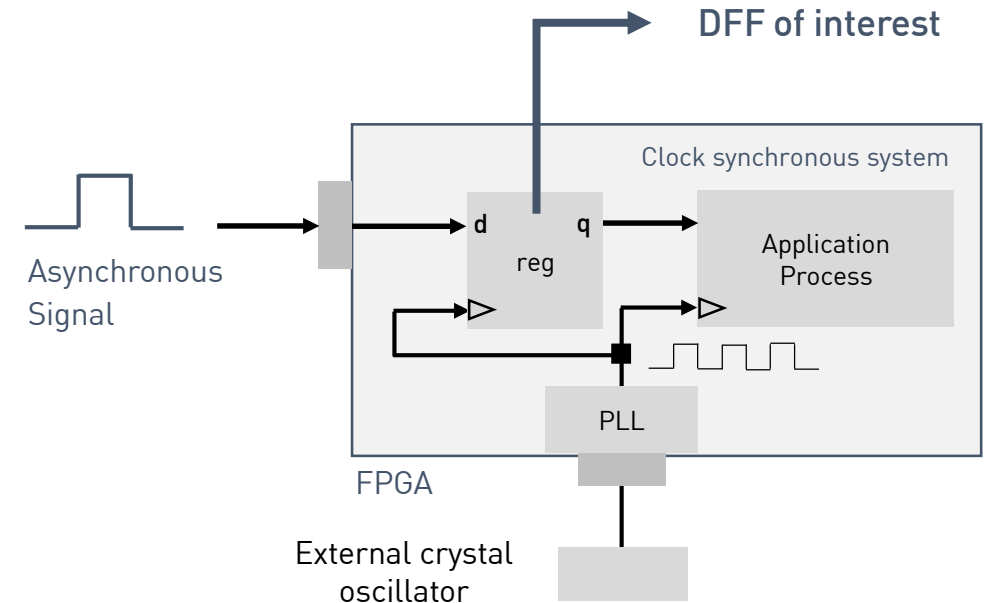


Metastability and Synchronization Techniques

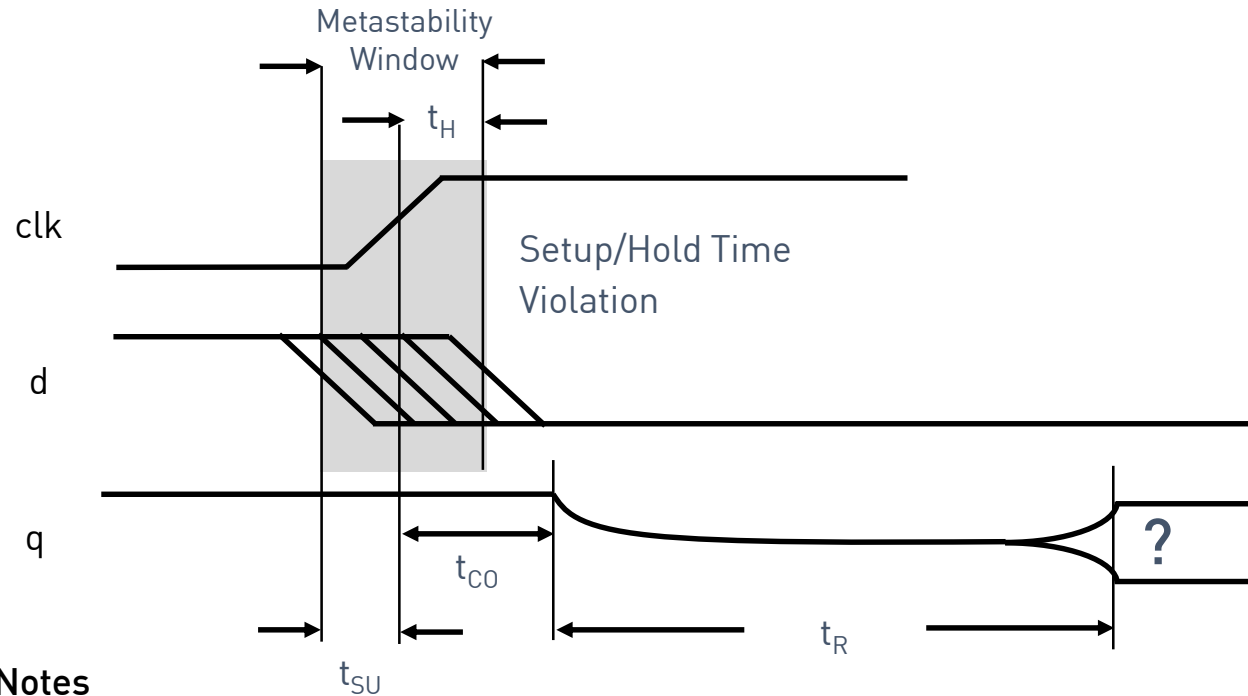


Notes

- If the flip-flop enters a metastable state, the output exhibits an in-between value for a certain time ...

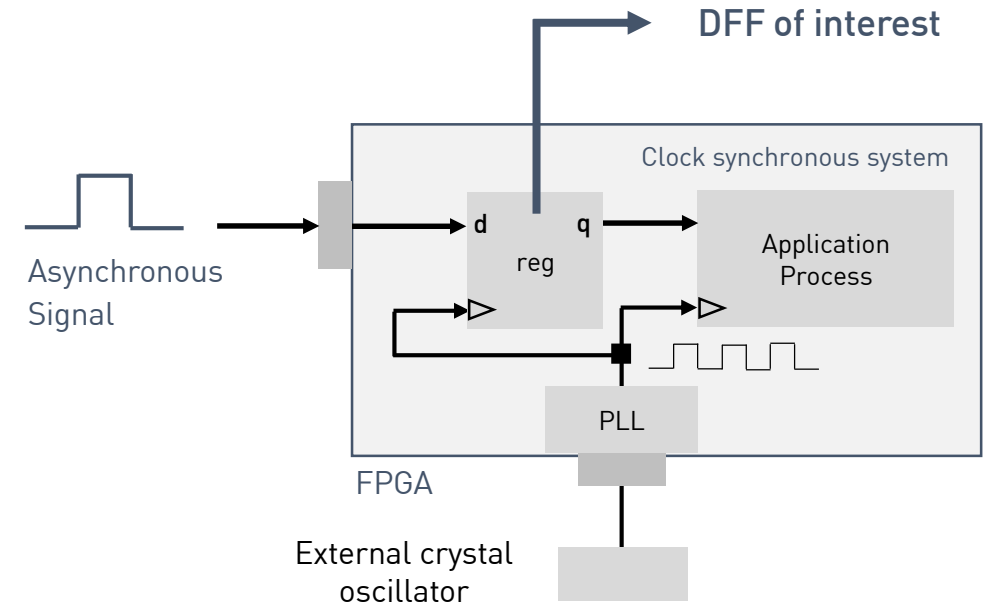


Metastability and Synchronization Techniques



Notes

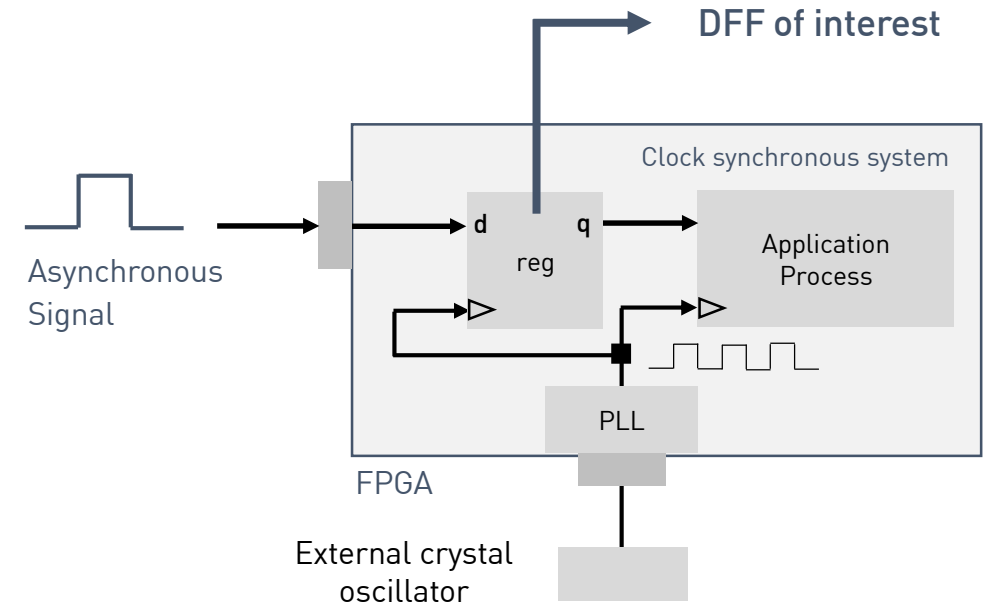
- If the flip-flop enters a metastable state, the output exhibits an in-between value for a certain time ...
- After a certain time the output resolves to a stable state



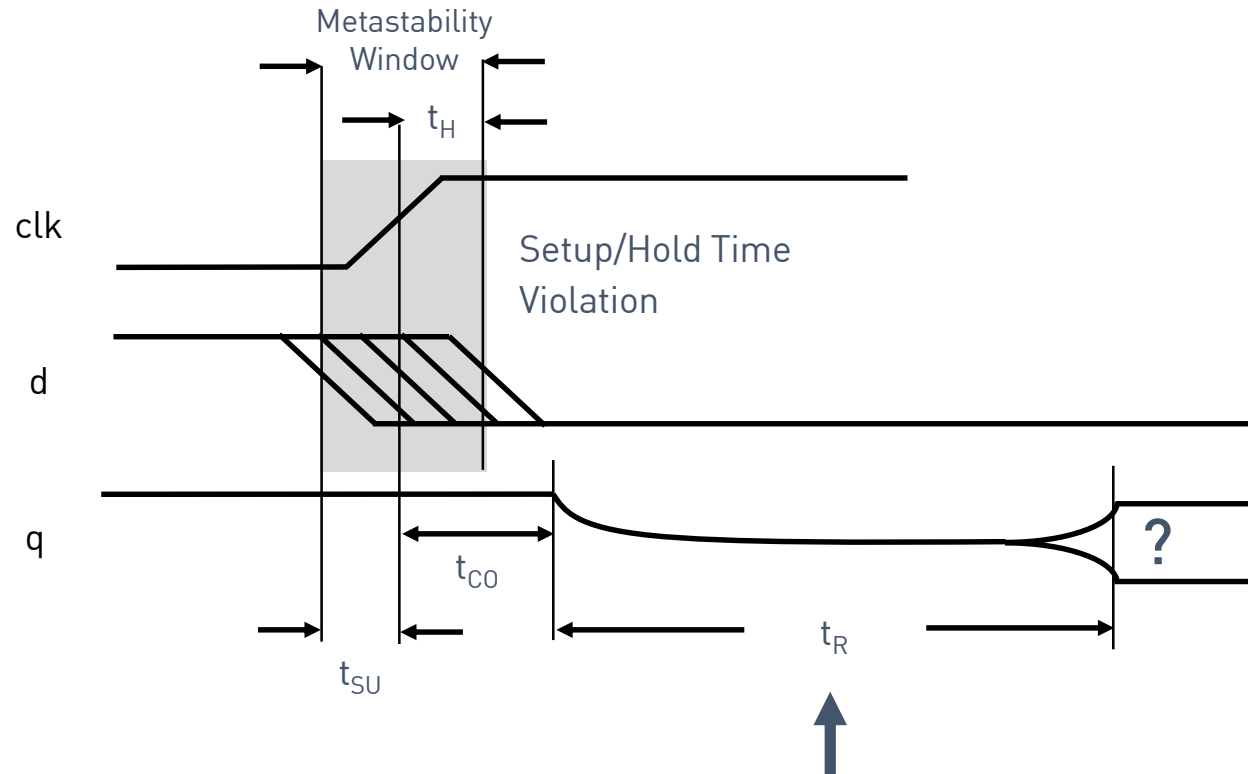
Timing diagram illustrating Setup and Hold Time Violations:

- clk**: Clock signal.
- d**: Data signal.
- q**: Output signal.
- Metastability Window**: A gray shaded region around the clock edge where the data signal is not stable.
- Setup/Hold Time Violation**: A hatched region within the metastability window indicating a violation.
- t_H** : Hold time, the time the data must remain stable after the clock edge.
- t_{CO}** : Propagation delay, the time from the clock edge to the output signal.
- t_R** : Recovery time, the time the output signal takes to return to a stable state.
- ?**: A question mark indicating an unknown or unstable output state.

- If the flip-flop enters a metastable state, the output exhibits an in-between value for a certain time ...
- After a certain time the output resolves to a stable state
- The additional time beyond the clock-to-output delay t_{CO} that a metastable state takes to resolve to a stable state is called the **resolution time** t_R (also known as settling time) ...

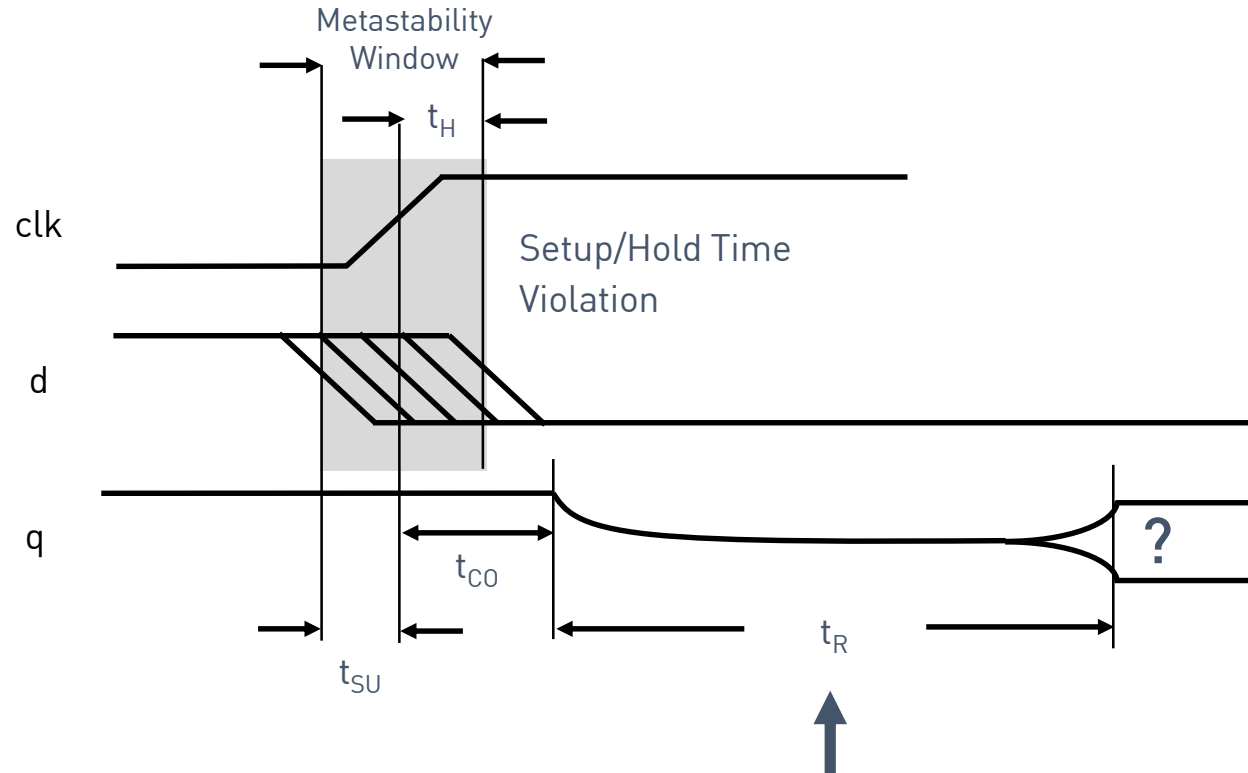


Metastability and Synchronization Techniques



Once the flip-flop has entered the metastable state, the probability that it will still be metastable some time later has been shown to be an exponentially decreasing function.

Metastability and Synchronization Techniques



Notes

- The probability that a metastable state persists longer than t_R can be computed as follows:

$$P(t_R) = e^{-t_R/\tau}$$

Once the flip-flop has entered the metastable state, the probability that it will still be metastable some time later has been shown to be an exponentially decreasing function.

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Describing Metastablity

FPGA based System Design

Metastability and Synchronization Techniques

Notes

- Synchronization failure: A flip-flop cannot resolve its metastable output within a certain time ...

Metastability and Synchronization Techniques

Notes

- Synchronization failure: A flip-flop cannot resolve its metastable output within a certain time ...
- Metastability Window (in seconds): t_{mw} (part of the setup/hold window)

Metastability and Synchronization Techniques

Notes

- Synchronization failure: A flip-flop cannot resolve its metastable output within a certain time ...
- Metastability Window (in seconds): t_{mw} (part of the setup/hold window)
- Mean Time Between Failure (**MTBF**): Basic measure in the context of metastability analysis

Metastability and Synchronization Techniques

Notes

- Synchronization failure: A flip-flop cannot resolve its metastable output within a certain time ...
- Metastability Window (in seconds): t_{mw} (part of the setup/hold window)
- Mean Time Between Failure (**MTBF**): Basic measure in the context of metastability analysis
- Average rate at which an flip-flop enters a metastable state: $R = t_{mw} \times f_{clk} \times f_{data}$

Metastability and Synchronization Techniques

Notes

- Synchronization failure: A flip-flop cannot resolve its metastable output within a certain time ...
- Metastability Window (in seconds): t_{mw} (part of the setup/hold window)
- Mean Time Between Failure (**MTBF**): Basic measure in the context of metastability analysis
- Average rate at which an flip-flop enters a metastable state: $R = t_{mw} \times f_{clk} \times f_{data}$
- Probability that a metastable flip-flop **cannot** resolve within t_R : $P(t_R) = e^{-t_R/\tau}$

Metastability and Synchronization Techniques

Notes

- Synchronization failure: A flip-flop cannot resolve its metastable output within a certain time ...
- Metastability Window (in seconds): t_{mw} (part of the setup/hold window)
- Mean Time Between Failure (**MTBF**): Basic measure in the context of metastability analysis
- Average rate at which an flip-flop enters a metastable state: $R = t_{mw} \times f_{clk} \times f_{data}$
- Probability that a metastable flip-flop cannot resolve within t_R : $P(t_R) = e^{-t_R/\tau}$
- Average number of synchronization failures within one second: $R \times P(t_R)$ (Failure rate)

Metastability and Synchronization Techniques

Notes

- Synchronization failure: A flip-flop cannot resolve its metastable output within a certain time ...
- Metastability Window (in seconds): t_{mw} (part of the setup/hold window)
- Mean Time Between Failure (**MTBF**): Basic measure in the context of metastability analysis
- Average rate at which an flip-flop enters a metastable state: $R = t_{mw} \times f_{clk} \times f_{data}$
- Probability that a metastable flip-flop cannot resolve within t_R : $P(t_R) = e^{-t_R/\tau}$
- Average number of synchronization failures within one second: $R \times P(t_R)$ (Failure rate)
- Mean (average) time between synchronization failures: $\tau, t_{mw} - const.$ (technology dependent)

$$MTBF(t_R) = \frac{1}{R \times P(t_R)} = \frac{e^{t_R/\tau}}{t_{mw} \times f_{CLK} \times f_{data}}$$

Metastability and Synchronization Techniques

Notes

- Mean (average) time between synchronization failures:

$\tau, t_{mw} - \text{const.}$ (technology dependent)

$$MTBF(t_R) = \frac{1}{R \times P(t_R)} = \frac{e^{t_R/\tau}}{t_{mw} \times f_{CLK} \times f_{data}}$$

- Important conclusion:

The MTBF is extremely sensitive to the resolution time: Increasing the resolution time increases the MTBF

Metastability and Synchronization Techniques

Notes

- Next, we will study a couple of synchronization circuitries ...

Metastability and Synchronization Techniques

A few things to keep in mind ...

- Please note that avoiding metastability turns out to be an impossible problem!
- The only solution to reduce the probability of metastability in unrelated or asynchronous clock domains is to increase the resolution time!
- Synchronizers provide enough time for the metastable event to be resolved ...

Metastability and Synchronization Techniques

Assume the specified values/parameters for the following MTBF analysis

- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps

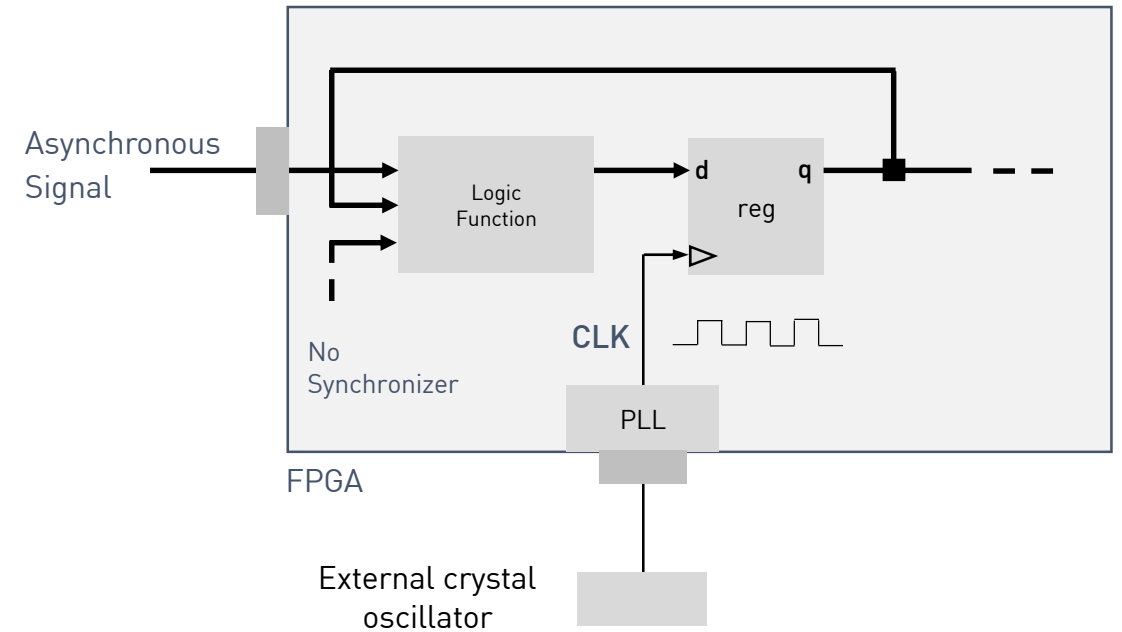
$$MTBF(t_R) = \frac{1}{R \times P(t_R)} = \frac{e^{t_R/\tau}}{t_{mw} \times f_{CLK} \times f_{data}}$$

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Scenarion #1

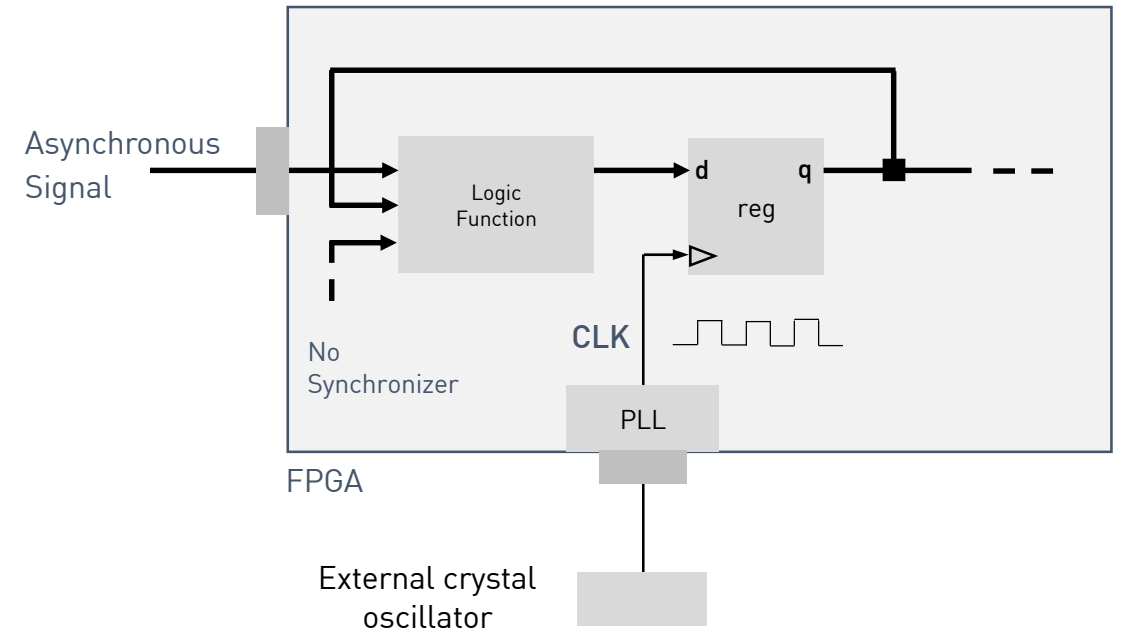
No synchronization used ...



Metastability and Synchronization Techniques

No synchronization used ...

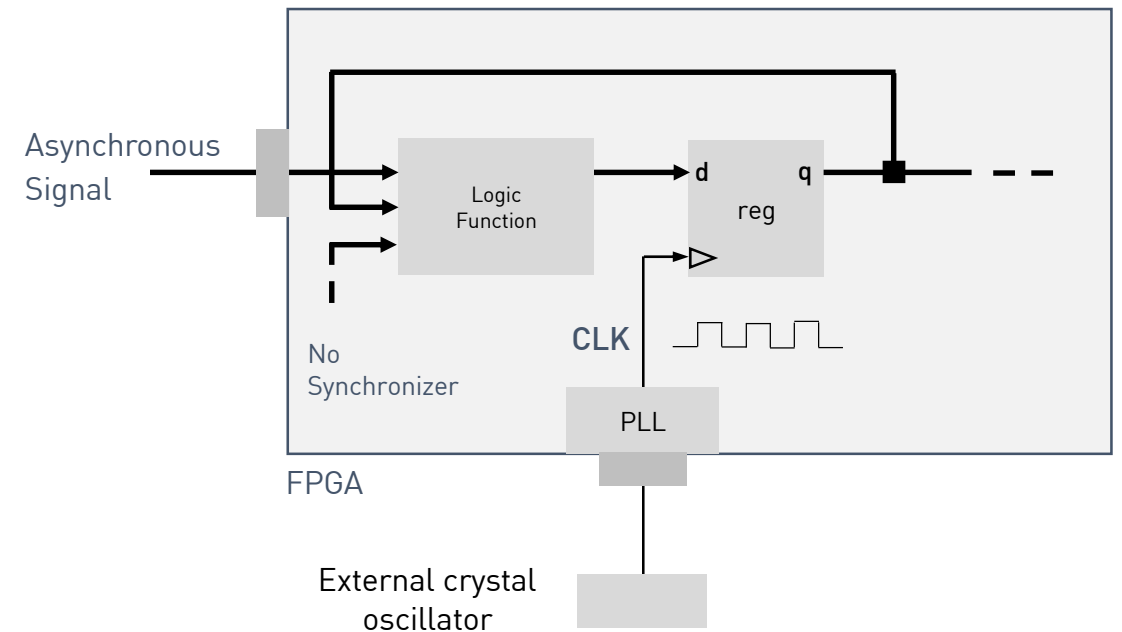
- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: 0ns



Metastability and Synchronization Techniques

No synchronization used ...

- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: ?

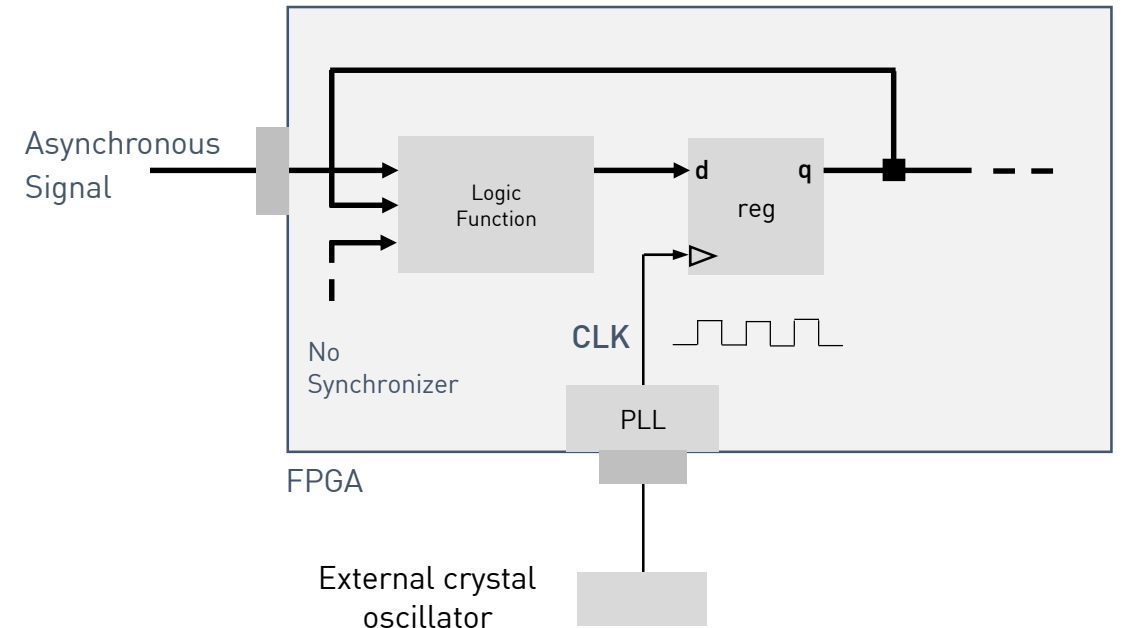


Metastability and Synchronization Techniques

No synchronization used ...

- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: 0ns
- MTBF (calculated): 3.8 μ s

Not reliable ...

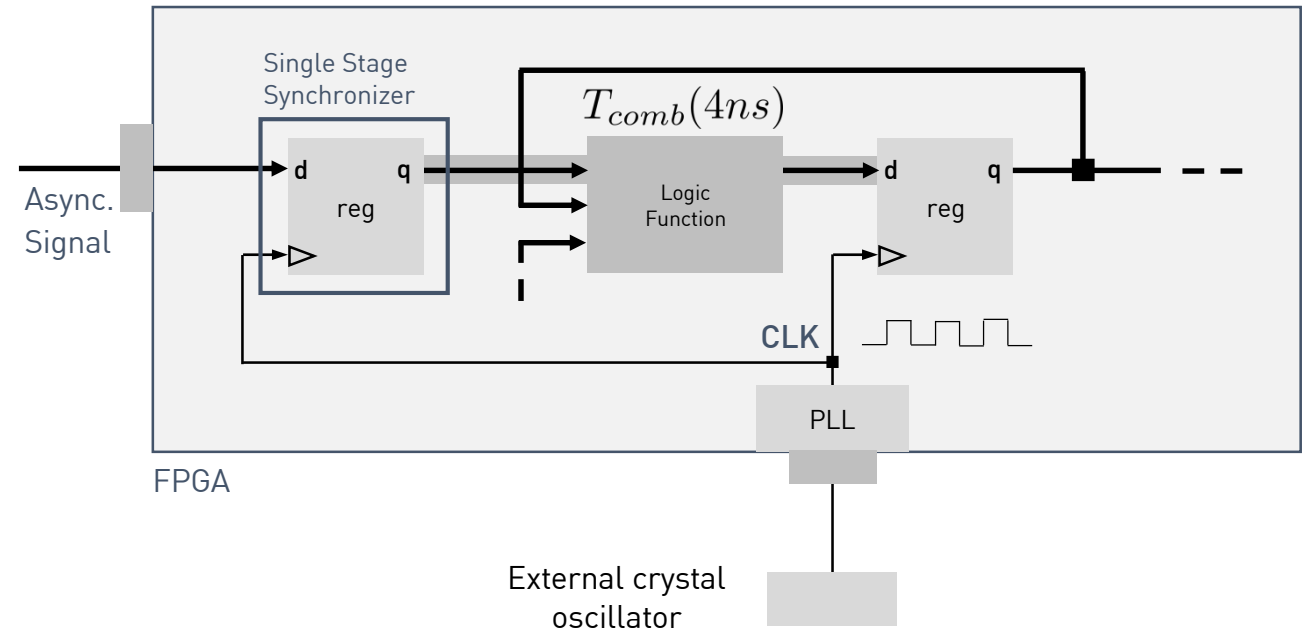


Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Scenarion #2

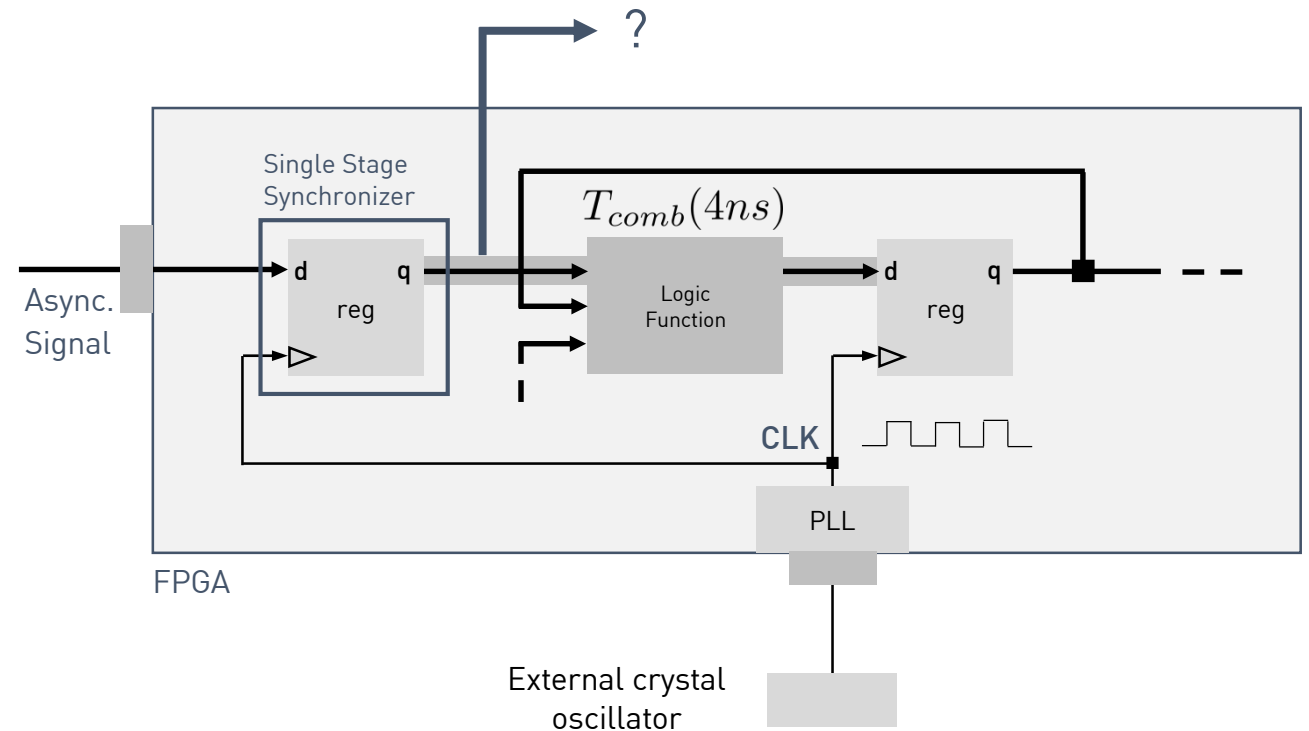
Single Stage Synchronizer ...



Metastability and Synchronization Techniques

Single Stage Synchronizer ...

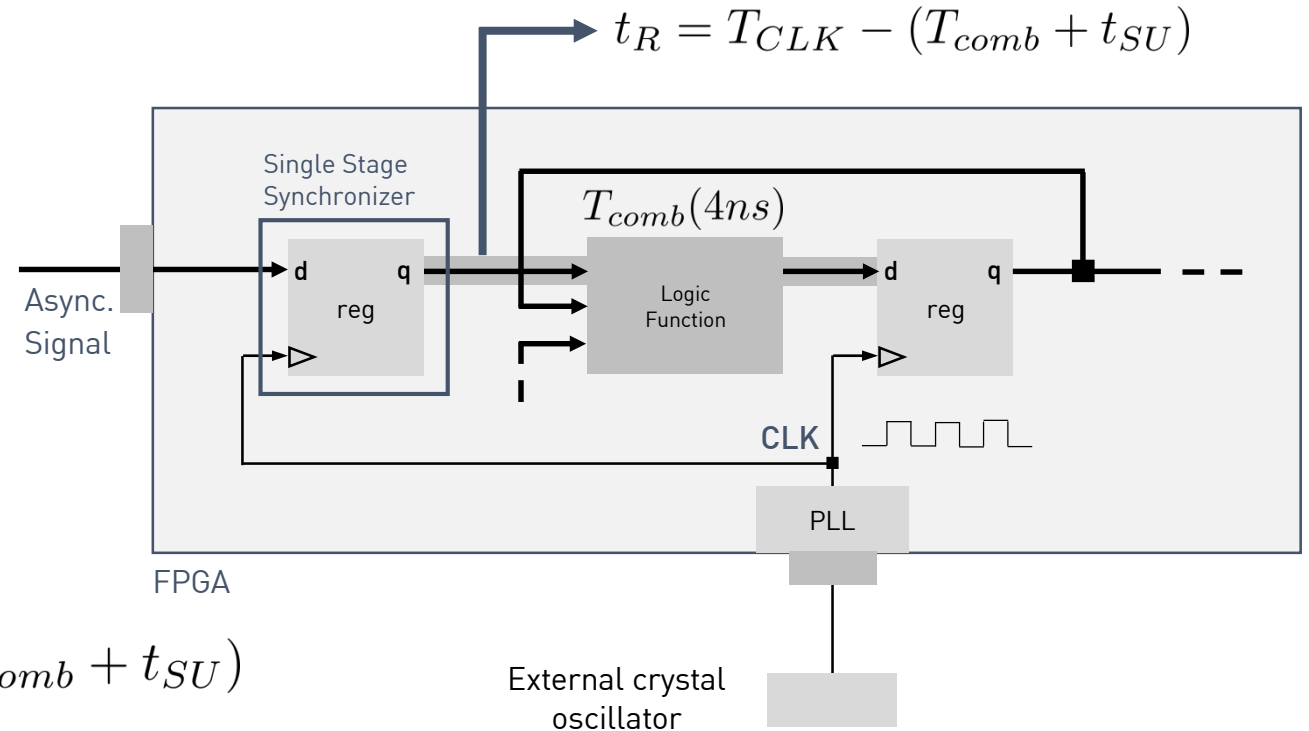
- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: ?



Metastability and Synchronization Techniques

Single Stage Synchronizer ...

- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: $t_R = T_{CLK} - (T_{comb} + t_{SU})$

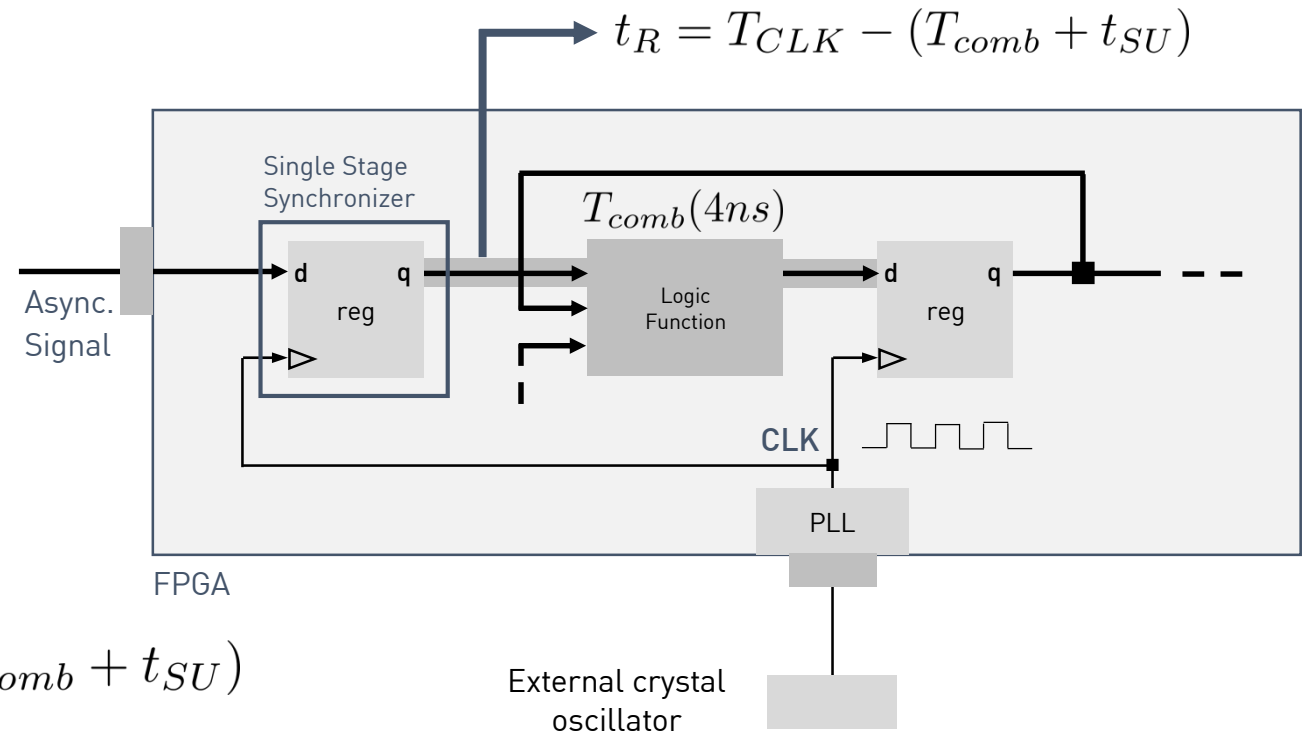


Metastability and Synchronization Techniques

Single Stage Synchronizer ...

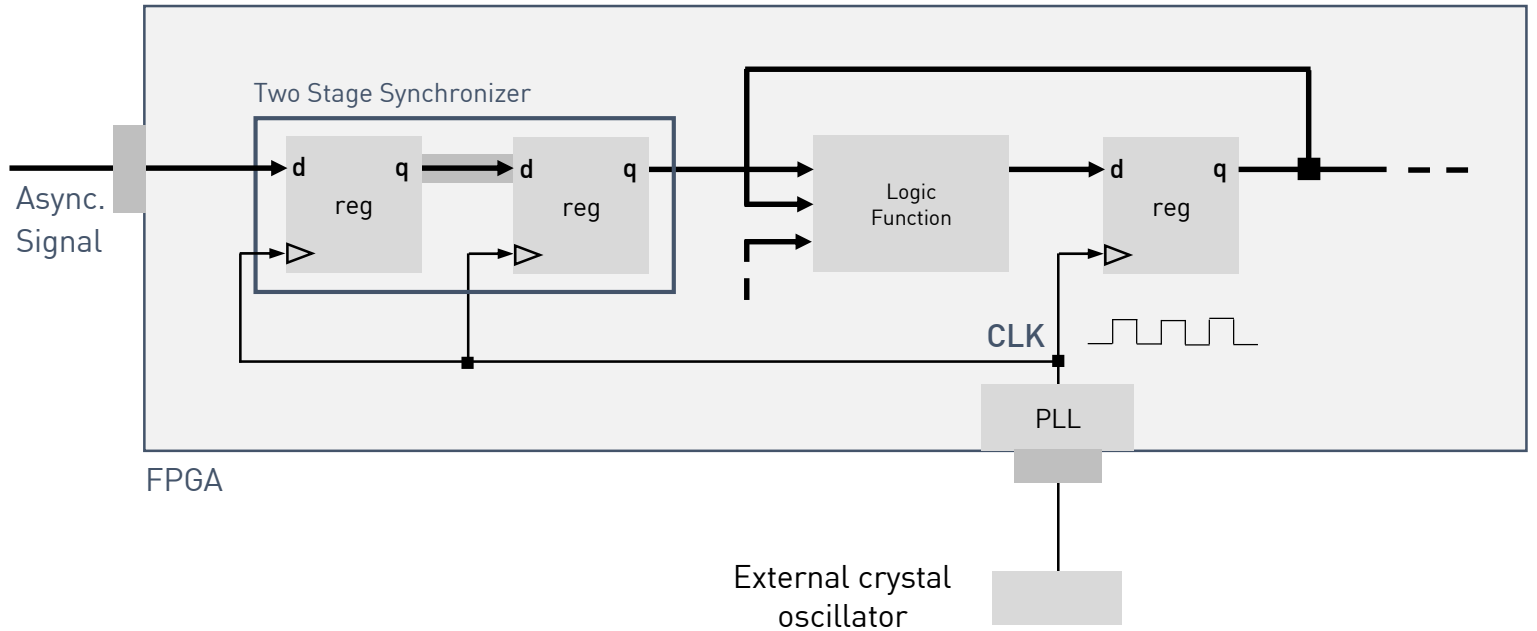
- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: $t_R = T_{CLK} - (T_{comb} + t_{SU})$
- MTBF (calculated): 31 days ...

Not reliable as well ...



Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design



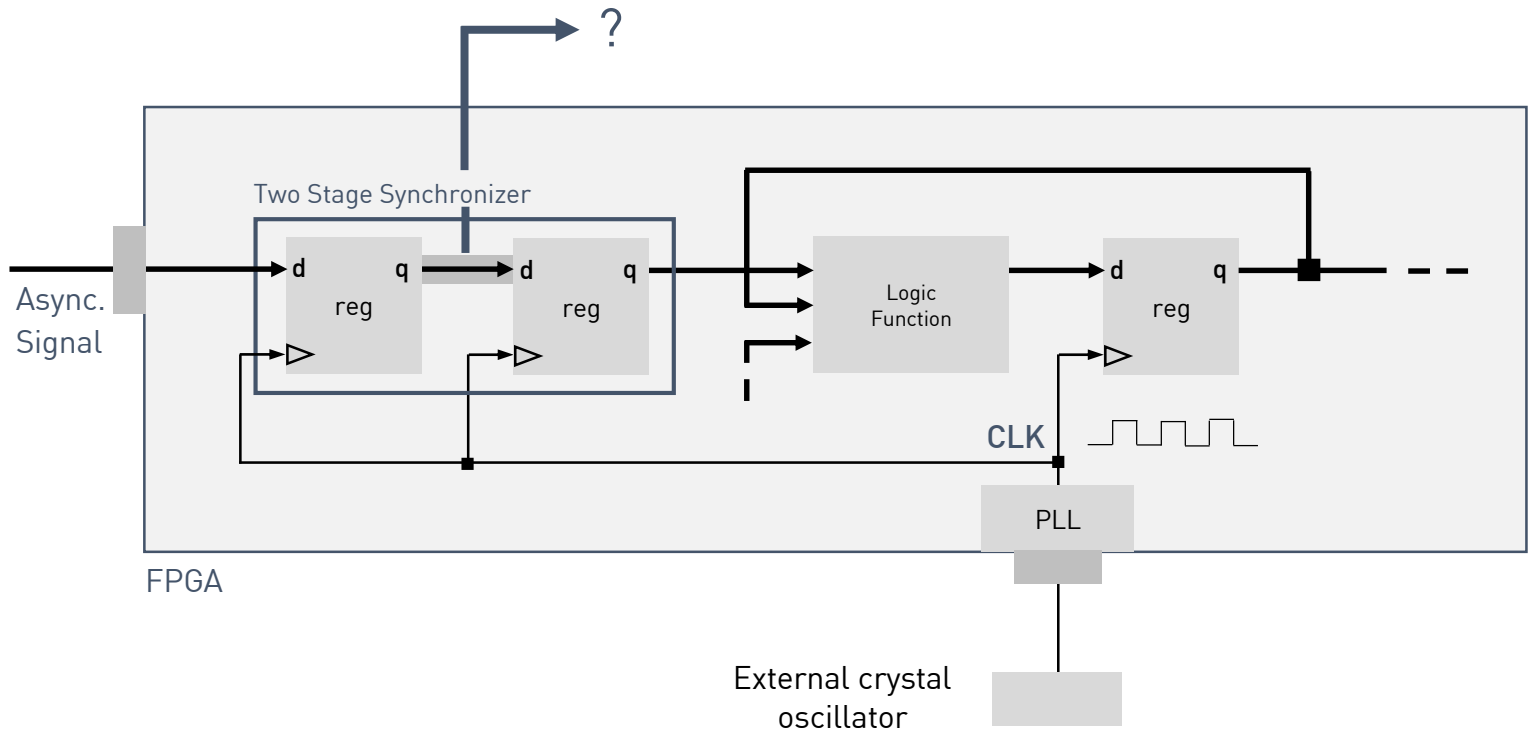
Scenarion #3

Two Stage Synchronizer ...

Metastability and Synchronization Techniques

Two Stage Synchronizer ...

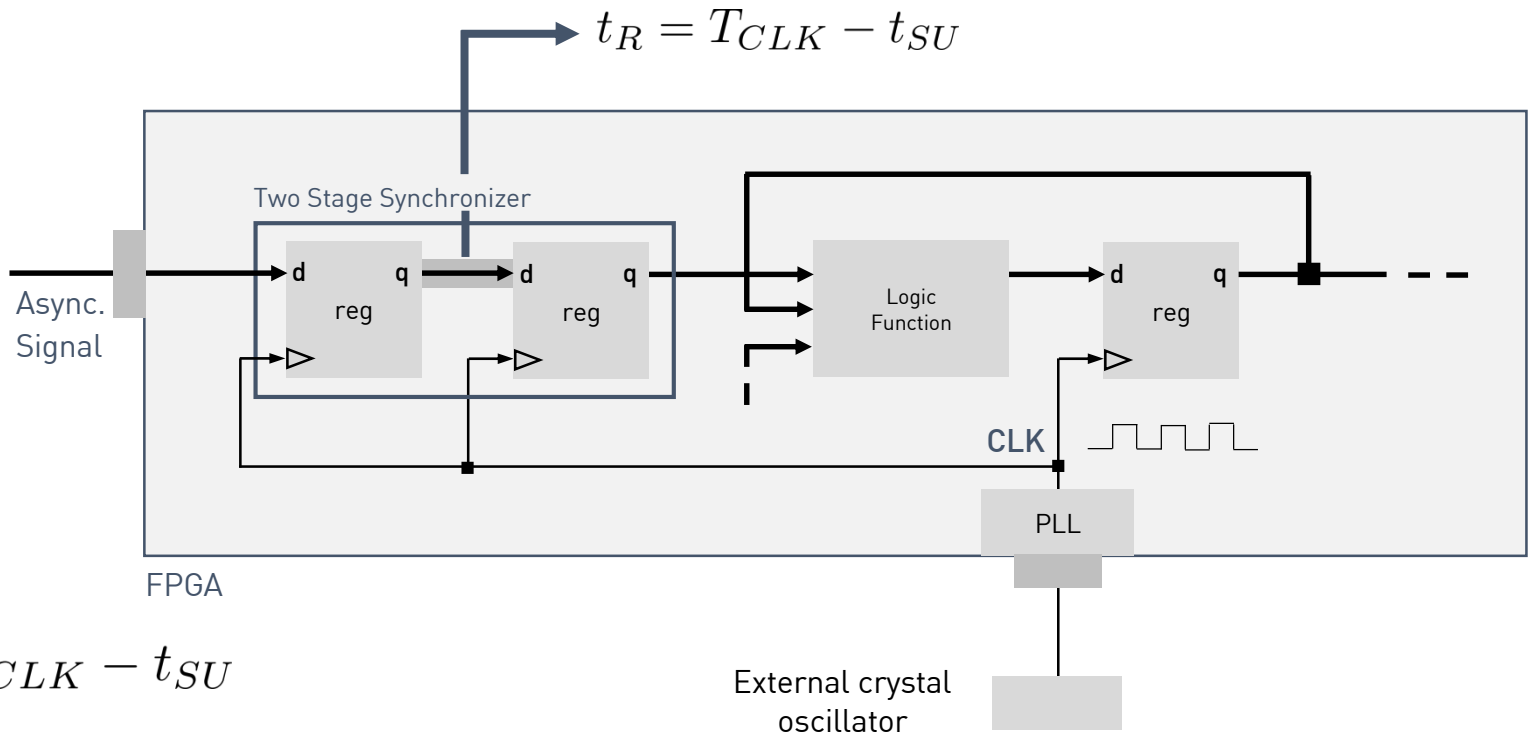
- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time:



Metastability and Synchronization Techniques

Two Stage Synchronizer ...

- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: $t_R = T_{CLK} - t_{SU}$

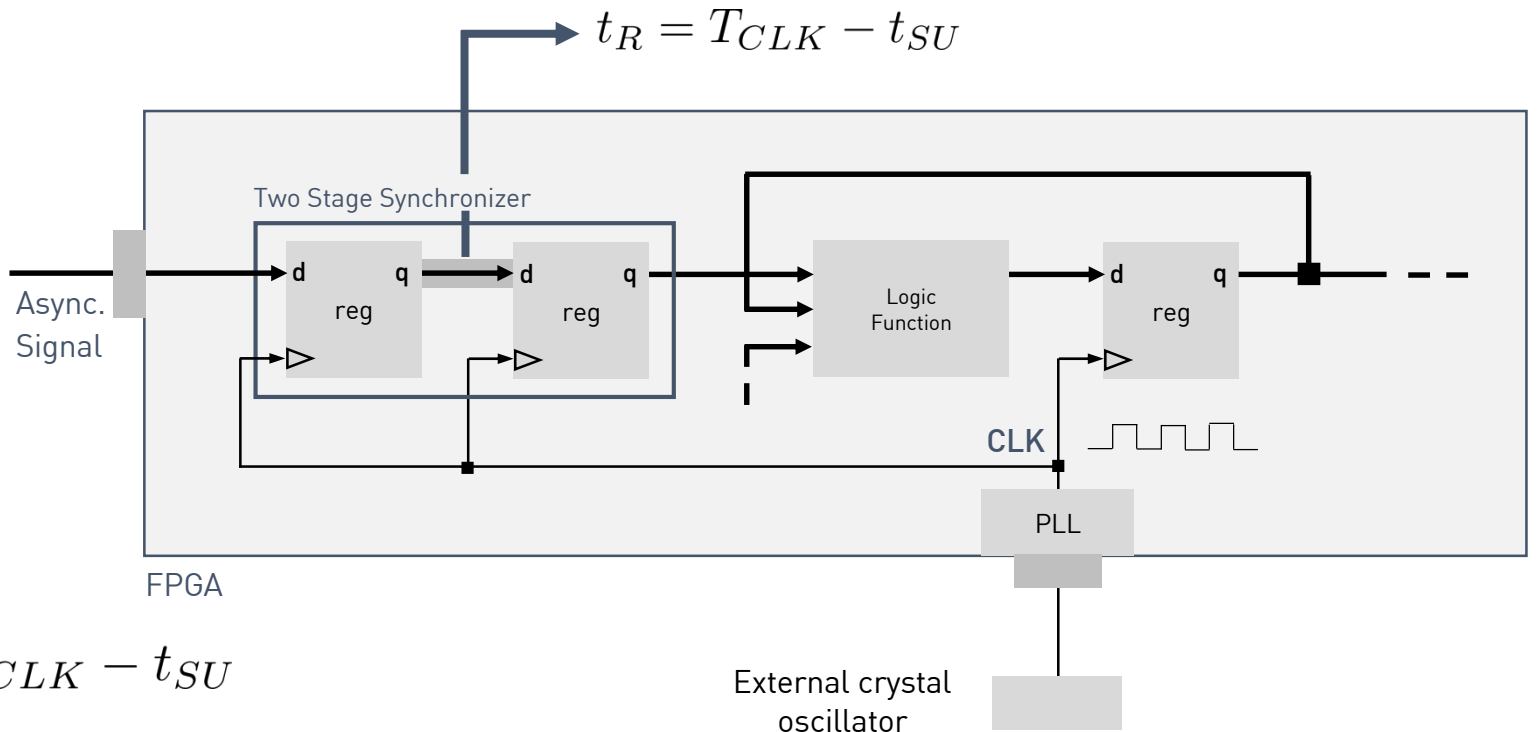


Metastability and Synchronization Techniques

Two Stage Synchronizer ...

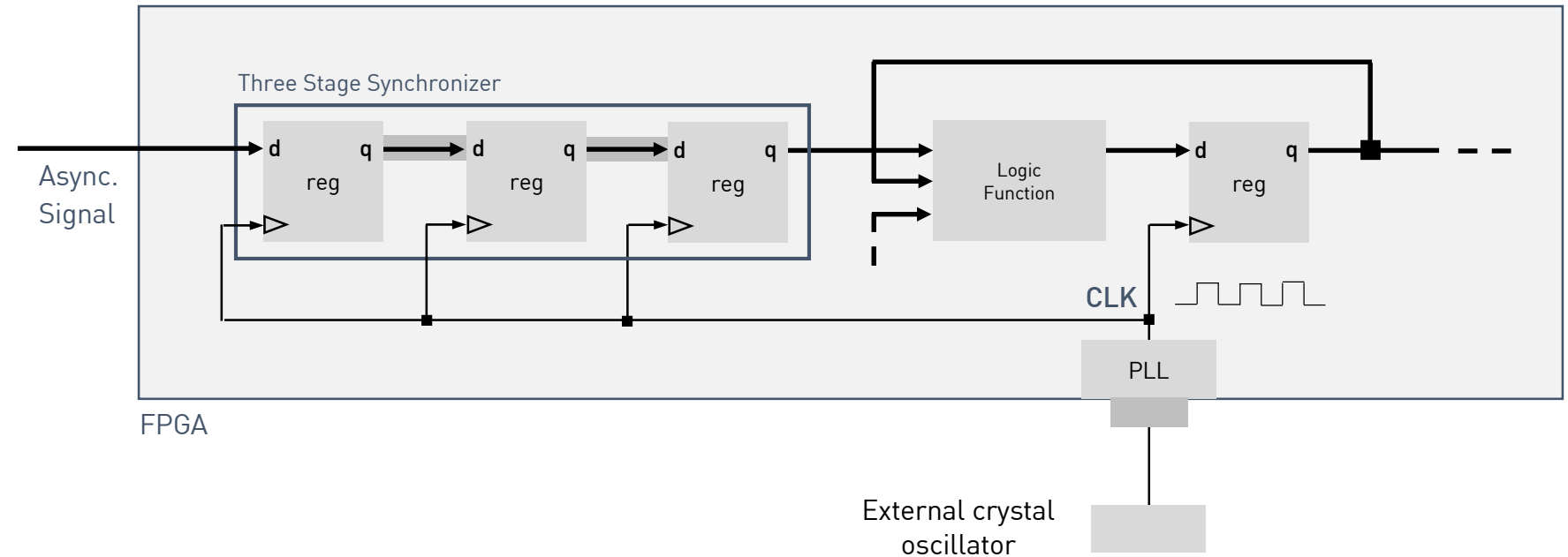
- Metastability window t_{mw} : 66ps
- Time constant τ : 33ps
- System frequency f_{clk} : 200MHz
- Data rate f_{data} : 20MHz
- Setup Time t_{SU} : 100ps
- Resolution time: $t_R = T_{CLK} - t_{SU}$
- MTBF (calculated): 3.7×10^{51} years

Most commonly used sync. architecture



Interfacing FPGA external Components

h_da – fbeit – FPGA – Computer Design Architectures – SS2019



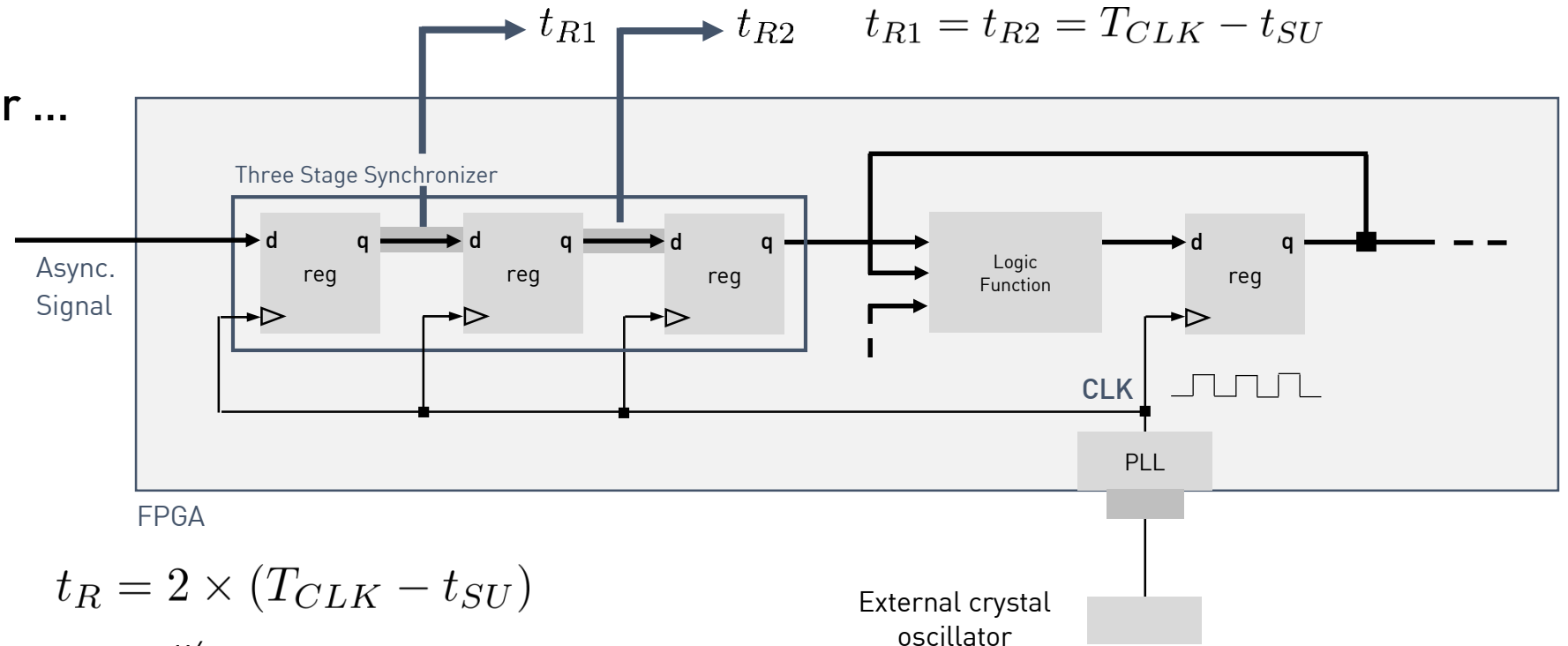
Scenarion #4

Three Stage Synchronizer ...

Metastability and Synchronization Techniques

Three Stage Synchronizer ...

- Same data as before ...



- Resolution time: $t_R = 2 \times (T_{CLK} - t_{SU})$
- MTBF (calculated): 1.1×10^{116} years

Hardly needed ...

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Conclusion so far ...

Metastability and Synchronization Techniques

Notes

- Metastability is basically an analog phenomena
- No physical circuitry can prevent metastability
- ...

Conclusion so far ... Contd.

Metastability and Synchronization Techniques

Notes

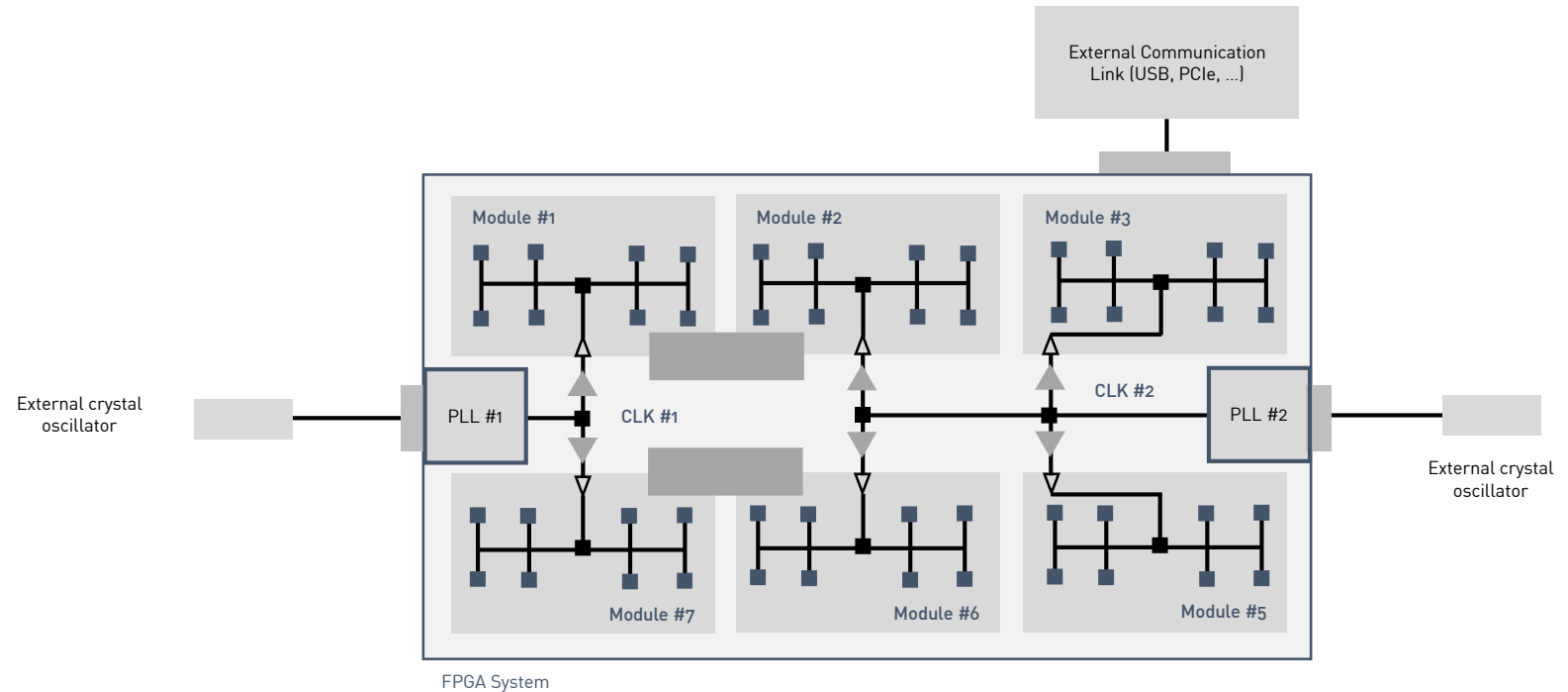
- No Magic involved, synchronizer just provide enough time for the metastable state to resolve ...

Potential disadvantage

- By the use of synchronization stages, we are **increasing the latency** in the synchronous logic observation path. However, this doesn't represent a serious problem in most applications ...

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design



Clock domain crossing of a single-bit CTRL signals (enable/start/...)

FPGA based System Design

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

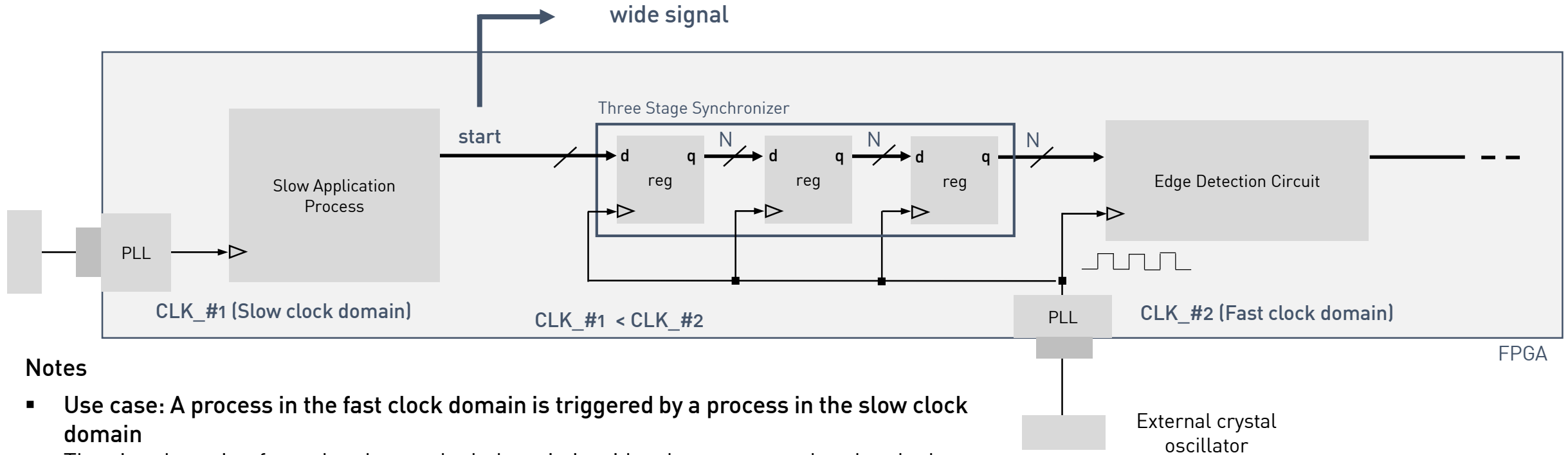
Scenario #1: Slow to fast clock domain crossing

FPGA based System Design

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Slow to fast clock domain crossing



Notes

- Use case: A process in the fast clock domain is triggered by a process in the slow clock domain
- The signal coming from the slower clock domain is wide when compared to the clock period of the fast clock domain!
- Use an edge detection scheme (see last lecture) to properly issue a start pulse in the fast clock domain ...

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

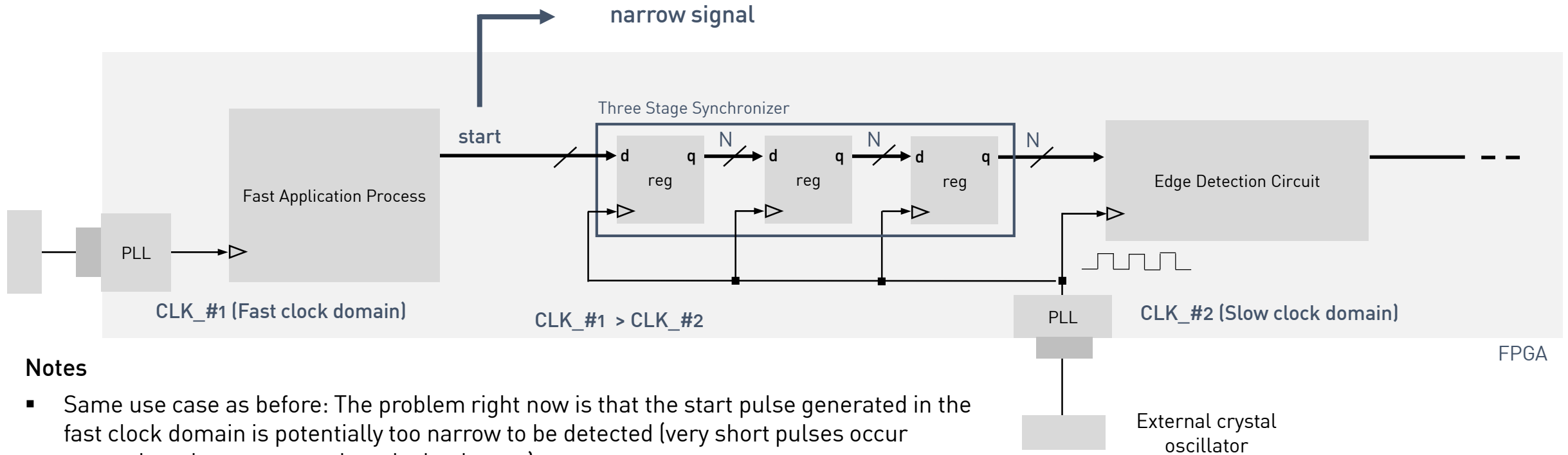
Scenario #2: Fast to slow clock domain crossing

FPGA based System Design

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Fast to slow clock domain crossing



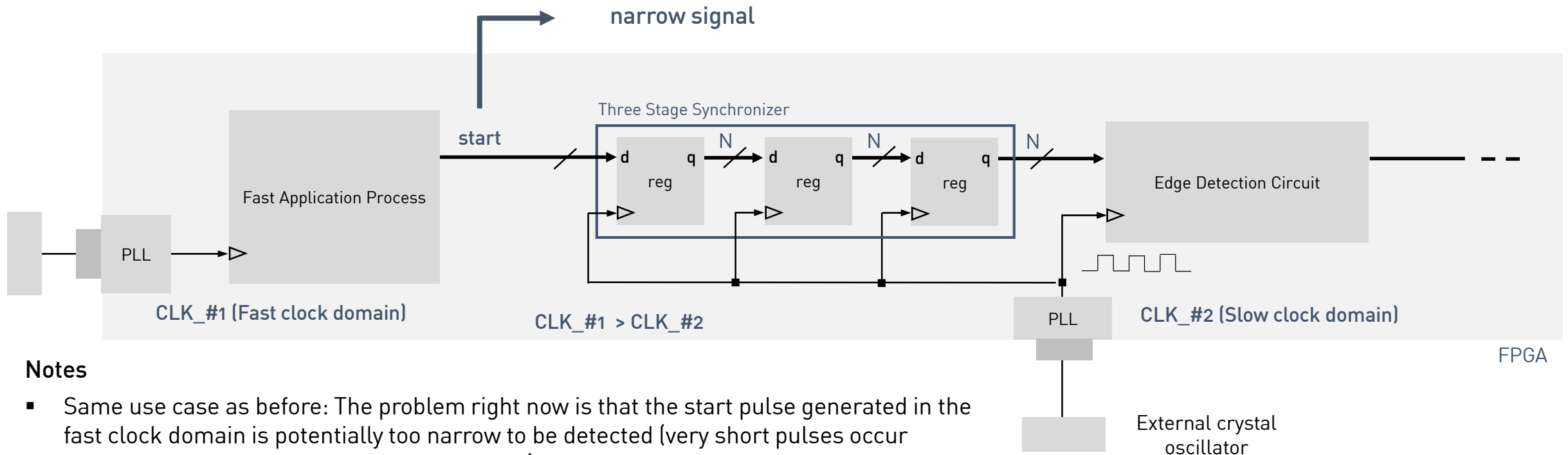
Notes

- Same use case as before: The problem right now is that the start pulse generated in the fast clock domain is potentially too narrow to be detected (very short pulses occur somewhere between two slow clock edges ...)

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Fast to slow clock domain crossing



Notes

- Same use case as before: The problem right now is that the start pulse generated in the fast clock domain is potentially too narrow to be detected (very short pulses occur somewhere between two slow clock edges ...)
- An additional circuitry in the fast clock domain is necessary to **stretch the start pulse** so that it is properly detected in the slow clock domain

Interfacing FPGA external Components

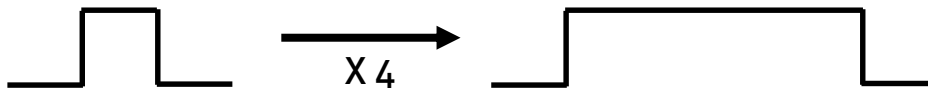
h_da – fbeit - FPGA-based SoC Design

Pulse Stretching in SystemVerilog ...

```
1. module pulse_stretcher #parameter (FACTOR = 4) (  
2.   input wire clk, input wire reset_n, input wire d,  
3.   output wire q_pulse_stretched  
4. );  
5. logic [FACTOR-1:0] sync_reg = 0;  
6. always_ff@(posedge clk)  
7.   if(reset_n == 1'b0)  
8.     sync_reg <= 0;  
9.   else  
10.    sync_reg <= {sync_reg[FACTOR-2:0], (d | sync_reg[0]) & (~sync_reg[FACTOR-1])};  
11.  
12. assign q_pulse_stretched = sync_reg[0];  
13.  
14. endmodule
```

1/1

[SystemVerilog] Source Code: pulse_stretcher.sv



Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Corresponding SystemVerilog test bench

```
1. `timescale 1ns/1ps
2. `define HALF_CLOCK_PERIOD      10
3. `define RESET_PERIOD           40
4. `define SIM_DURATION           5000000
5.
6. module pulse_stretcher_tb();
7.
8.     wire tb_q_pulse_stretched;
9.     // ### test clock generation ...
10.    reg tb_local_clock = 0;
11.    initial
12.        begin: clock_generation_process
13.            tb_local_clock = 0;
14.            forever
15.                #`HALF_CLOCK_PERIOD tb_local_clock = ~tb_local_clock;
16.        end
17.
```

1/1

[SystemVerilog] Source Code: pulse_stretcher_tb.sv Part #1

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Corresponding SystemVerilog test bench

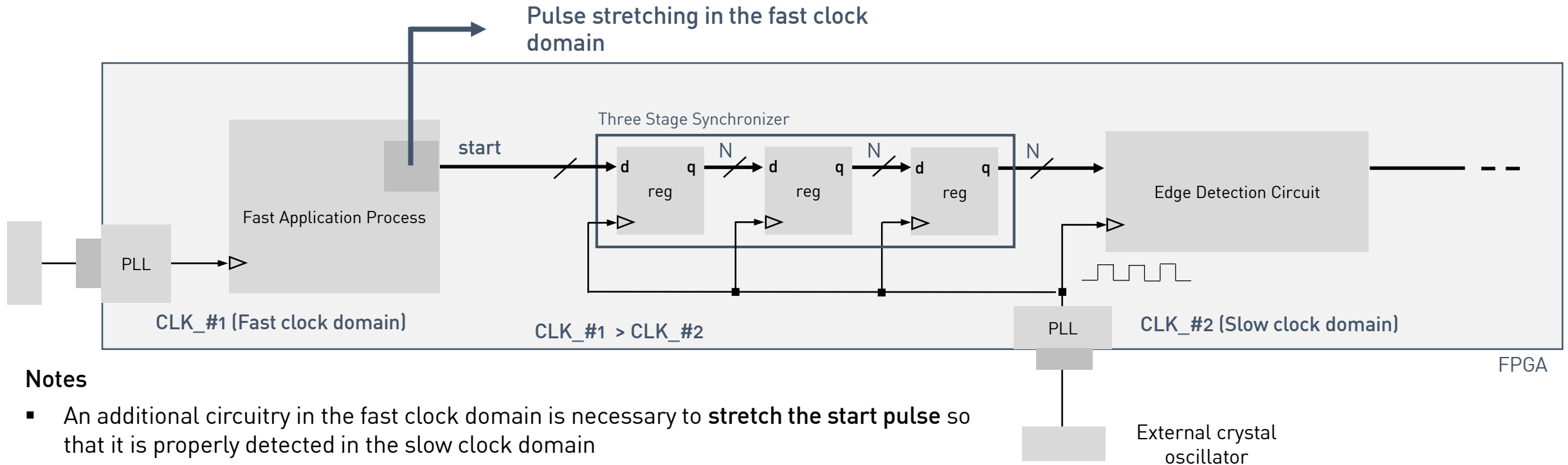
```
18. reg tb_local_reset_n = 0;
19. initial begin: sim_control_process
20.     $display ("Simulation starts ...");
21.     #`RESET_PERIOD tb_local_reset_n = 1'b1;
22.     #`SIM_DURATION
23.     $display ("Simulation done ...");
24.     $stop();
25. end
26.
27. reg [4:0] test_count = 0;
28. always@(posedge tb_local_clock) test_count = test_count + 1;
29. wire tb_pulse = &test_count;
30. pulse_stretcher inst_0 #(.FACTOR(4)) ( .clk(tb_local_clock),
31.                                         .reset_n(tb_local_reset_n),
32.                                         .d_pulse(tb_pulse),
33.                                         .q_pulse_stretched(tb_q_pulse_stretched)
34.                                         );
35. endmodule
```

[SystemVerilog] Source Code: pulse_stretcher_tb.sv Part #2

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Fast to slow clock domain crossing



Notes

- An additional circuitry in the fast clock domain is necessary to **stretch the start pulse** so that it is properly detected in the slow clock domain

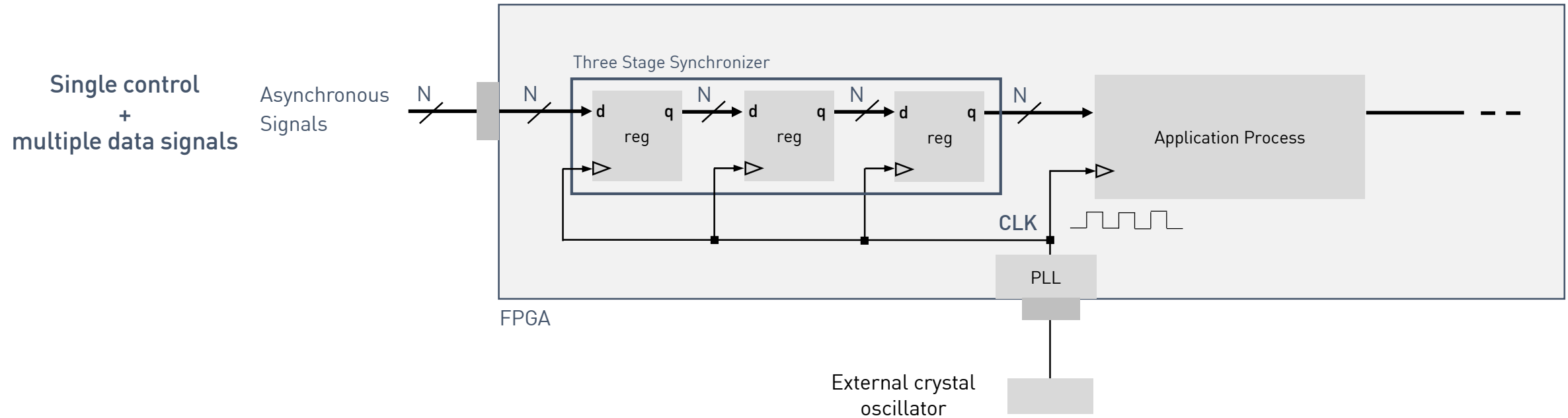
Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

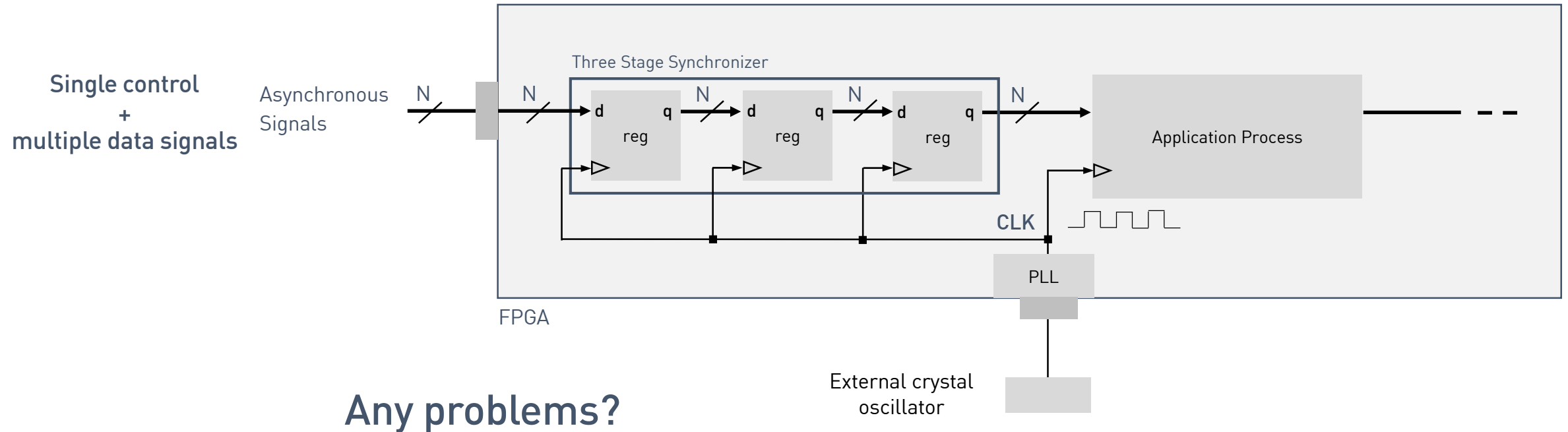
Clock domain crossing of a multi-bit signals ...

FPGA based System Design

What about Multi-Bit Synchronizers?



What about Multi-Bit Synchronizers?



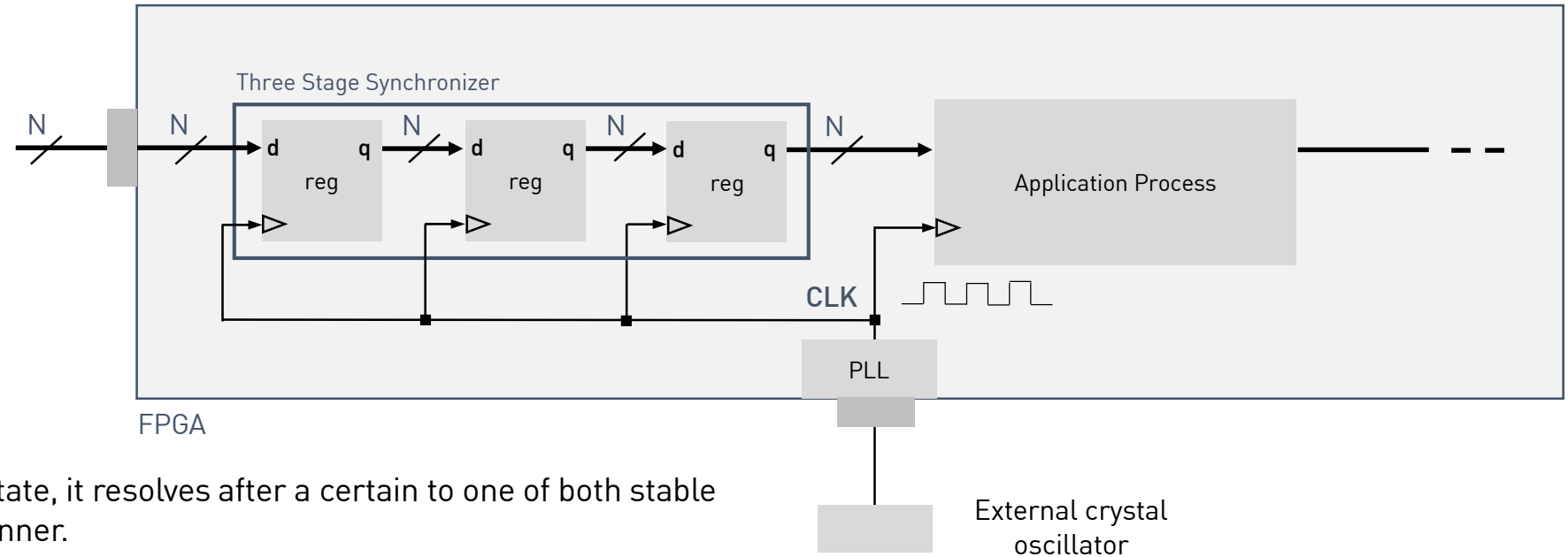
Any problems?

What about Multi-Bit Synchronizers?

Single control
+
multiple data signals

Asynchronous
Signals

Don't do this!



Notes

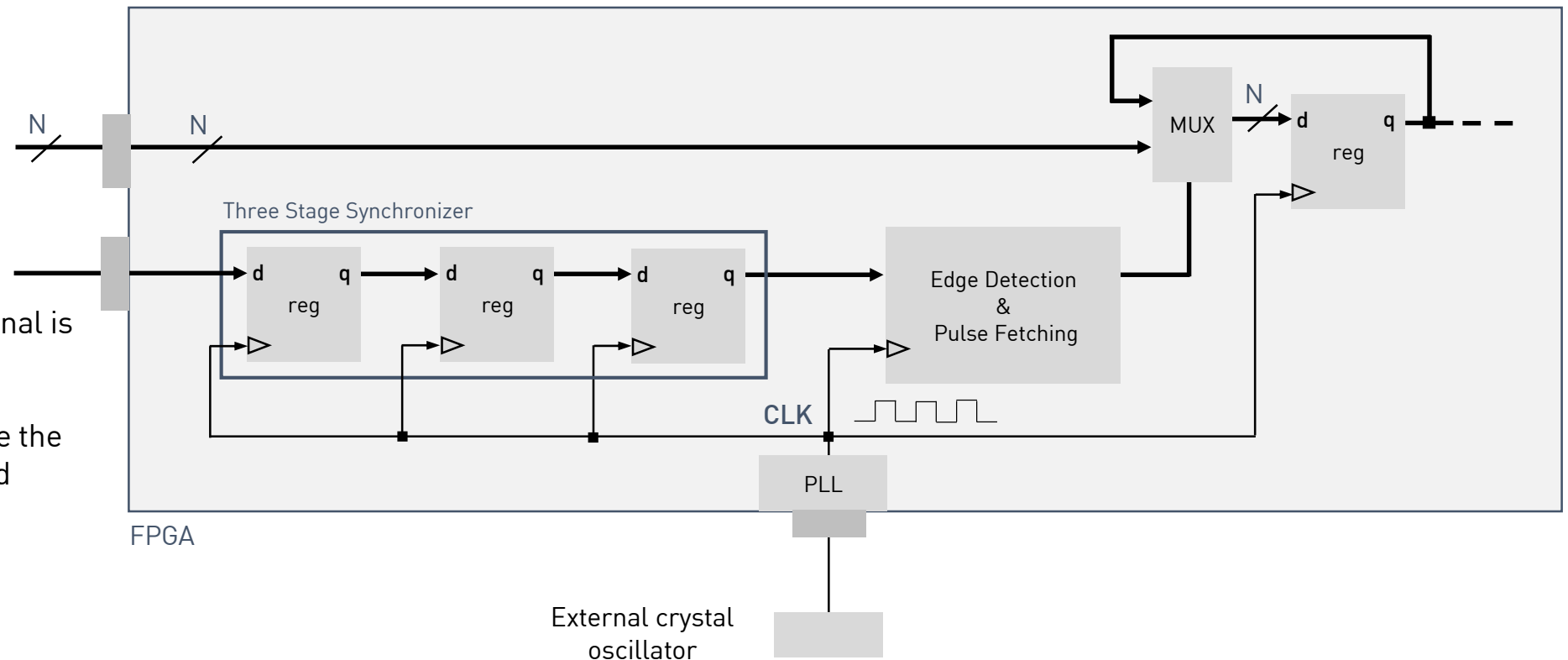
- If a flip-flop enters a metastable state, it resolves after a certain time to one of both stable states in a completely random manner.
- Moreover, some bits may be sampled correctly while some other may get sampled incorrectly because of metastable events ...
- Therefore, in case of a multi-bit synchronizer, data may get corrupted!

Slow to fast clock domain crossing

Reliable multi-bit data transfer between two clock domains

Notes

- Only the single bit control signal is synchronized
- The multi-bit data signal has enough time to stabilize while the control signal is synchronized



Reliable multi-bit data transfer between two clock domains using handshakes ...

Metastability and Synchronization Techniques

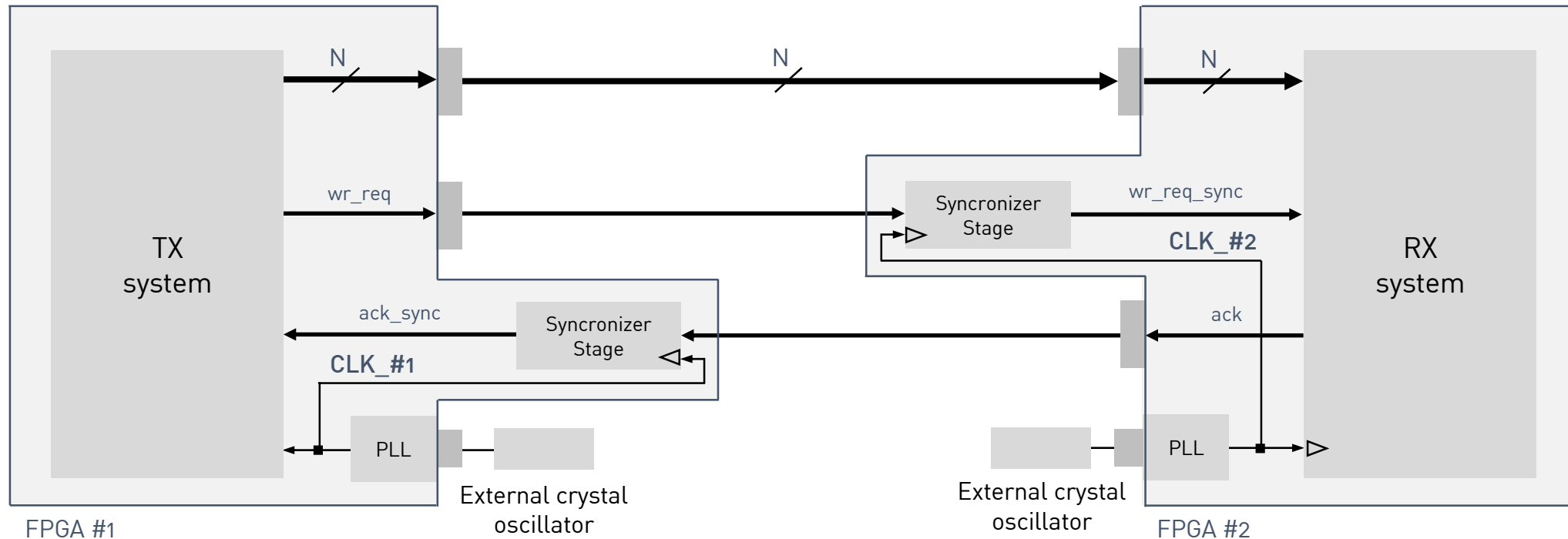
Notes

- How to know that new data is available for reading?
- How to know that the receiving instance is ready to receive new data?
- Use a handshaking scheme ...

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Reliable multi-bit data transfer between two clock domains using handshakes ...



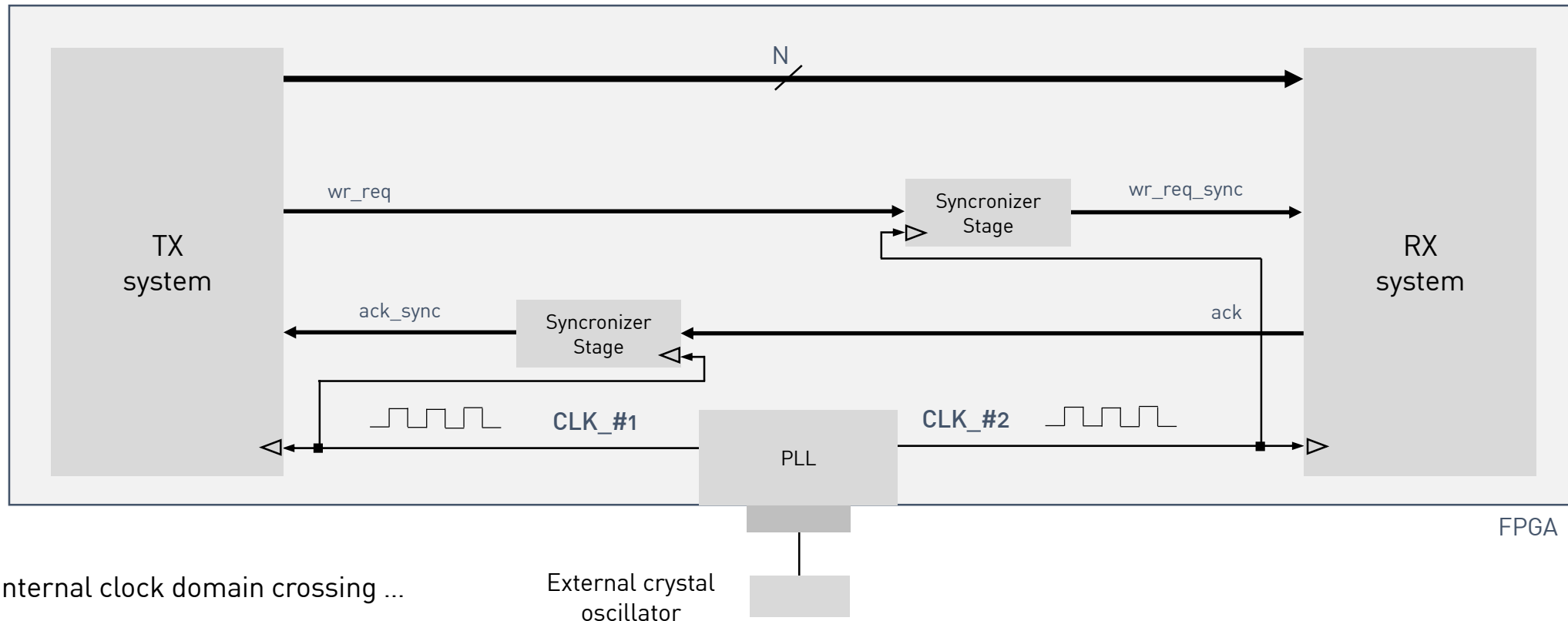
Notes

- Only the single-bit control signals are synchronized. We never use synchronizers for multi-bit data words ...
- We can be sure that when a request is detected by the rx system, data has already stabilized. If data is stable, it cannot violate the flip-flop timings

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Reliable multi-bit data transfer between two clock domains using handshakes ...



Notes

- FPGA internal clock domain crossing ...

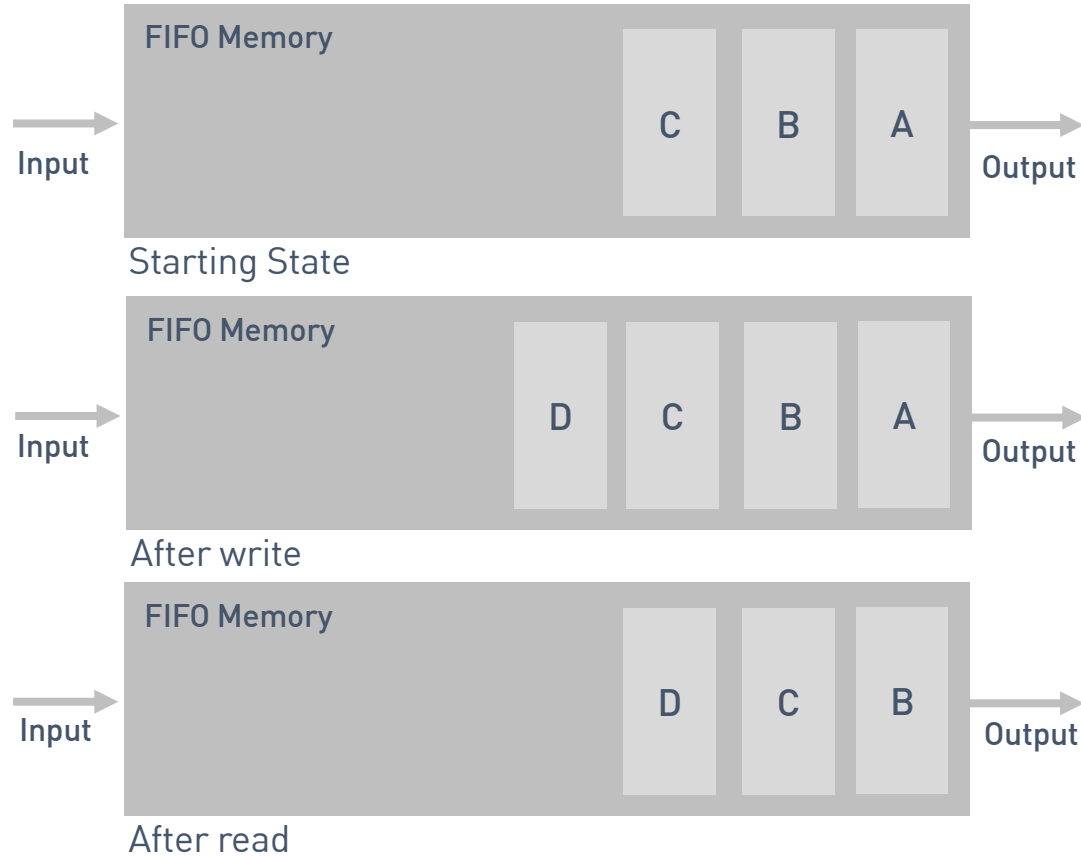
Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Design Example: Dual clock First-In-First-Out (FIFO) Memory

FPGA based System Design

Dual clock First-In-First-Out (FIFO) Memory

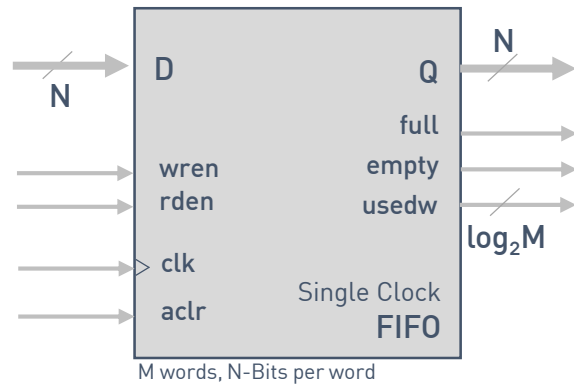


FIFO Memories

- Used to implement queues
- Used to decouple actions of **producer** and **consumer**
- The producing side can perform many writes without that the consumer performs a single read (or vice versa). However, because of the finite buffer size, we need, on average, an equal number of reads and writes.

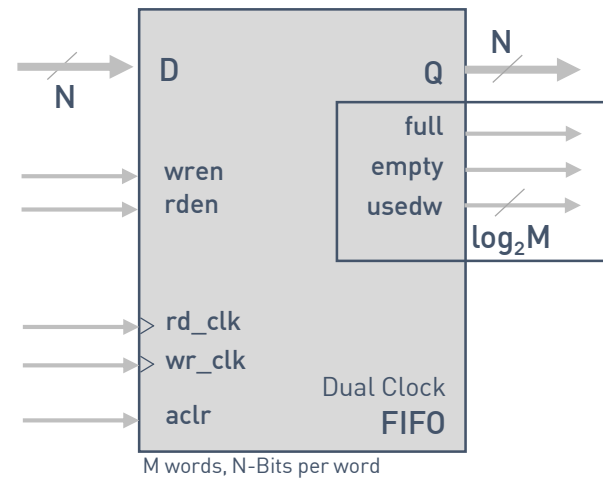
Dual clock First-In-First-Out (FIFO) Memory

Single Clock FIFO



- Common clock for reading and writing to the FIFO

Dual Clock FIFO



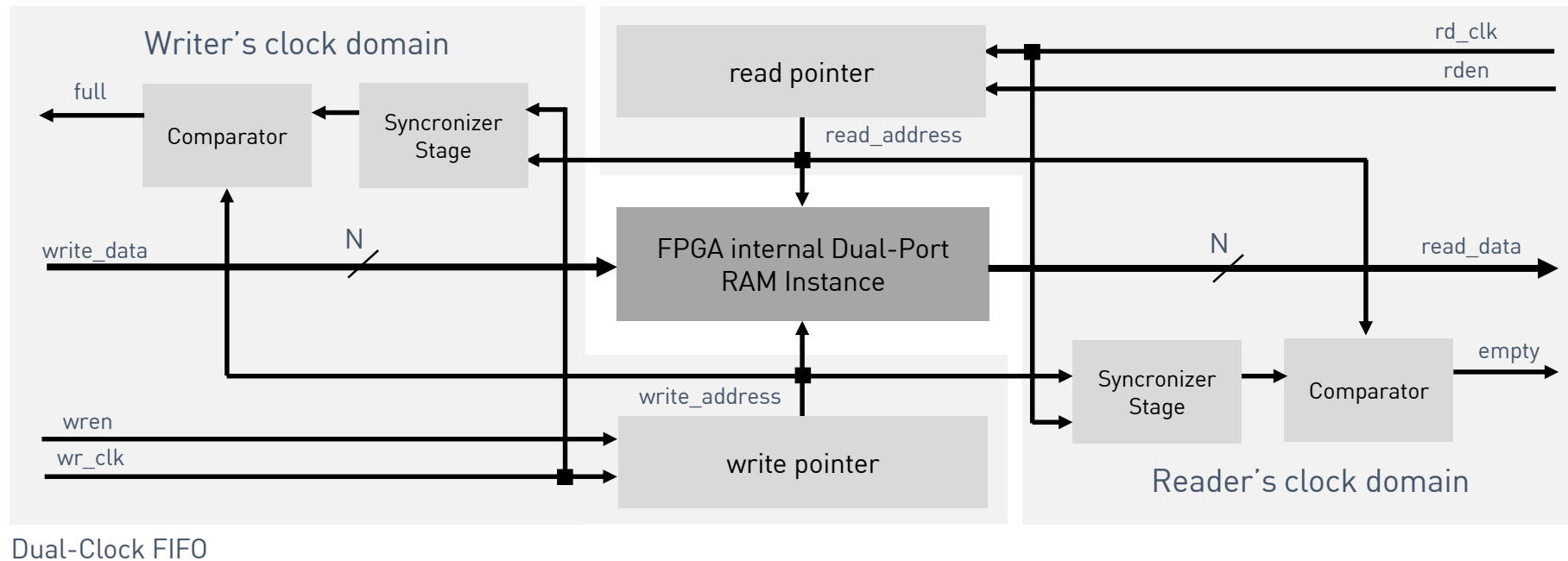
These signals are used by another unit to control reading and writing to/from the FIFO

- Writing is synchronized to the write clock, reading is synchronized to the read clock
- The FIFO status signals full, empty and usedw can be either synchronized to the read or the write clock.
- Most FIFOs support show-ahead synchronous modes. Here, the data becomes available before rden is asserted.

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Dual clock First-In-First-Out (FIFO) Memory



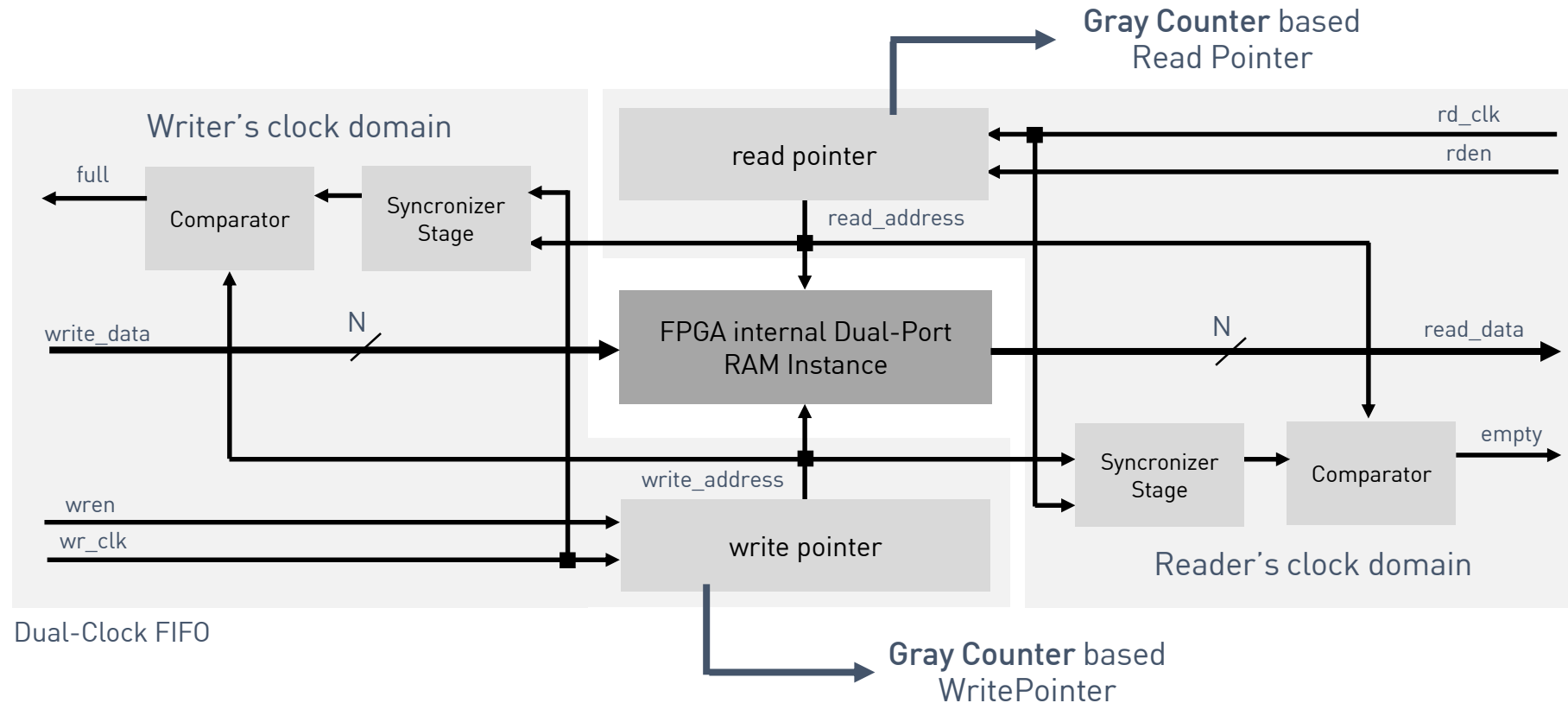
Notes

- Why do we synchronize the multi-bit write/read pointer ???

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Dual clock First-In-First-Out (FIFO) Memory



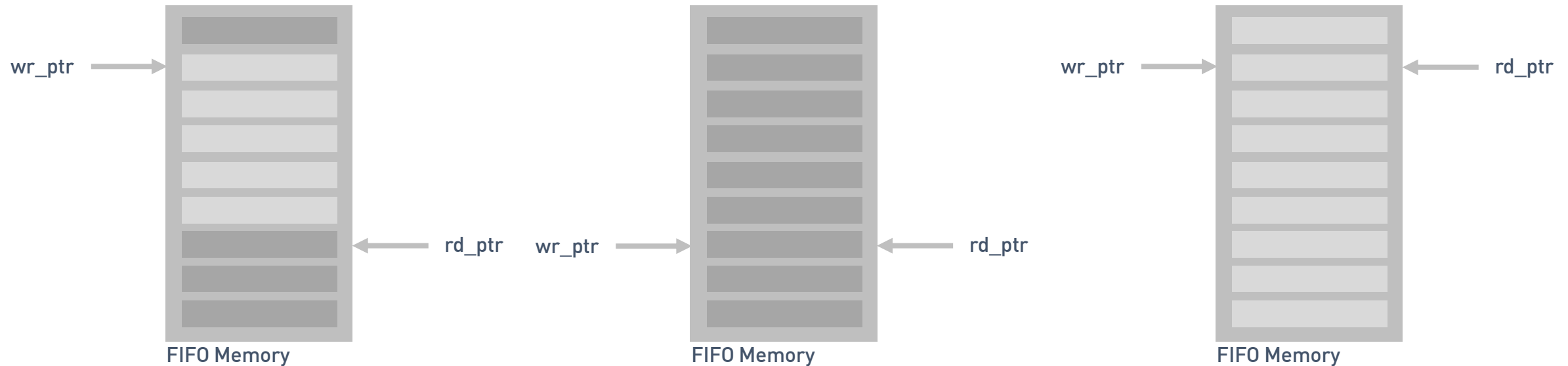
4-bit gray code

▪ 0:	0000
▪ 1:	0001
▪ 2:	0011
▪ 3:	0010
▪ 4:	0110
▪ 5:	0111
▪ 6:	0101
▪ 7:	0100
▪ 8:	1100
▪ 9:	1101
▪ 10:	1111
▪ 11:	1110
▪ 12:	1010
▪ 13:	1011
▪ 14:	1001
▪ 15:	1000

Notes

- Pointers use a gray code scheme, this means that only a single bit changes in case of state transition
- Therefore, it is uncritical to synchronize the multi-bit pointer ...

Dual clock First-In-First-Out (FIFO) Memory

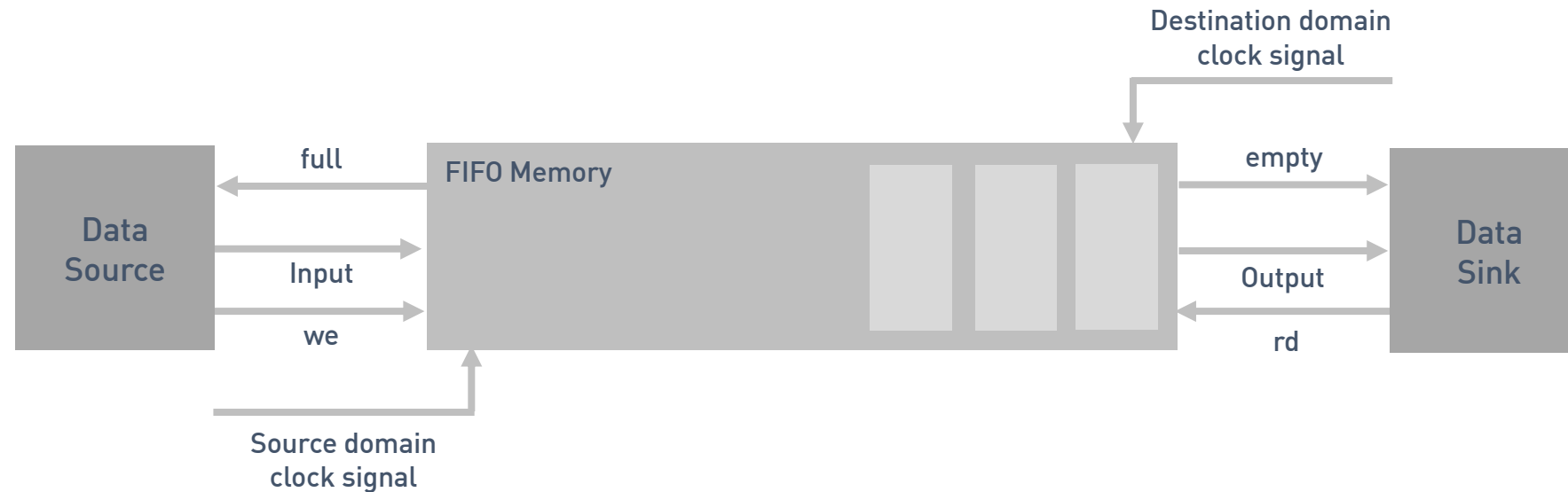


- Address pointers are used internally to keep the next write and read position within the dual-port memory
- If both pointers have the same values after a write operation, the FIFO is full
- If both pointers have the same values after a read operation, the FIFO is empty

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Dual clock First-In-First-Out (FIFO) Memory



Notes

- This dual clock FIFO is designed as a way for two circuits operating in different clock frequencies to communicate with each other.

Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

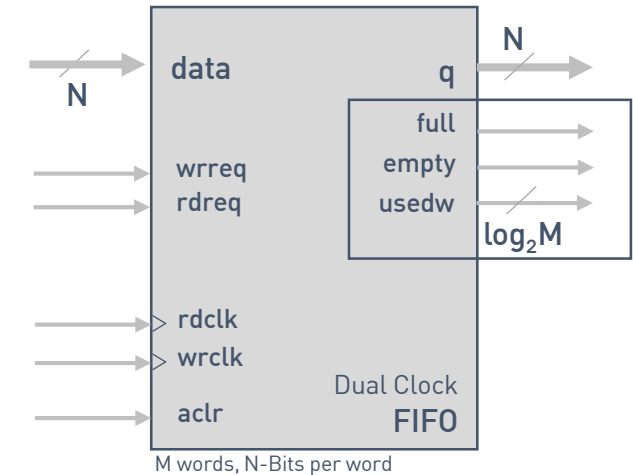
Dual clock First-In-First-Out (FIFO) Memory in SystemVerilog

```
1.  
2. // instantiating the dc fifo ...  
3. dc_8bit_256_fifo inst_0 ( .data(data_sig),  
4.                             .rdclk(rdclk_sig),  
5.                             .rdreq(rdreq_sig),  
6.                             .wrclk(wrclk_sig),  
7.                             .wrreq(wrreq_sig),  
8.                             .q(q_sig),  
9.                             .usedw(usedw_sig),  
10.                            .rdempty(rdempty_sig ),  
11.                            .wrfull(wrfull_sig )  
12.                            );
```

[Verilog] Source Code Excerpt: Verilog design using an FPGA internal dual clock FIFO

Notes

- This dual clock FIFO is designed using the Intel Quartus Prime IP catalog: Open Quartus and select **Tools | IP Catalog** from the tool bar



Interfacing FPGA external Components

h_da – fbeit - FPGA-based SoC Design

Summary

Metastability and Synchronization Techniques

Notes

- Metastability cannot be avoided at the boundary between two systems that are asynchronous with respect to each other
- However, the probability that metastable states are encountered can be significantly reduced by
 - the use of synchronizers
 - the use of fast flip-flops (narrow metastable window ...)
 - reducing the sampling rate
 - Avoiding input signals with low dV/dt ...